

A No-nonsense Introduction to Artificial Intelligence

v0.1

David Hutasoit

14 September 2026

Abstract

Artificial Intelligence (AI) dominates contemporary discourse, often obscured by hype, science fiction, and technical jargon. This article offers a grounded, no-nonsense introduction to modern AI for a broad audience. No prior expertise is assumed. Balancing conceptual clarity with practical relevance, it examines the foundations of machine intelligence, demystifies the technical mechanics behind today's systems, and candidly acknowledges the limits of current theoretical understanding. A companion notebook reproduces every experiment, from a simple logistic regression to a character-level language model, and runs on a laptop CPU in under ten minutes. It is available at https://github.com/davidteaching/no-nonsense_intro_ai/. Drawing on insights from mathematics, physics, and computer science, it explores ethical dimensions, governance challenges, and future trajectories. Rather than predicting or promoting AI, the article aims to equip readers to evaluate claims and participate thoughtfully in shaping this transformative technology.

Contents

I	Foundations	5
1	Introduction: Why Another AI Introduction?	5
1.1	The hype cycle and the need for grounded exposition	5
1.2	What this article is and is not	5
1.3	Audience and reading paths	6
1.4	Roadmap and key themes	6
2	Defining AI: Terms, Scope, and Misconceptions	7
2.1	AI, machine learning, deep learning, generative AI	7
2.2	Narrow vs. general vs. superintelligence	8
2.3	Symbolic versus connectionist approaches	9
2.4	What AI is not: consciousness, magic, autonomous agency	9
2.5	A working definition	10
3	A Brief History of AI: From Logic to Large Models	11
3.1	Cybernetics, Turing, and the Dartmouth workshop	11
3.2	Symbolic AI and expert systems	12
3.3	AI winters and revival	12
3.4	Statistical learning and the data-driven turn	12
3.5	Deep learning, transformers, and foundation models	13

3.6	Lessons from history	13
4	Mathematical and Conceptual Foundations	14
4.1	Linear algebra: vectors, matrices, embeddings	14
4.2	Probability and statistics: uncertainty, Bayes, distributions	15
4.3	Calculus and optimization: gradients, loss surfaces	16
4.4	Information theory: entropy, KL divergence, compression	16
4.5	Computation and complexity: what is tractable?	17
4.6	No free lunch and inductive biases	18
5	How Machine Learning Works	18
5.1	Data, features, labels, and tasks	18
5.2	Four paradigms of learning	19
5.3	Training as optimization	20
5.4	Generalization and overfitting	22
5.5	Evaluation and its pitfalls	23
5.6	A minimal worked example	25
II	Mechanics and Practice	27
6	Core Model Families	28
6.1	Linear and logistic regression	28
6.2	Decision trees, forests, and boosting	28
6.3	Support vector machines and kernels	29
6.4	Neural networks: from perceptrons to MLPs	29
6.5	Convolutional and recurrent networks	30
6.6	Transformers and attention	31
6.7	Generative models	32
6.8	A summary view	33
7	The Mechanics of Modern AI Systems	34
7.1	Backpropagation and gradient descent	34
7.2	Embeddings and representation learning	34
7.3	Attention and sequence modeling	35
7.4	Pretraining, fine-tuning, and alignment	36
7.5	Scaling laws and emergent behaviour	37
7.6	Inference, prompting, retrieval, and agents	37
7.7	Hardware and compute economics	38
8	From Prototype to Practice: The AI Lifecycle	39
8.1	Problem framing and feasibility	39
8.2	Data collection, cleaning, and governance	40
8.3	Baselines and iteration	41
8.4	Deployment, monitoring, and maintenance	41
8.5	MLOps, reproducibility, and security	41
8.6	Case studies	42

III	Limits, Context, and Future	45
9	Limits and Open Problems	45
9.1	Theoretical understanding: why does deep learning work?	45
9.2	Robustness, distribution shift, and adversarial examples	46
9.3	Causality and counterfactuals	48
9.4	Reasoning, planning, and common sense	48
9.5	Sample efficiency and continual learning	49
9.6	Interpretability and explainability	49
9.7	Energy, compute, and environmental cost	49
9.8	What we do not know	50
10	Insights from Mathematics and Physics	50
10.1	Statistical mechanics and phase transitions	51
10.2	The strange geometry of high dimensions	51
10.3	Dynamical systems, optimisation, and criticality	52
10.4	Thermodynamics and the physical cost of computation	52
10.5	What physics can and cannot explain about intelligence	53
11	Ethics, Safety, and Governance	53
11.1	Bias, fairness, and discrimination	54
11.2	Privacy, surveillance, and data rights	54
11.3	Misinformation, manipulation, and synthetic media	55
11.4	Labor, economy, and inequality	55
11.5	Autonomy, accountability, and lethal applications	56
11.6	Alignment and existential risk: separating evidence from speculation	56
11.7	Regulation, standards, and global governance	57
11.8	Principles versus practice	57
12	Future Trajectories and Challenges	58
12.1	AGI: definitions, debates, and timelines	59
12.2	Human–AI collaboration and augmentation	59
12.3	AI for science, health, and climate	60
12.4	Education and public understanding	60
12.5	Open research directions	60
12.6	Scenarios: plausible, preferable, avoidable	61
13	Conclusion: Participating in the Trajectory	62
13.1	Summary of key takeaways	62
13.2	How to evaluate AI claims	62
13.3	Call to informed participation	63
IV	The Companion Notebook	64
14	The Notebook	64
14.1	Where to find the notebook	64
14.2	Setup and how to run it	65
14.3	The core loop from scratch	65

14.4	What each experiment demonstrates	66
14.5	The character-level language model	67
14.6	Exercises and extensions	68
A	Mathematical Primer	69
A.1	Notation	69
A.2	Linear algebra	69
A.3	Calculus and optimisation	70
A.4	Probability	70
A.5	Information theory	71
A.6	Useful identities	72
B	Glossary	72
C	Exercises and Discussion Questions	75
C.1	Part I: Foundations	75
C.2	Part II: Mechanics and Practice	76
C.3	Part III: Limits, Context, and Future	76
D	Notes on Sources and Evidence Quality	77
D.1	Three kinds of claim	77
D.2	What this article is not	78
D.3	Evaluating sources	78
D.4	A note on citations	79
D.5	A final caution	79
E	Annotated Bibliography	79

Part I

Foundations

1 Introduction: Why Another AI Introduction?

Artificial intelligence has become the rare technology that is simultaneously overdiscussed and underexplained. Headlines oscillate between utopian promises – cures for disease, personalised education, scientific acceleration – and existential warnings about machines that may one day surpass or replace us. Between these extremes sits a vast, mostly quiet reality: millions of people using statistical systems that classify images, rank search results, translate text, recommend videos, and increasingly write, draw, and code. The gap between the discourse and the reality is not merely a matter of degree. It is a gap in understanding how these systems actually work, what they can and cannot do, and why.

This article is an attempt to close that gap. It is not a textbook, and it is not a popular-science tour of the latest announcements. It is a grounded introduction to the ideas and machinery behind modern AI, written for readers who want to understand the subject rather than simply have opinions about it.

1.1 The hype cycle and the need for grounded exposition

Every generation of AI research has produced its own wave of excitement, followed by disappointment, followed by a quieter period of consolidation. The pattern is old enough to have a name: the “AI winter”, and old enough that serious researchers have learned to be sceptical of both promises and panics. And yet each wave brings a new audience that has not seen the pattern before: journalists, policymakers, students, professionals in adjacent fields, and curious general readers who encounter the topic through headlines and product launches rather than through research papers.

The problem is not that these readers are uninformed. It is that the available material tends to fall into two unsatisfying categories. At one end are technical treatments that assume graduate-level mathematics and treat the reader as a future practitioner. At the other end are popular accounts that explain AI through metaphor – “the brain is a neural network”, “the model learns like a child” – which are often misleading and sometimes simply wrong. Neither category serves the reader who wants to understand what is actually happening inside these systems without either simplifying to the point of distortion or drowning in notation.

This article occupies the space between those two categories. It assumes intellectual seriousness but no specialist background. It uses mathematics where mathematics clarifies, and prose where prose suffices. It is candid about what is known, what is engineering convention, and what remains genuinely open.

1.2 What this article is and is not

This article *is*:

- An explanation of the core ideas behind machine learning, from optimisation and generalisation to the architecture of modern neural networks.
- A companion to a working Python notebook (Section 14) that reproduces every experiment discussed in the text, from a logistic regression classifier written in NumPy to a convolutional network and a character-level language model.

- A candid discussion of limits – theoretical, practical, and ethical – including the many things about modern AI that remain poorly understood even by the people who build it.
- A guide to evaluating claims. By the end, a reader should be able to distinguish a genuine capability from a demo, a benchmark from a deployment, and a speculative forecast from an engineering fact.

This article is *not*:

- A survey of the latest models or a ranking of products.
- A philosophical treatise on consciousness, personhood, or the nature of mind. These are important questions, but they are not the subject here.
- A prediction about when or whether artificial general intelligence will arrive. We discuss the debate, but we do not pretend to resolve it.
- A substitute for hands-on practice. The notebook is not optional garnish; it is where much of the understanding is built.

1.3 Audience and reading paths

The intended reader is a motivated non-specialist: a scientist in another field, an engineer, a physician, a lawyer, a journalist, a graduate student, or simply a curious person willing to think carefully. Some familiarity with basic algebra and the idea of a function is helpful but not required; the mathematical primer in the appendix covers what is needed.

Readers may approach the material in different ways. A conceptual reader can follow the main text and skim the mathematical boxes. A curious reader or a practitioner can work through the notebook alongside the relevant sections. A sceptic can begin with the sections on limits and failure modes, then return to the foundations. The structure is designed to support different paths or reading.

1.4 Roadmap and key themes

The article is organised in three parts. Part I establishes the foundations: what AI is and is not, how the field arrived at its current state, and the mathematical ideas such as linear algebra, probability, optimisation, information – that underpin everything that follows. Part II turns to mechanics and practice: how machine learning actually works, what the major model families are, how modern systems like large language models are built, and what it takes to move from a prototype to a deployed system. Part III confronts limits, context, and the future: what we do not understand, what insights mathematics and physics offer, what ethical and governance questions demand attention, and what plausible trajectories lie ahead.

Three themes run throughout. The first is *grounding*: every claim should be traceable to an experiment, a theorem, or an honest admission of uncertainty. The second is *separation*: established science, engineering practice, and open speculation are three different things, and conflating them is the source of most confusion. The third is *agency*: AI is not a force of nature. It is a set of technologies built by people, under incentives, within institutions, and subject to choices. Understanding it is a prerequisite for shaping it.

A note on the companion notebook. All experiments described in the text are implemented in a single Python notebook designed to run on a laptop CPU in under ten minutes. The running

example is handwritten digit recognition – the MNIST dataset – which is simple enough to be understood at a glance but rich enough to expose nearly every important phenomenon in machine learning: training, overfitting, failure modes, distribution shift, and adversarial examples. A final section extends the same machinery to character-level text generation, bridging the gap between image classification and the language models that dominate contemporary discourse. The notebook is not a tutorial in a library. It builds models from scratch first, so that the reader sees the loss, the gradient, and the update rule before any framework hides them.

No-nonsense takeaway. This article is for readers who want to understand AI, not just have opinions about it. It is grounded in working code, honest about limits, and written to be read by anyone willing to think carefully.

2 Defining AI: Terms, Scope, and Misconceptions

The phrase “artificial intelligence” is used so broadly that it has nearly lost descriptive power. It appears in product marketing, regulatory documents, philosophical debates, and casual conversation, often meaning different things in each. Before we can discuss what AI can and cannot do, we need to be clear about what we are talking about. This section establishes a working vocabulary, distinguishes several commonly conflated terms, and clears away a few persistent misconceptions.

2.1 AI, machine learning, deep learning, generative AI

The terms are nested, not synonymous. Figure 1 illustrates the relationship, and the following definitions apply throughout this article.

Artificial intelligence (AI). The broadest term: any attempt to make a machine perform tasks that, when performed by humans, would be said to require intelligence. This includes symbolic reasoning, planning, game-playing, natural language processing, perception, and more. The term is defined by its ambition, not by any particular method. It has been applied to systems as different as a 1950s theorem prover and a 2020s language model.

Machine learning (ML). A subset of AI in which the system is not explicitly programmed with rules but instead *learns* patterns from data. The programmer specifies a model architecture and an objective; the training process adjusts the model’s parameters to improve performance on that objective. Nearly all of the recent progress in AI has come from machine learning, which is why the two terms are often used interchangeably in practice – a habit that obscures the fact that AI includes non-learning approaches as well.

Deep learning (DL). A subset of machine learning based on neural networks with many layers. The “deep” refers to depth of composition: each layer transforms the output of the previous one, and the final layers represent increasingly abstract features. Deep learning became dominant around 2012, when it dramatically outperformed prior methods on image recognition, and has since spread to nearly every domain of ML.

Generative AI. A class of models that produce new content – text, images, audio, code – rather than only classifying or predicting. Large language models, diffusion models, and image generators are the best-known examples. Generative models are not a distinct algorithmic family; they are a

category defined by their output. A generative model can be built from the same building blocks as a classifier, and the boundary between the two is often blurry.

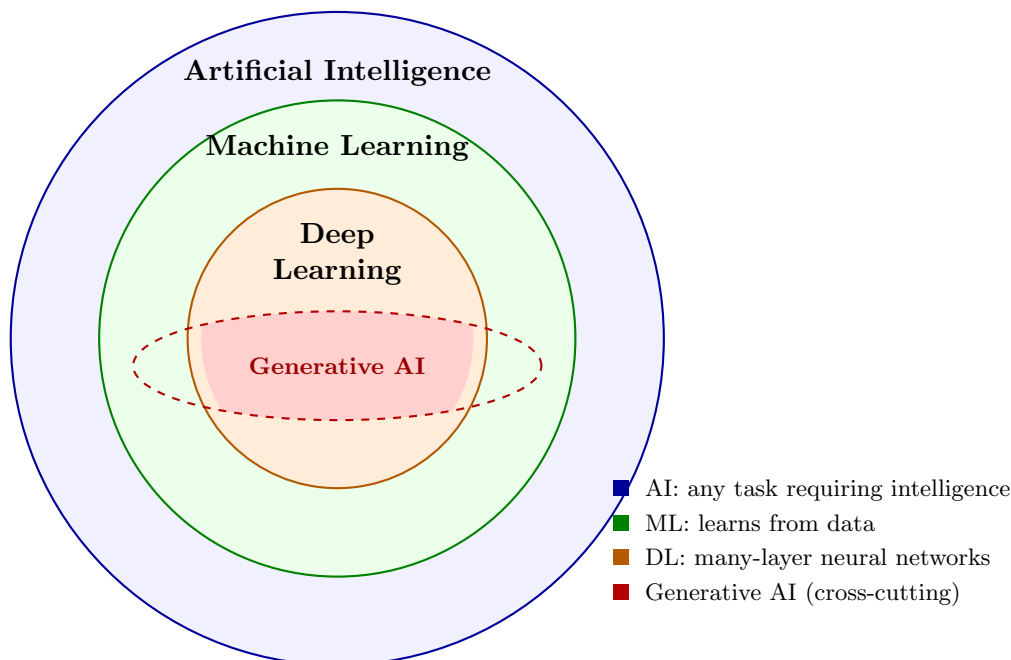


Figure 1: The nested vocabulary of contemporary AI. Machine learning is a subset of AI, deep learning is a subset of machine learning, and generative AI is a category defined by its output rather than by its mechanism. The dashed region for generative AI crosses the deep learning boundary because a generative model can be built from any family of models, deep or shallow; what distinguishes it is that it produces new content rather than predicting a label.

The nesting matters because claims about “AI” sometimes apply only to one narrow subfield. A statement about deep learning, for instance, may not hold for symbolic AI, and a statement about language models may not hold for reinforcement learning. Precision here is not pedantry; it is the difference between a meaningful claim and a slogan.

2.2 Narrow vs. general vs. superintelligence

A second distinction concerns the *scope* of capability, not the method.

Narrow AI. A system that performs a specific task or a small set of related tasks, usually within a well-defined domain. Image classifiers, speech recognisers, recommendation engines, chess engines, and language models are all narrow in this sense. Even a language model that can discuss any topic is narrow in the sense that it does one thing: predict the next token in a sequence. Its apparent generality comes from the breadth of its training data, not from a general-purpose reasoning architecture.

Artificial general intelligence (AGI). A hypothetical system with the ability to learn and perform any intellectual task a human can, transferring knowledge across domains without task-specific retraining. AGI does not exist. Whether it is achievable, how it might be recognised if it were, and what its arrival would imply are all contested questions. We return to them in

Section 12, but for now the key point is that contemporary AI, including the most impressive systems, is narrow.

Superintelligence. A hypothetical system whose capabilities substantially exceed those of the best human minds across all relevant domains. Superintelligence is a subject of serious speculation and serious disagreement. It is not an engineering target, and no current system is on a credible path to it. It appears in this article only in the context of debates about long-term risk, and we will be explicit when we are discussing speculation rather than fact.

Confusing these categories is one of the most common sources of misunderstanding. A headline claiming that “AI can now do X” usually means that a narrow system performs a specific version of X under specific conditions. That may still be significant, but it is not the same as a general capability.

2.3 Symbolic versus connectionist approaches

Historically, AI research has been divided between two broad traditions, and understanding them clarifies why modern systems look the way they do.

Symbolic AI. Also called classical or rule-based AI. Knowledge is represented explicitly as symbols, rules, and logical relationships. Reasoning proceeds by manipulating those symbols according to formal rules. Expert systems of the 1970s and 1980s were the high-water mark of this tradition. Symbolic systems are transparent and interpretable, but they struggle with perception, ambiguity, and the messy statistics of real-world data.

Connectionist AI. Knowledge is distributed across a large network of simple units whose collective behaviour emerges from learning. Neural networks are the canonical example. Connectionist systems are powerful on perceptual and statistical tasks, but they are opaque: it is often difficult to say why a particular input produced a particular output.

Modern AI is overwhelmingly connectionist, but the symbolic tradition has not disappeared. It survives in formal verification, knowledge graphs, and hybrid systems that combine learned components with explicit rules. The tension between interpretability and capability that separates the two traditions is not resolved; it is a recurring theme throughout this article.

2.4 What AI is not: consciousness, magic, autonomous agency

Three misconceptions deserve direct refutation.

AI is not consciousness. Current systems have no subjective experience, no feelings, no goals of their own. When a language model says “I want,” it is producing a statistically plausible continuation of text, not reporting an internal state. Whether a machine *could* be conscious is a legitimate philosophical question, but it is separate from the engineering question of what current systems do. Conflating the two leads to both overestimation (treating chatbots as persons) and underestimation (dismissing real capabilities because they are not accompanied by understanding).

AI is not magic. Every capability of a modern system comes from a combination of architecture, data, optimisation, and compute. These are engineering choices, and they are inspectable. The fact that a system’s internal representations are difficult for humans to interpret does not make the

system mysterious; it makes it complex. Throughout this article, we will show, with working code, where the capabilities come from.

AI is not autonomous agency. Most systems described as AI have no goals beyond the objective they were trained to optimise. An image classifier does not “want” to classify images; it computes a function. A language model does not “decide” what to say; it samples from a probability distribution. This matters because attributing agency to systems that lack it distorts both technical discussion and policy debate. The question of whether future systems could acquire genuine agency is important, but it should not be confused with the behaviour of systems that merely appear agentic.

The model learns a function from inputs to symbols, and the symbols are arbitrary. Figure 2 makes this concrete: a systematic relabeling of every digit is learned as easily as the original, because nothing about the task has changed except the names of the categories.

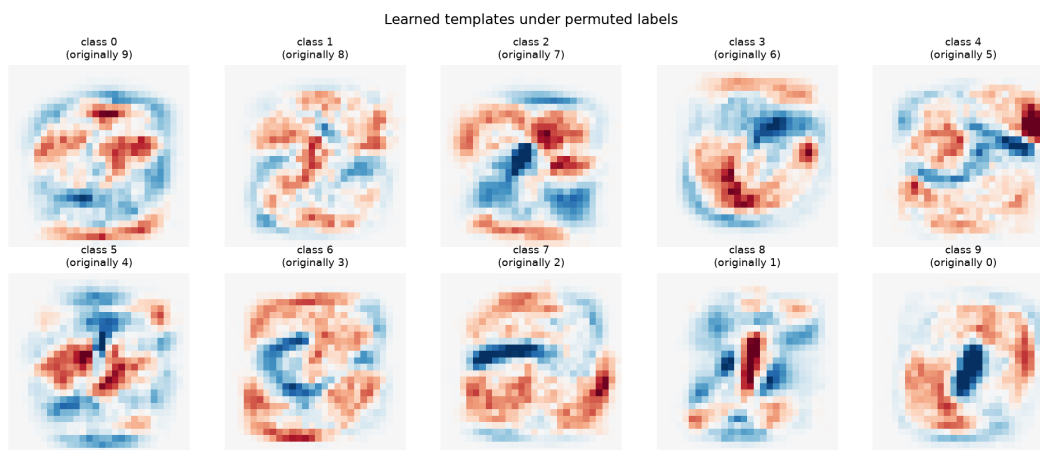


Figure 2: Learned weight templates after every label has been systematically relabeled: 0 becomes 9, 1 becomes 8, and so on. Each template is unchanged in shape; only the class it represents has been permuted. The model has no prior belief about what a label means, and consequently no confusion about the relabeling. A human would find these templates misleading; the model finds them perfectly coherent. Generated by the companion notebook, Part 4.4.

2.5 A working definition

For the purposes of this article, we will use the following definition:

AI refers to computational systems that perform tasks commonly associated with intelligent behaviour, typically by learning statistical patterns from data rather than by following explicitly programmed rules.

This definition is deliberately modest. It says nothing about consciousness, generality, or autonomy. It acknowledges that the tasks are defined by human judgment – what counts as “intelligent” is a moving target – and it locates the mechanism in learning from data, which is where nearly all contemporary capability comes from.

With vocabulary in hand, we can now turn to how the field arrived here.

No-nonsense takeaway. AI, machine learning, deep learning, and generative AI are nested categories, not synonyms. Contemporary AI is narrow, statistical, and non-conscious. Most confusion about AI comes from conflating these distinctions.

3 A Brief History of AI: From Logic to Large Models

The current excitement about AI is not the first, and understanding the pattern of previous waves is useful for calibrating the present one. This section is not a comprehensive history – excellent book-length treatments exist – but a compressed account of how the field arrived at its current state, with attention to the recurring dynamics of hype, disappointment, and consolidation. Figure 3 shows the full arc.

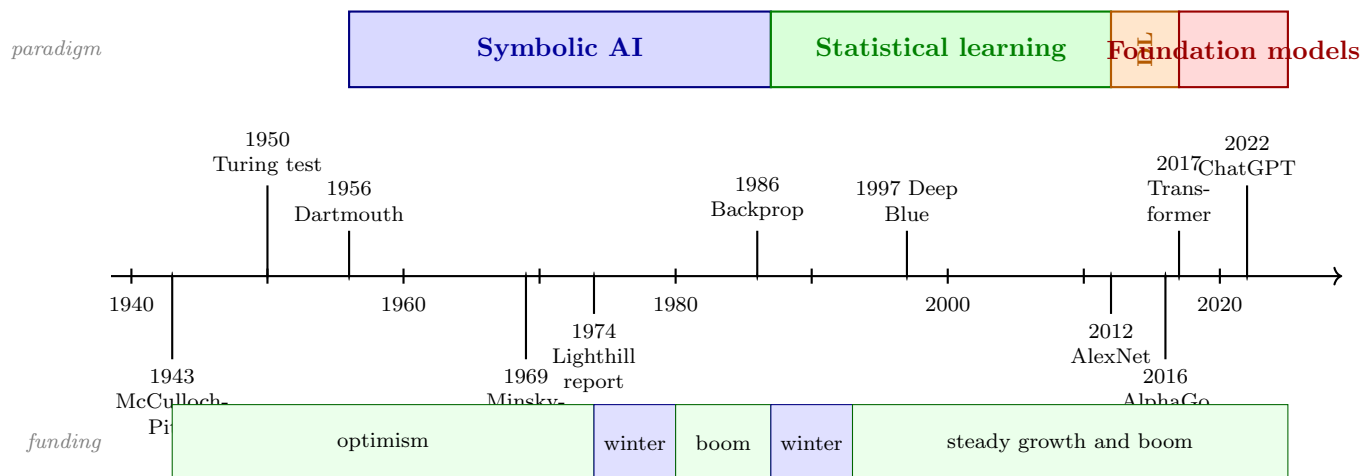


Figure 3: A timeline of AI research and deployment, 1943–2025. The top band shows the dominant research paradigm in each period; the bottom band shows the funding climate, with shaded regions marking the two AI winters. Milestones above the axis are foundational results; those below are systems and applications. The pattern of optimism, disappointment, and consolidation is visible at a glance: two winters follow two waves of enthusiasm, and the current boom is the third such wave. Compare with the discussion in Section 3.

3.1 Cybernetics, Turing, and the Dartmouth workshop

The intellectual roots of AI reach back to the 1940s. McCulloch and Pitts proposed a mathematical model of the neuron in 1943, showing that simple threshold units could in principle compute logical functions. Norbert Wiener’s work on cybernetics framed control and communication as unified problems spanning biology and machines. These early efforts were as much philosophical as technical: they asked whether thought itself could be mechanised.

Alan Turing’s 1950 paper “Computing Machinery and Intelligence” posed the question directly and proposed a test – now called the Turing test – that sidestepped the unanswerable question of whether a machine could truly think. If its behaviour were indistinguishable from a human’s under specified conditions, Turing argued, we should treat the question of inner experience as separate from the question of capability. This move, pragmatic then and still useful now, set the tone for much of what followed.

The field acquired its name in 1956 at a summer workshop at Dartmouth College, organised by John McCarthy, Marvin Minsky, Claude Shannon, and Nathaniel Rochester. Their proposal was strikingly ambitious: they believed that a two-month, ten-person effort could make significant progress on the problem of making machines use language, form abstractions, and solve problems reserved for humans. That estimate was wrong by a wide margin, and the gap between ambition and achievement has been a recurring feature of the field ever since.

3.2 Symbolic AI and expert systems

From the late 1950s through the 1980s, the dominant paradigm was symbolic. Researchers built systems that represented knowledge as explicit rules and manipulated those rules through formal logic. Early successes were real: the Logic Theorist and General Problem Solver, both developed by Allen Newell and Herbert Simon, solved problems that looked impressive. Programs like SHRDLU could manipulate a simulated blocks world in response to natural-language commands. The optimism was genuine.

The most commercially successful phase was the expert-system boom of the 1980s. Systems such as MYCIN (for diagnosing blood infections), DENDRAL (for chemical analysis), and XCON (for configuring computer systems at DEC) encoded the knowledge of human specialists in rule bases and outperformed non-experts on narrow tasks. Corporations invested heavily. A new industry of LISP machine manufacturers emerged to sell specialised hardware for running these systems.

The limitations, however, were structural. Rule bases were expensive to build and maintain, brittle at their boundaries, and incapable of learning from data. They could not handle the ambiguity and noise of real-world perception, and every new domain required starting from scratch. When the promised returns failed to materialise, the boom collapsed.

3.3 AI winters and revival

The first “AI winter” arrived in the mid-1970s, triggered in part by the Lighthill report in the United Kingdom and by funding cuts at DARPA in the United States. The second arrived in the late 1980s with the collapse of the LISP machine market and the withdrawal of corporate investment in expert systems. Both winters followed the same pattern: ambitious claims, insufficient results, withdrawn funding, and a quiet period during which the surviving researchers continued working with reduced resources.

The winters were not mere pauses. Important work continued, and some of it laid the groundwork for what came later. But they are instructive because they show that progress in AI is not monotonic and that the field’s institutional health depends on the gap between promises and deliverables staying within tolerable bounds. The current wave of investment rests on the same implicit bargain, and it is worth keeping the historical pattern in mind.

3.4 Statistical learning and the data-driven turn

Beginning in the late 1980s and accelerating through the 1990s, the field shifted from hand-coded rules to learning from data. Several developments converged.

Support vector machines, formalised by Corina Cortes and Vladimir Vapnik in 1995, provided a principled framework for classification with strong theoretical guarantees. Bayesian networks offered a rigorous language for reasoning under uncertainty. Hidden Markov models revolutionised speech recognition. Ensemble methods such as random forests and gradient boosting proved remarkably effective on tabular data. And neural networks, which had been marginalised since Minsky and

Papert’s 1969 critique of the perceptron, were revived by the popularisation of backpropagation in the mid-1980s.

The shift was not only technical but cultural. Benchmarks, datasets, and empirical evaluation became central. The field began to reward results on shared tasks rather than clever architectures in isolation. This is the culture that produced the current era, and it is why modern AI research looks more like experimental science than like mathematical logic.

Two milestone events during this period illustrated the growing strength of narrow systems: IBM’s Deep Blue defeated Garry Kasparov at chess in 1997, and IBM’s Watson won the quiz show *Jeopardy!* in 2011. Both were engineering triumphs of considerable sophistication, and both were also reminders that beating humans at a well-defined task does not imply general intelligence. Deep Blue could not play checkers; Watson could not hold a conversation.

3.5 Deep learning, transformers, and foundation models

The deep learning era is conventionally dated to 2012, when a convolutional neural network called AlexNet, developed by Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton, won the ImageNet competition by a large margin, reducing top-5 error from roughly 26% to 15%. Three factors made this possible: large labelled datasets, GPU compute, and architectural improvements that had been developed over the preceding two decades, notably by Yann LeCun and others. The result triggered a rapid shift of the entire field toward neural methods.

Progress accelerated through the 2010s. Recurrent networks and sequence-to-sequence models transformed machine translation. Generative adversarial networks produced strikingly realistic images. Reinforcement learning, combined with deep networks, achieved superhuman performance in Go (AlphaGo, 2016) and later in more complex games.

The pivotal architectural development was the transformer, introduced by Vaswani et al. in 2017 in the paper “Attention Is All You Need.” By replacing recurrence with a mechanism called self-attention, transformers made it possible to train very large models efficiently on very large corpora. The consequences unfolded quickly: BERT and GPT-1 in 2018, GPT-3 in 2020, and the public release of ChatGPT in late 2022, which brought large language models into mainstream awareness.

Around the same time, empirical scaling laws – described by Kaplan et al. in 2020 and refined by the Chinchilla results in 2022 – showed that model performance improved predictably with increases in parameters, data, and compute. This gave labs a clear recipe: scale up. The result has been a concentration of resources in a small number of organisations and a research culture increasingly oriented around large-scale engineering rather than small-scale experimentation.

3.6 Lessons from history

Three lessons stand out.

First, progress has been uneven and often surprising. Symbolic methods dominated for decades and then receded; neural networks were marginalised and then became dominant. The field has repeatedly been wrong about which approaches would succeed and about how long progress would take.

Second, hardware and data have mattered as much as algorithms. The transformer is an elegant architecture, but it became transformative only when combined with GPUs and internet-scale text. Conversely, the AI winters were triggered as much by the limits of available compute and data as by theoretical shortcomings.

Third, the field has a persistent tendency to overpromise. The Dartmouth proposal, the expert-system boom, and the current discourse all share a pattern of extrapolating from genuine achievements to unwarranted generality. This is not a reason for cynicism, but it is a reason for careful reading of claims, which is one of the aims of this article.

A useful observation, sometimes called the AI effect, is that once a capability becomes routine – optical character recognition, speech transcription, chess – it tends to stop being called AI. The goalposts move. What remains labelled “AI” is what has not yet been solved. This is worth remembering when evaluating both the achievements and the promises of the present moment.

No-nonsense takeaway. AI has been through multiple cycles of hype, disappointment, and consolidation. Progress has been real but uneven, driven as much by hardware and data as by ideas. The current wave is not the first, and historical caution is warranted.

4 Mathematical and Conceptual Foundations

Machine learning rests on a small number of mathematical ideas. None of them is exotic, and none requires more than undergraduate preparation to follow. But they do require patience, because they appear everywhere and in slightly different guises. This section introduces the five ideas that underpin everything that follows: linear algebra, probability, optimisation, information theory, and computational complexity. A reader who absorbs these will find the rest of the article substantially easier.

The treatment is deliberately compressed. Each subsection states the core idea, gives one concrete example, and points to the notebook where the idea becomes executable. Readers who prefer a conceptual path can read the prose and skip the boxes; readers who want the mathematics can work through them.

4.1 Linear algebra: vectors, matrices, embeddings

A *vector* is an ordered list of numbers. A *matrix* is a rectangular array of numbers, and it can be viewed as a collection of vectors, or as a function that maps one vector to another. This dual reading is the single most useful thing to internalise about linear algebra: matrices are both *data* and *operations*.

In machine learning, data almost always enters as vectors. An image of 28×28 pixels is a vector of 784 numbers. A word is represented as a vector of a few hundred numbers, called an *embedding*. A sentence is a matrix whose rows are word embeddings. A batch of 64 images is a matrix of shape 64×784 . Once the data is in this form, every computation the model performs is a matrix multiplication, followed by a nonlinearity, repeated many times.

Math box: matrix multiplication. If A has shape $m \times n$ and B has shape $n \times p$, then AB has shape $m \times p$ and its (i, j) entry is $\sum_{k=1}^n A_{ik}B_{kj}$. The operation composes linear maps: applying B and then A is the same as applying AB . This is why deep networks are called *compositions* of layers.

The concept of an *embedding* deserves special emphasis because it recurs throughout the article. An embedding is a learned map from discrete objects – words, images, users, products – to vectors in a continuous space, chosen so that similar objects are close together. In the notebook, the final layer of the digit classifier effectively produces an embedding of each image into a 10-dimensional space, and a 2D projection of those embeddings shows digits clustering by class. In large language

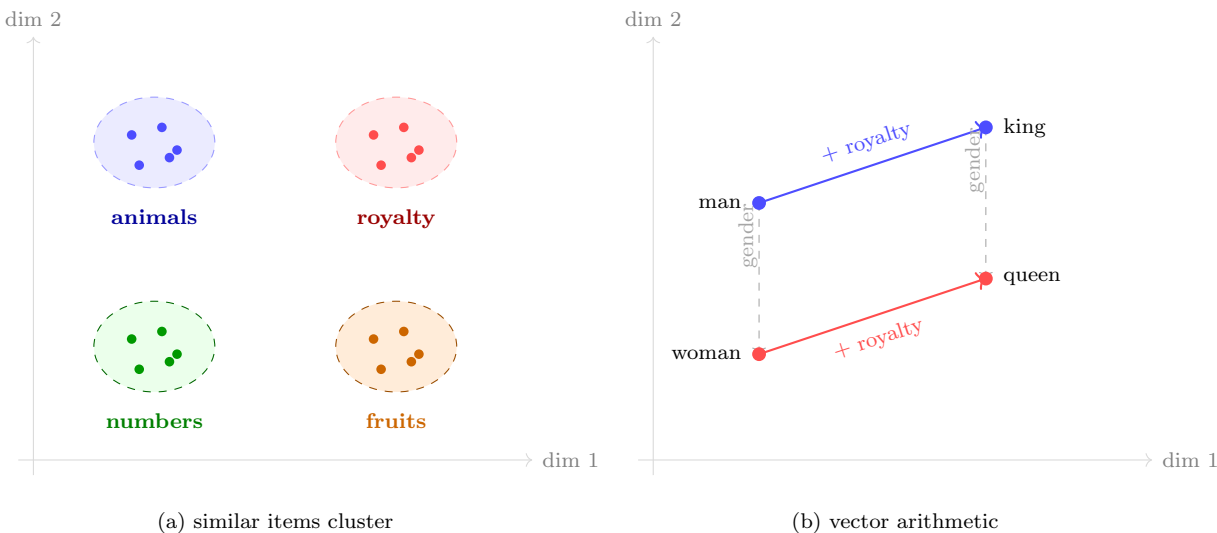


Figure 4: Two views of an embedding space. **(a)** A 2D projection of a high-dimensional embedding: items with similar meanings cluster together. The points are individual words; the colours and ellipses are annotations, not part of the model. **(b)** The famous analogy relation. The vector from *man* to *king* is approximately equal to the vector from *woman* to *queen*: both represent the same semantic displacement, and the model has learned this geometry from co-occurrence statistics alone, without any supervision. The effect is real but less clean in practice than the textbook example suggests. Generated from the discussion in Section 4.

models, word embeddings have become famous for capturing semantic relationships: the vector for “king” minus “man” plus “woman” lands near “queen”. These properties are not designed; they emerge from training. They are also not as clean as the popular examples suggest, but the underlying geometry is real. Figure 4 shows two views of what such a space looks like.

4.2 Probability and statistics: uncertainty, Bayes, distributions

Machine learning is applied probability. Every model produces a distribution over outputs, every loss function is a statement about uncertainty, and every claim about generalisation is a statistical claim. Three concepts matter most.

Random variables and distributions. A random variable is a quantity whose value depends on chance. A distribution specifies how likely each value is. The two distributions that appear most often are the *categorical* (a probability over a finite set of outcomes, such as the ten digit classes) and the *Gaussian* (the familiar bell curve, used to model continuous quantities and as a building block for many others). When a classifier outputs probabilities over ten digits, it is producing a categorical distribution.

Conditional probability and Bayes’ rule. The probability of A given B , written $P(A \mid B)$, is the central object of inference. Bayes’ rule relates it to its inverse:

$$P(A \mid B) = \frac{P(B \mid A) P(A)}{P(B)}.$$

This identity is elementary, but its consequences are not. It says that what we should believe after seeing evidence depends on both the evidence and our prior beliefs. Much of the confusion in applied AI – about class imbalance, about base rates, about medical testing – is confusion about conditional probability.

Estimation and uncertainty. A model trained on finite data has uncertainty about its own parameters. This uncertainty comes in two flavours: *aleatoric* (irreducible noise in the data) and *epistemic* (uncertainty that would shrink with more data). Distinguishing them matters. When a model is wrong on a rare input, the question is whether the input is genuinely ambiguous or whether the model has simply not seen enough examples of its kind. The notebook’s calibration plot, which compares predicted confidence to actual accuracy, is a practical window into this question.

4.3 Calculus and optimization: gradients, loss surfaces

Training a model means finding parameters that minimise a loss function. The loss is a scalar summary of how badly the model is doing; the parameters are the numbers the model is allowed to change. The search is almost always performed by *gradient descent* or one of its variants.

The gradient of a function at a point is the vector of its partial derivatives. It points in the direction of steepest ascent; its negative points in the direction of steepest descent. Gradient descent takes a small step in that direction, recomputes the gradient, and repeats.

Math box: the update rule. If θ denotes the model’s parameters, $L(\theta)$ the loss, and $\eta > 0$ a small step size (the *learning rate*), then one step of gradient descent is

$$\theta \leftarrow \theta - \eta \nabla_{\theta} L(\theta).$$

Everything in modern deep learning is a variation on this line.

Three practical subtleties matter. First, L is rarely convex: the loss surface has many local minima, saddle points, and flat regions. That gradient descent works as well as it does in high dimensions is one of the genuine surprises of the field, and one we return to in Section 9. Second, the learning rate η is the single most important hyperparameter. Too small and training is slow; too large and it diverges. The notebook’s learning-rate sweep makes this visceral (see Figure 5). Third, gradients in deep networks are computed by *backpropagation*, which is nothing more than the chain rule applied systematically, layer by layer. We derive it informally in Section 7.

4.4 Information theory: entropy, KL divergence, compression

Information theory, developed by Claude Shannon in 1948, provides the language for talking about uncertainty, surprise, and the cost of representing data. Three quantities recur in machine learning.

Entropy $H(p)$ measures the average surprise of a distribution p . A distribution concentrated on one outcome has low entropy; a uniform distribution has high entropy. Entropy is also, up to a constant, the minimum number of bits per symbol needed to encode samples from p .

Cross-entropy $H(p, q)$ measures the cost of encoding samples from p using a code optimised for q . When $q = p$, cross-entropy equals entropy; otherwise it is larger. This is why cross-entropy is the standard loss function for classification: minimising it forces the model’s predicted distribution q to match the true distribution p .

Kullback–Leibler (KL) divergence $D_{\text{KL}}(p \| q)$ is the cross-entropy minus the entropy, and it measures how much q differs from p . It is not symmetric, which makes it a divergence rather than

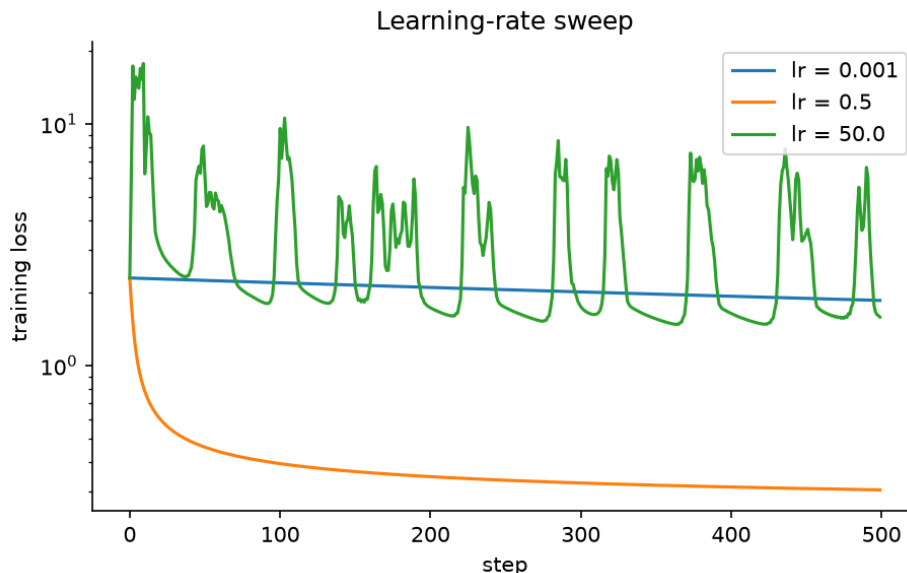


Figure 5: Training loss for logistic regression on MNIST at three learning rates. Too small (0.001) and training barely moves; moderate (0.5) and it converges smoothly; too large (50) and it diverges. The learning rate is the single most consequential hyperparameter, and this is the simplest demonstration of why. Generated by the companion notebook, Part 4.1.

a distance, but it is the natural measure of how much information is lost when q is used in place of p .

The connection to compression is not a metaphor. A model that assigns high probability to the data is, equivalently, a model that compresses the data well. Training a language model to minimise cross-entropy on text is the same as training it to be a good compressor of text, and the compression ratio it achieves is a measure of how much structure it has learned. This equivalence is one of the deepest ideas in the field and will reappear in Sections 7.

4.5 Computation and complexity: what is tractable?

Not every function can be computed, and not every computable function can be computed efficiently. Complexity theory classifies problems by the resources – time and memory – required to solve them. Two distinctions matter for AI.

First, *P versus NP*: problems solvable in polynomial time versus problems whose solutions can be verified in polynomial time. Many problems of practical interest in AI (planning, constraint satisfaction, some forms of inference) are NP-hard, meaning no known algorithm solves them efficiently in the worst case. Deep learning does not make NP-hard problems easy; it produces good approximate answers to instances that arise in practice.

Second, *approximation and heuristics*: when exact solutions are intractable, we settle for approximate ones. Gradient descent is itself an approximate method. It does not find the global minimum of a non-convex loss; it finds a local one that generalises well. This is not a bug. It is the reason the method works at scale.

4.6 No free lunch and inductive biases

The no-free-lunch theorem states, informally, that no learning algorithm is better than any other when averaged over all possible problems. Any algorithm that excels on some problems must pay for it by performing poorly on others. This sounds like a limitation, and it is, but it also explains why machine learning works at all.

Real-world problems are not uniformly distributed over the space of all possible problems. They have structure: images are smooth, language is compositional, nearby points in time are correlated. A learning algorithm performs well when its *inductive bias* – the assumptions it makes about the structure of the problem – matches the structure of the data. Convolutional networks assume that nearby pixels are related and that patterns are translation-invariant. Transformers assume that relationships between elements are pairwise and can be learned from data. These assumptions are why the architectures succeed on images and text, respectively. They are also why they fail on problems whose structure does not match.

The practical lesson is that architecture choice is not a matter of fashion. It is a statement about what kind of structure you expect in your data. The notebook’s transition from a fully connected network to a convolutional one is precisely this lesson made concrete: the same data, the same loss, a different inductive bias, and a measurable improvement in accuracy.

No-nonsense takeaway. Five mathematical ideas – linear algebra, probability, optimisation, information theory, and complexity – underpin machine learning. Each is elementary in isolation. Together they explain why learning from data is possible, why it is hard, and why the choice of architecture is a statement about the structure of the problem.

5 How Machine Learning Works

Section 4 supplied the mathematical ingredients. This section assembles them into a procedure. The procedure is surprisingly uniform across domains: define a model, define a loss, optimise the loss on data, and then worry about whether the result generalises. Almost every modern AI system, from a spam filter to a large language model, is an instance of this loop. The differences lie in the data, the model class, and the objective – not in the underlying logic.

5.1 Data, features, labels, and tasks

The fundamental move of machine learning is to replace explicit rules with examples. Instead of writing “if the email contains this phrase, mark it as spam,” we collect thousands of labelled emails and let the algorithm find the patterns. This is both the source of the method’s power and the source of most of its failure modes. Figure 6 shows the dataset used throughout this article.

Formally, we are given a dataset of input–output pairs $\{(x_i, y_i)\}_{i=1}^N$ and asked to find a function f such that $f(x) \approx y$ for new inputs the model has not seen. The input x is called the *feature vector*; the output y is called the *label* or *target*. The function f is the *model*, and finding it is the *learning* problem.

Three points deserve emphasis. First, *the data is a proxy for the world, not the world itself*. A model trained on emails from 2019 will not necessarily work on emails from 2025. A medical classifier trained on one hospital’s population may fail at another. This gap – between the training distribution and the deployment distribution – is called *distribution shift*, and it is the single most common cause of real-world AI failures. Section 9 returns to it.

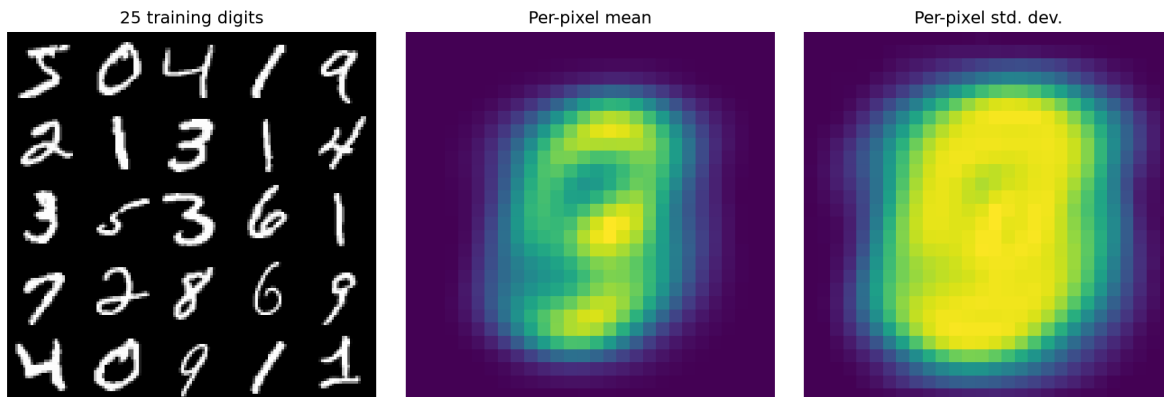


Figure 6: The MNIST dataset. Left: 25 training examples. Centre: per-pixel mean across the training set. Right: per-pixel standard deviation. The border pixels carry almost no information — they are dark in both statistics — which is a preview of feature selection: not all inputs matter equally, and discovering which matter is part of the learning problem. Generated by the companion notebook, Part 1.

Second, *features are representations*. A raw image is a grid of pixels; a raw email is a string of characters. Neither is a natural input to a learning algorithm. Historically, practitioners hand-engineered features — edges for images, n-grams for text, summary statistics for tabular data. Deep learning changed this by learning features directly from raw data, which is why it displaced so much of the earlier pipeline. The embeddings introduced in Section 4 are the learned-feature analogue for discrete objects.

Third, *tasks vary*. Classification predicts a discrete label; regression predicts a continuous value; ranking orders items by relevance; generation produces new objects.

Figure 7 shows a related practical lesson: even the scale of the input can determine whether training succeeds.

5.2 Four paradigms of learning

Machine learning is conventionally divided into four paradigms, according to what kind of supervision is available.

Supervised learning. The dataset contains labelled examples, and the goal is to predict the label for new inputs. Image classification, spam detection, and medical diagnosis are supervised problems. Supervised learning is the most mature paradigm and the easiest to evaluate, because correct answers are known.

Unsupervised learning. No labels are given. The goal is to discover structure: clusters, latent factors, or a lower-dimensional representation. Dimensionality reduction and anomaly detection are the canonical tasks. Unsupervised learning is conceptually appealing but harder to evaluate, since there is no ground truth against which to check the result.

Self-supervised learning. A hybrid that has become the dominant paradigm of the current era. The labels are derived from the data itself. A language model is trained to predict the next word given previous words; the “label” is simply the next word, already present in the text. A vision

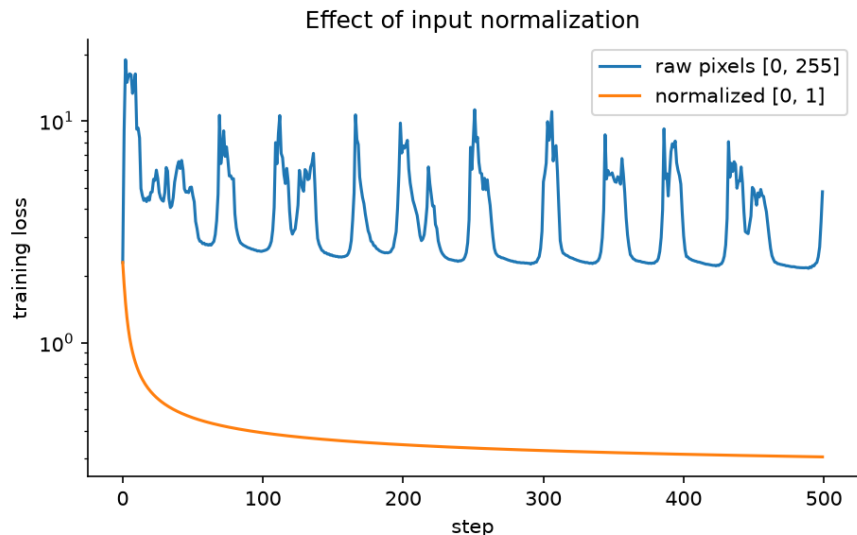


Figure 7: Same model, same learning rate, same number of steps. The only difference is whether the input pixels are scaled to $[0, 1]$ or left in their original $[0, 255]$ range. Unnormalized inputs make the loss surface ill-conditioned, and training stalls. Normalization is not a detail. Generated by the companion notebook, Part 4.2.

model can be trained to predict whether two crops of the same image are related. Self-supervision requires no human annotation, which is why it scales to internet-sized corpora. It is the reason large language models can be trained on trillions of tokens. We return to this in Section 7.

Reinforcement learning. An agent interacts with an environment, takes actions, and receives rewards. The goal is to learn a policy that maximises cumulative reward over time. Reinforcement learning is responsible for game-playing systems such as AlphaGo, for robotics, and for the fine-tuning stage of modern language models (RLHF). It is powerful but notoriously sample-inefficient: agents often require millions of interactions to learn behaviours that humans acquire in a few trials.

In practice, the boundaries blur. A modern language model is pretrained with self-supervision, fine-tuned with supervision, and aligned with reinforcement learning. The pipeline is not a single paradigm but a sequence of them.

5.3 Training as optimization

Training a model means finding parameters θ that minimise a loss function $L(\theta)$ averaged over the training data. This is an optimisation problem, and the standard algorithm is gradient descent, as introduced in Section 4. In practice, three refinements are universal.

First, *stochastic* gradient descent. Computing the gradient over the entire dataset is expensive, so we estimate it using a small *minibatch* of examples, typically 32 to 512. The estimate is noisy but unbiased, and the noise actually helps: it allows the optimiser to escape shallow local minima and saddle points.

Second, *momentum and adaptive step sizes*. Pure gradient descent oscillates on ill-conditioned problems. Methods such as Adam maintain a running estimate of the gradient’s mean and variance and adapt the step size per parameter. Adam and its variants are the default optimisers for deep learning.

Third, *epochs and schedules*. One pass through the dataset is an *epoch*. Training typically runs for many epochs, with the learning rate reduced according to a schedule (e.g., cosine decay) as training proceeds.

Math box: the training loop. For each minibatch B :

1. Forward pass: compute predictions $\hat{y} = f_{\theta}(x)$ for $x \in B$.
2. Loss: compute $L = \frac{1}{|B|} \sum_{(x,y) \in B} \ell(\hat{y}, y)$.
3. Backward pass: compute gradients $\nabla_{\theta} L$ by backpropagation.
4. Update: $\theta \leftarrow \theta - \eta \nabla_{\theta} L$ (or an Adam variant).

Repeat until the validation loss stops improving.

Figure 8 shows the loop as a diagram; Figure 9 shows what happens when the same model is trained for too few or too many steps.

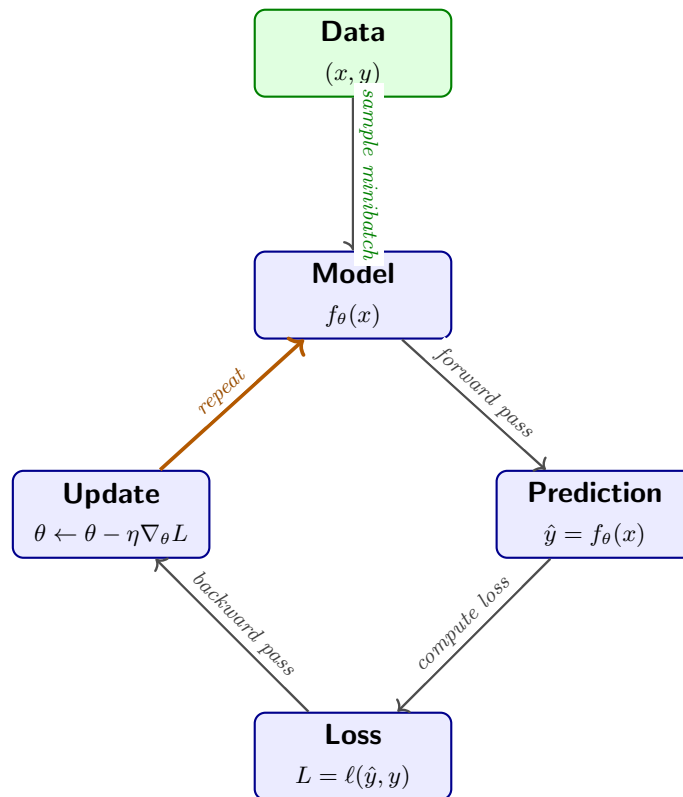


Figure 8: The supervised learning loop. A minibatch of inputs and labels is drawn from the data and fed to the model, which produces predictions. The loss measures the discrepancy between predictions and labels. Its gradient with respect to the parameters is computed by backpropagation (the backward pass), and the parameters are updated by gradient descent. The loop repeats until the validation loss stops improving. Every neural network in this article is a refinement of this loop: the differences between a digit classifier and a frontier language model lie in the model, the data, and the scale, not in the structure of the loop. Reproduced in the companion notebook, Part 3.

This loop, in six lines, is the entire training procedure of every neural network in use today. The differences between a small digit classifier and a frontier language model lie in the size of the model, the amount of data, and the number of iterations – not in the structure of the loop.

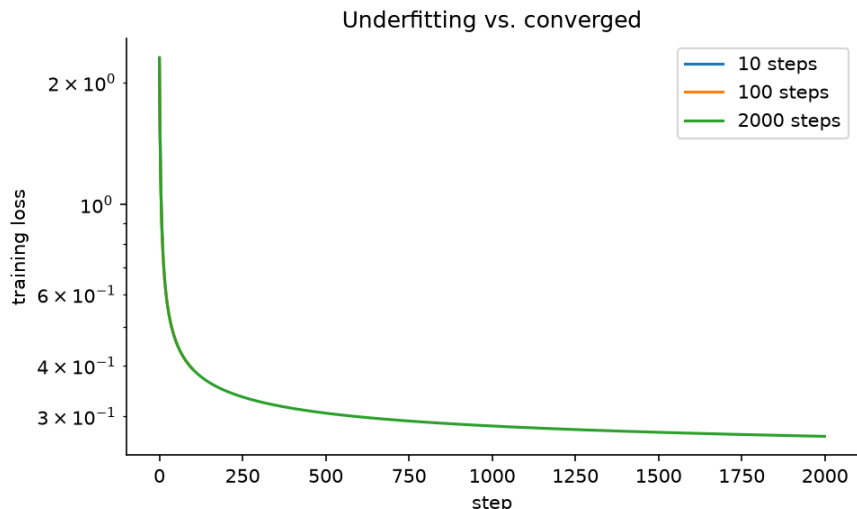


Figure 9: The same model trained for 10, 100, and 2000 steps. Ten steps is underfitting: the loss has not begun to fall. One hundred steps is partway. Two thousand is converged. The training loop is simple; the question of when to stop it is not. Generated by the companion notebook, Part 4.3.

5.4 Generalization and overfitting

Minimising the loss on the training data is not the goal. The goal is performance on new data. A model that memorises its training set but fails on everything else is useless, and the tendency to memorise is called *overfitting*.

The opposite failure is *underfitting*: the model is too simple, or trained for too few steps, to capture the structure of the data. The practical art of machine learning is finding the point between the two.

To detect overfitting, we split the data. A typical split is 80% for training, 10% for validation, and 10% for testing. The model is trained on the training set, hyperparameters are tuned on the validation set, and the final performance is reported on the test set. The test set is touched *once*. Peeking at it repeatedly – even indirectly, by choosing the model that scores best on it – is one of the most common methodological sins in applied machine learning, and it silently invalidates reported results.

The classical picture of overfitting is a “U-shaped” curve: training loss falls monotonically, validation loss falls and then rises. This picture is correct for simple models and small datasets, and the notebook’s overfitting lab (training on 100 examples) reproduces it exactly. But the picture is incomplete for modern deep networks. In many cases, validation loss continues to fall even after the model has enough capacity to memorise the training set, a phenomenon known as *double descent*. The reasons are not fully understood, and we will be honest about that in Section 9. For now, the classical framing remains the right place to start. Figure 10 shows the curve on a small training set.

Regularization. Several techniques combat overfitting. Weight decay penalises large parameters. Dropout randomly disables units during training, forcing the network to be robust. Data augmentation (rotations, crops, noise) effectively enlarges the training set. Early stopping halts training when validation loss stops improving. Each of these is a small intervention, and their combined effect is often dramatic. The notebook applies them one at a time to the overfitting model

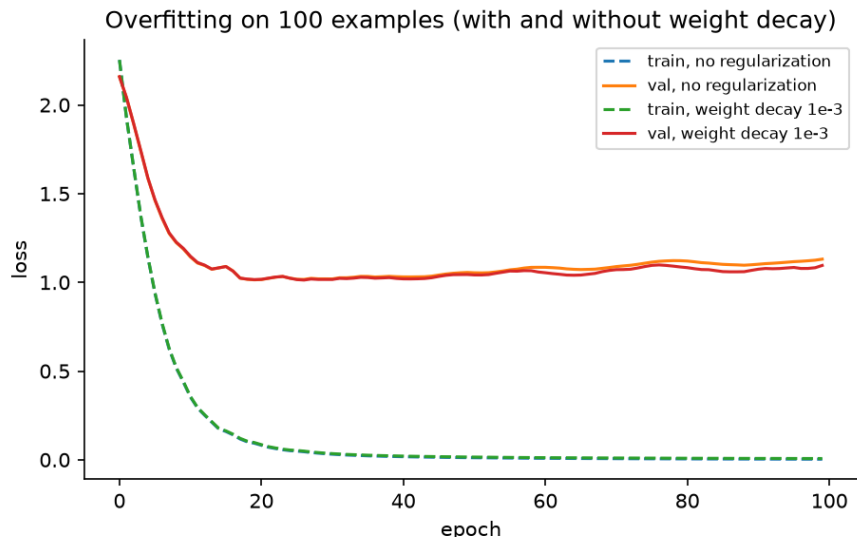


Figure 10: Overfitting on 100 examples. Training loss falls monotonically; validation loss falls and then rises. The gap between the two curves is overfitting, and it is visible within a hundred epochs. Weight decay delays the divergence but does not eliminate it. Generated by the companion notebook, Part 10.

and shows each delaying the onset of divergence. A different test asks whether the model has learned anything about the meaning of the labels. Figure 11 shows that a systematic permutation is learned as easily as the original task, while a random assignment is not learned at all.

Bias and variance. A useful (though imperfect) framing: models with high *bias* are systematically wrong because they cannot represent the true function; models with high *variance* are sensitive to the particular training sample and fail to generalise. Simple models tend toward high bias; complex models toward high variance. The tradeoff is real but often overstated, especially in the deep-learning regime where the classical curve does not always apply. We treat it as a useful heuristic rather than a law. Figure 12 shows both the classical decomposition and the modern empirical picture, in which test error can decrease again past the interpolation threshold.

5.5 Evaluation and its pitfalls

Accuracy is the most intuitive metric and frequently the wrong one. If 99% of emails are not spam, a classifier that always predicts “not spam” achieves 99% accuracy and is useless. When classes are imbalanced, *precision* (of the emails flagged as spam, how many actually were) and *recall* (of the actual spam, how much was caught) are more informative. The F1 score combines them.

For multi-class problems, a *confusion matrix* shows which classes are being confused with which. In the notebook, the digit classifier’s confusion matrix reveals that 4s and 9s are frequently mixed up, and 3s and 5s less so. Inspecting the model’s most confidently wrong predictions turns out to be far more instructive than looking at aggregate accuracy. Several of the misclassified digits are genuinely ambiguous even to a human reader.

Two further pitfalls deserve mention. First, *calibration*: a model that outputs 0.9 probability should be correct about 90% of the time on those predictions. Neural networks are often overconfident, producing probabilities near 1.0 on inputs they get wrong. The notebook’s calibration plot

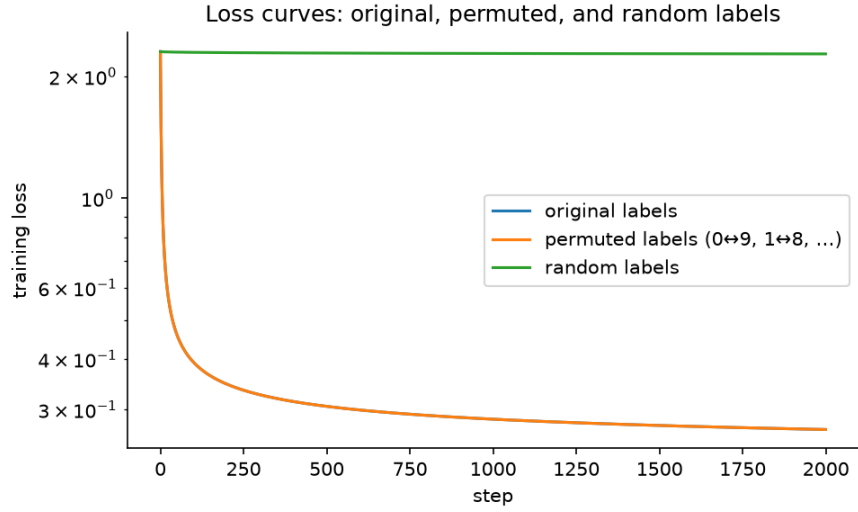


Figure 11: Training loss for three label assignments: original, systematically permuted ($0 \rightarrow 9, 1 \rightarrow 8, \dots$), and random. The permuted-label curve is indistinguishable from the original — a consistent relabeling is learned as easily as the original task. The random-label curve plateaus at a much higher loss because there is no consistent pattern to learn. Structure in the mapping, not the model, determines whether learning succeeds. Generated by the companion notebook, Part 4.4.

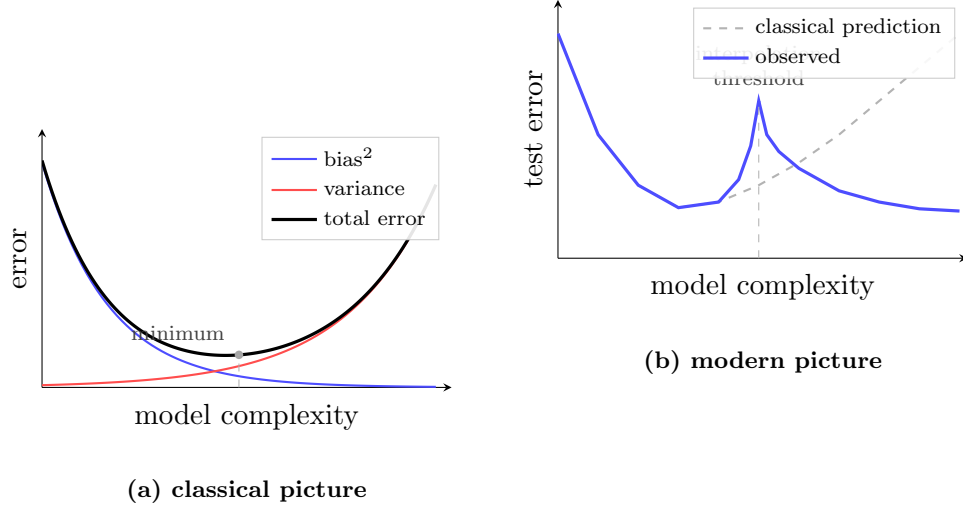


Figure 12: Bias and variance as a function of model complexity. **(a)** The classical decomposition. As complexity increases, bias falls and variance rises. Their sum is U-shaped, with a minimum at intermediate complexity. This is the framework described in textbooks, and it was developed for models where complexity can be measured by parameter count. **(b)** The modern empirical picture. Test error follows the classical U-curve until the interpolation threshold – the point at which the model has enough capacity to fit the training data perfectly – then rises to a peak before falling again. The second descent was documented in the late 2010s and is not predicted by classical theory. Its mechanisms remain an open question; see Section 9.

makes this visible. Second, *evaluation on the wrong distribution*: a model evaluated on data drawn from the same distribution as its training set may look excellent while failing badly on the data it will see in deployment. Rotation and occlusion tests in the notebook show this collapse directly. Figure 13 plots the calibration curve for the digit classifier. Figure 14 shows the confusion matrix, and Figure 15 shows the model’s twenty most confidently wrong predictions.

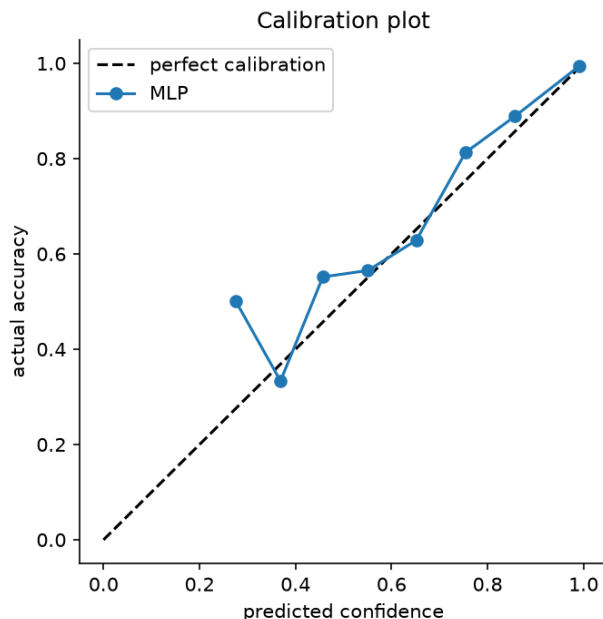


Figure 13: Calibration plot. The diagonal is perfect calibration: a prediction made with 90% confidence should be correct 90% of the time. The model’s curve lies below the diagonal, which means it is overconfident. This matters anywhere the probability itself is used as a decision input. Generated by the companion notebook, Part 8.

5.6 A minimal worked example

It is worth seeing the entire procedure in one place, stripped to its essentials. Suppose we want to classify digits using logistic regression. Figure 16 shows both the loss curve and the learned weight templates.

The model is $f_{\theta}(x) = \text{softmax}(Wx + b)$, where W is a 10×784 weight matrix and b is a 10-dimensional bias. The softmax converts raw scores into a probability distribution over the ten digit classes.

The loss is cross-entropy: for a true label y and predicted distribution $\hat{p}$, $L = -\log \hat{p}_y$. Averaged over a minibatch, this is the quantity we minimise.

The gradient has a remarkably simple form: for each example, the gradient of the loss with respect to the logits is $\hat{p} - \text{onehot}(y)$. This says that the model is pushed away from its prediction in proportion to how wrong it was. The full gradient with respect to W is the outer product of this error with the input x .

Training is then just the loop from Section 5.3 above: forward pass, loss, gradient, update. On MNIST, this model reaches about 92% test accuracy after a few hundred iterations on a laptop CPU. Every subsequent model in this article – convolutional networks, transformers, language

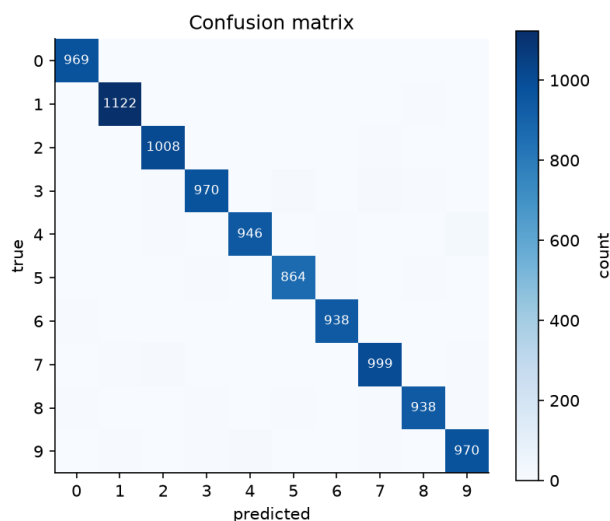


Figure 14: Confusion matrix for the digit classifier. The 4/9 confusion is the largest off-diagonal entry, followed by 3/5. Aggregate accuracy conceals which errors occur and where they concentrate; the confusion matrix makes this visible at a glance. Generated by the companion notebook, Part 8.

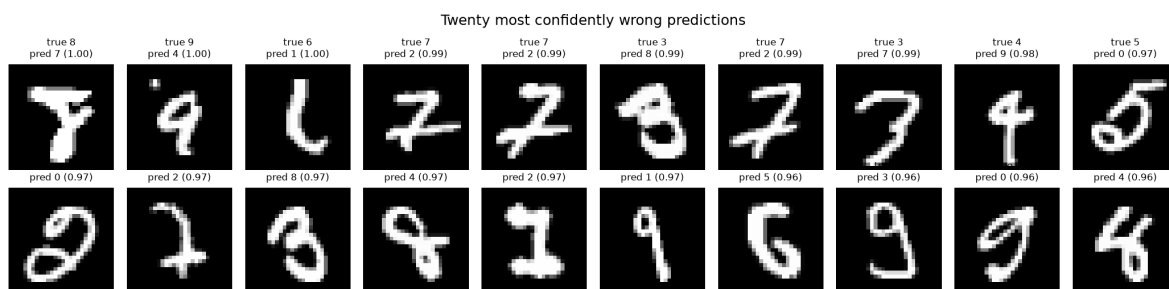


Figure 15: The twenty most confidently wrong predictions. Several are genuinely ambiguous even to a human reader. The model's confidence on these errors is itself the problem: a system that is wrong and knows it is wrong is easier to use than one that is wrong and confident. Generated by the companion notebook, Part 8.

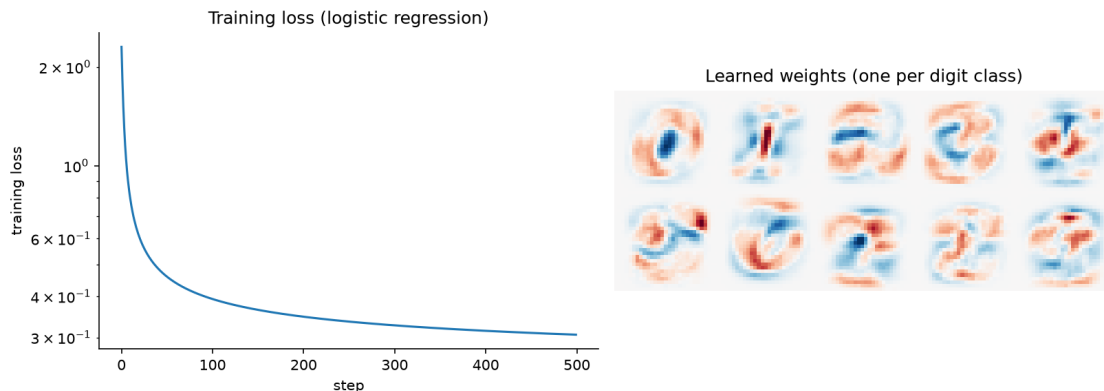


Figure 16: Logistic regression on MNIST, trained from scratch in NumPy. Left: cross-entropy loss falls smoothly over 500 steps. Right: the learned weight matrix, reshaped as ten 28×28 templates, one per digit class. Red pixels push the input toward that class; blue pixels push away. Gradient descent produced interpretable templates without any human instruction about which pixels matter. Generated by the companion notebook, Part 3.

models – is a refinement of this same structure, with more layers, more parameters, and better inductive biases, but the same loss, the same gradient, and the same update rule.

This is why the article insists on the notebook. A reader who has written these twenty lines of NumPy has a working mental model of what training does. The rest is scale and architecture.

No-nonsense takeaway. Machine learning is a loop: define a model, define a loss, optimise with gradients, and check that the result generalises. Supervised, unsupervised, self-supervised, and reinforcement learning differ in what supervision is available, not in the core mechanism. Overfitting, distribution shift, and miscalibration are the practical failure modes. Everything else in modern AI is a refinement of this loop.

Further reading for Part I

For a comprehensive textbook treatment covering both symbolic and statistical traditions, the standard reference remains Russell and Norvig’s *Artificial Intelligence: A Modern Approach* (4th ed., 2024), which is encyclopedic but readable in sections. For the history, Nilsson’s *The Quest for Artificial Intelligence* (2009) is the standard account, written by a participant and covering the field from its eighteenth-century precursors to the modern era. For the mathematical foundations at a gentler pace than most ML texts, Deisenroth, Faisal, and Ong’s *Mathematics for Machine Learning* (2020) is freely available online and assumes only undergraduate preparation. Readers who want a sense of how a working practitioner thinks before touching any textbook should read Karpathy’s “A Recipe for Training Neural Networks” (2019), which is short, opinionated, and accurate. For the deep learning era specifically, Goodfellow, Bengio, and Courville’s *Deep Learning* (2016) remains the definitive technical reference and is available free online, though it predates transformers and should be supplemented accordingly.

Part II

Mechanics and Practice

6 Core Model Families

Section 5 described the training loop in the abstract. This section fills in the models that loop is applied to. The families below are not a chronological history; several were developed in parallel, and older methods remain in active use. They are best understood as different *inductive biases* – different assumptions about the structure of the data – each suited to a particular kind of problem.

A useful framing, which we will return to repeatedly: choosing an architecture is not choosing a tool from a menu. It is making a claim about the world. A convolutional network claims that nearby pixels are related and that patterns recur across locations. A transformer claims that relationships between elements are pairwise and learnable from data. When the claim is right, the model learns efficiently. When it is wrong, no amount of data or compute fully compensates.

6.1 Linear and logistic regression

The simplest models are still the right place to start. Linear regression predicts a continuous output as a weighted sum of inputs: $\hat{y} = w^\top x + b$. Logistic regression predicts a probability over discrete classes by passing the same weighted sum through a softmax: $\hat{p} = \text{softmax}(Wx + b)$.

Both are *linear* models: their decision boundaries are straight lines (or hyperplanes in higher dimensions). This makes them interpretable – the weight w_j tells you how much feature j contributes – and computationally cheap. It also limits them. A linear classifier cannot represent the XOR function, cannot learn that a digit is a digit regardless of where it appears in the image, and cannot capture any interaction between features unless the interaction is explicitly encoded.

The reason these models matter beyond their own modest capabilities is that they are the last layer of nearly every deep network. A modern image classifier is, at its head, a logistic regression on a learned representation. The deep layers do the feature engineering; the final linear layer does the classification. Understanding logistic regression is therefore understanding the output stage of everything that follows.

6.2 Decision trees, forests, and boosting

Tree-based methods take a different approach. A decision tree partitions the input space by asking a sequence of yes/no questions: is pixel 5 above threshold t ? If so, is pixel 12 above threshold s ? Each leaf of the tree corresponds to a region, and predictions are constant within each region. Trees handle nonlinearity naturally, require no feature scaling, and are interpretable in the sense that any prediction can be traced back to the questions that produced it.

Single trees are unstable: small changes in the data can produce very different trees, and they overfit readily. Two remedies dominate.

Random forests train many trees on bootstrap samples of the data and average their predictions. The averaging reduces variance without increasing bias much, and the result is a robust, general-purpose classifier.

Gradient boosting trains trees sequentially, each one fitted to the residual errors of the previous ensemble. Modern implementations such as XGBoost, LightGBM, and CatBoost are among the most successful algorithms in applied machine learning.

A point worth stating plainly, because it is often lost in the neural network enthusiasm: *on tabular data, gradient-boosted trees routinely outperform deep networks*. This has been documented in numerous benchmark studies and remains true in practice. The reason is inductive bias: tabular data often contains features of different scales, mixed types, and sharp threshold effects that trees model naturally and neural networks do not. Anyone who claims neural networks are universally better has not worked with tabular data.

6.3 Support vector machines and kernels

Support vector machines (SVMs), developed in the 1990s, find the linear boundary that maximises the margin between classes. The insight is that only the points closest to the boundary – the support vectors – determine the solution, which makes the method robust and theoretically well-understood.

The more consequential idea is the *kernel trick*. Many problems that are not linearly separable in the original feature space become linearly separable after mapping to a higher-dimensional space. The kernel trick computes inner products in that high-dimensional space without ever constructing it explicitly, which makes the mapping computationally tractable even when the space is infinite-dimensional.

Math box: the kernel trick. A kernel $k(x, x')$ computes $\langle \phi(x), \phi(x') \rangle$ for some feature map ϕ , without requiring ϕ to be computed. For example, the polynomial kernel $k(x, x') = (x^\top x' + 1)^2$ implicitly maps to all degree-2 monomials. Any algorithm that depends on the data only through inner products – SVMs, kernel PCA, Gaussian processes – can be kernelised this way.

Kernel methods were the dominant paradigm before deep learning and remain useful for small datasets, for problems with clear theoretical structure, and as a conceptual bridge. The idea that a model can work in a high-dimensional feature space implicitly is the same idea that underlies neural network *embeddings*, and the connection is not coincidental.

6.4 Neural networks: from perceptrons to MLPs

The perceptron, proposed in 1958, is a single linear threshold unit. It can only separate linearly separable classes, a limitation famously highlighted by Minsky and Papert in 1969 and often cited as a cause of the first AI winter.

The multilayer perceptron (MLP) removes this limitation by stacking layers: a linear transformation, followed by a nonlinear activation (ReLU, sigmoid, tanh), followed by another linear transformation, and so on. The composition of linear maps and nonlinearities can approximate any continuous function on a compact domain, given enough hidden units. This is the *universal approximation theorem*, and while it is often cited as a justification for neural networks, it is worth being precise: the theorem says such a network *exists*, not that gradient descent will find it, and not that it will generalise. Figure 17 shows the effect of the nonlinearity directly: a single hidden layer with ReLU reaches roughly 97% on MNIST; remove the activation and accuracy drops back to the linear baseline.

The essential ingredient is the nonlinearity. Remove it, and the composition of linear maps is still linear – the whole network collapses to a single linear model. The notebook demonstrates this directly: removing the activation function from a two-layer MLP drops accuracy back to the logistic regression baseline.

Depth matters. A deep network can represent certain functions with exponentially fewer parameters than a shallow one of equivalent expressive power. This is the theoretical case for depth,

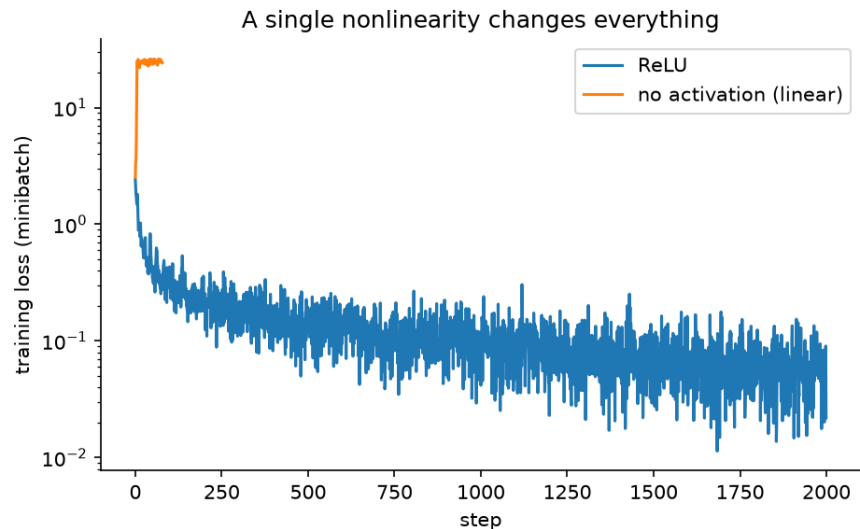


Figure 17: A single hidden layer with ReLU reaches roughly 97% on MNIST. Remove the activation and the network collapses back to the logistic regression baseline, because a composition of linear functions is itself linear. Nonlinearity is not a detail of the architecture; it is the reason depth is meaningful. Generated by the companion notebook, Part 5.

and it is borne out empirically: deep networks consistently outperform shallow ones on structured problems.

6.5 Convolutional and recurrent networks

The MLP treats the input as a flat vector, which discards all spatial and sequential structure. Two architectures restore that structure.

Convolutional neural networks (CNNs). A convolution applies a small filter (typically 3×3 or 5×5) across the input, producing a feature map at each location. The same filter is applied everywhere, so the network learns translation-equivariant features: a pattern learned in one part of the image is recognised in another. This is a powerful inductive bias for images, and it is why CNNs dominated computer vision from 2012 until the recent rise of vision transformers.

The bias is also a limitation. CNNs assume locality and translation equivariance. They struggle with tasks where global relationships matter more than local patterns, and they are less flexible than attention-based models at mixing information across distant regions of the input. Figure 18 compares a small CNN to the MLP on MNIST, and Figure 19 shows what the first convolutional layer learns: edge detectors, exactly what the inductive bias predicts.

Recurrent neural networks (RNNs). An RNN processes a sequence one element at a time, maintaining a hidden state that summarises what it has seen so far. This is a natural fit for language, audio, and time series. Variants such as LSTMs and GRUs addressed the vanishing-gradient problem that plagued early RNNs and enabled practical sequence modelling.

RNNs have two structural weaknesses that eventually displaced them. First, they process sequences sequentially, which prevents parallelisation across the sequence and makes training on

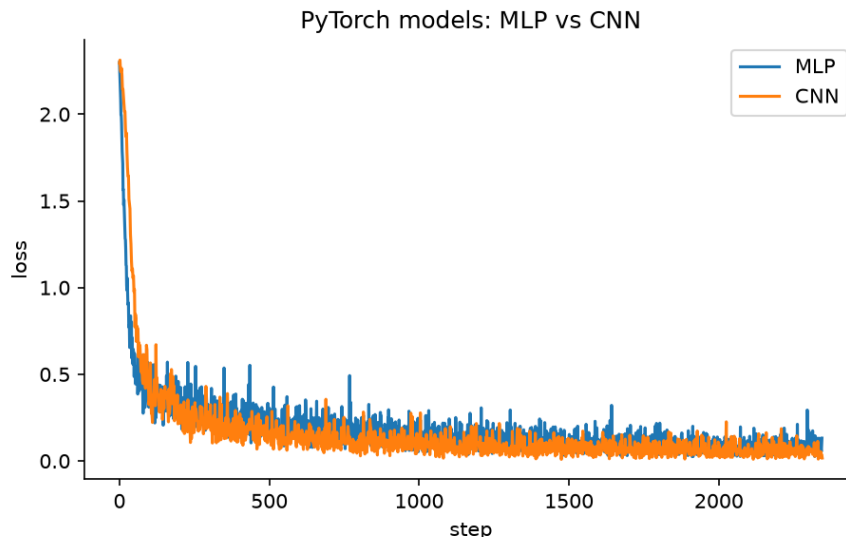


Figure 18: Same data, same loss, same optimizer. The MLP uses two fully connected layers; the CNN uses two convolutional layers. The CNN reaches roughly 99%, about two points above the MLP. The improvement is due to inductive bias: convolution assumes locality and translation equivariance, and MNIST happens to satisfy both assumptions. Generated by the companion notebook, Part 6.

long sequences slow. Second, even with gating mechanisms, they struggle to retain information over very long ranges. Both weaknesses are addressed by attention.

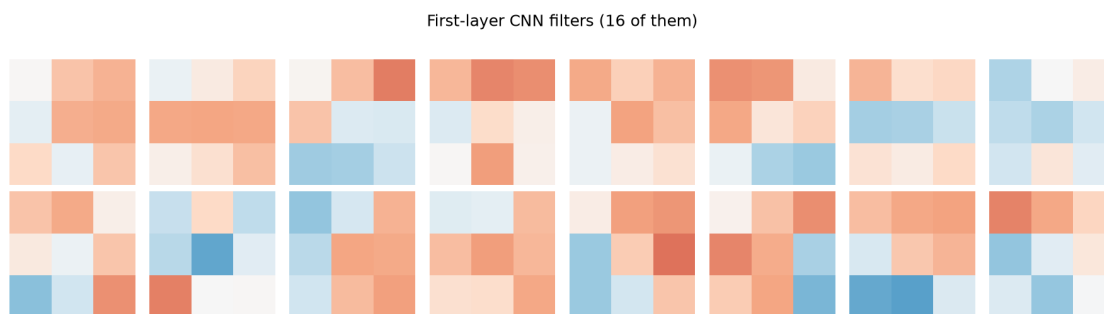


Figure 19: The sixteen learned filters of the first convolutional layer. Each is a 3×3 patch. The filters are edge detectors of various orientations, which is exactly what the inductive bias of convolution predicts. Nobody told the network to detect edges; the constraint of locality plus the task of digit classification produced them. Generated by the companion notebook, Part 7.

6.6 Transformers and attention

The transformer, introduced in 2017, replaced recurrence with *self-attention*. At each position in the sequence, the model computes a weighted sum over all other positions, with weights determined by learned similarity. This allows any position to attend to any other in a single step, eliminating the sequential bottleneck and enabling parallel training on very long sequences.

We defer the mechanics of attention to Section 7, where they can be treated properly. For now,

the conceptual points are:

- Attention is a form of *content-based routing*. Instead of hard-coding which inputs are relevant to which outputs, the model learns to route information dynamically based on the content of the inputs.
- Transformers make minimal assumptions about structure. Unlike CNNs (which assume locality) or RNNs (which assume sequential order), transformers assume only that relationships between elements are pairwise and learnable. This makes them flexible but data-hungry: they need large datasets to learn the structure that other architectures encode as a prior.
- The flexibility is why transformers have displaced prior architectures across modalities – text, images, audio, protein structures – and why they are the substrate of every current frontier model.

The cost is the quadratic attention complexity: doubling the sequence length quadruples the compute. This is an active research area, and one of the practical limits discussed in Section 9.

6.7 Generative models

The families above are primarily *discriminative*: they predict a label given an input. *Generative* models learn the distribution of the data itself, which allows them to produce new samples. Four approaches dominate.

Variational autoencoders (VAEs). Learn a latent variable model by encoding inputs into a distribution over a low-dimensional latent space and decoding samples back into data. The objective combines reconstruction accuracy with a penalty on the divergence between the learned latent distribution and a prior. VAEs produce smooth latent spaces but tend to generate blurry samples.

Generative adversarial networks (GANs). Train two networks in competition: a generator that produces samples and a discriminator that tries to distinguish them from real data. GANs produce sharp images but are notoriously unstable to train, prone to mode collapse, and difficult to evaluate.

Diffusion models. Learn to reverse a gradual noising process. Training is stable, sample quality is high, and the framework is principled. The cost is that generation requires many iterative steps, which makes sampling slow. Diffusion models underlie most of the image-generation systems in current use.

Autoregressive models. Factorise the joint distribution of the data into a product of conditionals, each predicted from the preceding elements. Large language models are autoregressive: they predict the next token given previous tokens. This is the subject of Section 7. Figure 20 shows samples from a small autoencoder, illustrating that the same machinery that classifies can also generate.

These families differ in their objectives, their stability, and the kind of data they handle best. They are not competitors so much as different answers to the question of what it means for a model to “understand” a distribution. The practical choice depends on the modality, the compute budget, and the quality requirements.

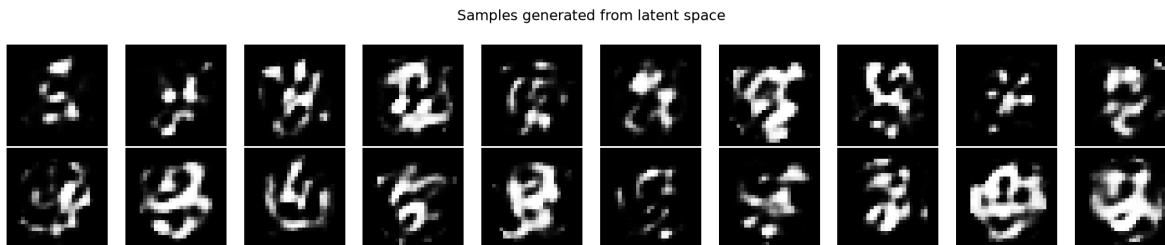


Figure 20: Samples generated by a small autoencoder trained on MNIST. The model compresses each digit into a 64-dimensional latent vector, then decodes samples from that latent space back into images. The samples are recognisably digits and recognisably not real ones. The same machinery that classifies can also generate; the difference is in the objective and the output layer. Generated by the companion notebook, Part 11.

6.8 A summary view

Table 1 summarises the families discussed above. The columns correspond to the inductive bias each family encodes, the data type it suits best, and its principal limitation. The table is not a decision procedure, but it makes the central point of this section visible at a glance: architecture choice is a statement about structure.

Family	Inductive bias	Best for	Main limitation
Linear / logistic	Linear decision boundary	Baselines, interpretability	Cannot model interactions
Trees / boosting	Axis-aligned partitions	Tabular data	Poor on images, text
SVMs / kernels	Margin maximisation, implicit feature maps	Small datasets, theory	Does not scale to large data
MLPs	Universal approximation, no structural prior	Generic function fitting	Data-hungry, no structure
CNNs	Locality, translation equivariance	Images, local patterns	Weak on global structure
RNNs	Sequential order, hidden state	Time series, short sequences	Slow, long-range failures
Transformers	Pairwise learnable relations	Text, multimodal, large-scale	Quadratic cost, data-hungry
VAEs	Latent variable model	Smooth representations	Blurry samples
GANs	Adversarial game	Sharp images	Unstable, mode collapse
Diffusion	Iterative denoising	High-quality generation	Slow sampling
Autoregressive	Factorised conditionals	Language, sequences	Sequential generation

Table 1: Model families, their inductive biases, and their limitations.

No-nonsense takeaway. Every model family encodes assumptions about the structure of the data. Linear models and trees remain essential baselines; kernel methods introduced the idea of implicit high-dimensional representations; neural networks learn features from raw data; CNNs

and RNNs encode structural priors that transformers replace with learned attention; and generative models differ in how they represent and sample from distributions. No family dominates across all problems.

7 The Mechanics of Modern AI Systems

Section 6 surveyed the model families. This section opens the hood on the two that dominate current practice: how a network actually learns its parameters, and how the transformer processes a sequence. It then follows the modern pipeline from raw data through pretraining, fine-tuning, and alignment, and closes with the practical matters of inference, hardware, and cost. The aim is not to make the reader a practitioner, but to remove the sense that any of this is mysterious.

7.1 Backpropagation and gradient descent

Training a network means computing the gradient of the loss with respect to every parameter, and doing so efficiently. Backpropagation is the algorithm that makes this possible. It is not a separate theory; it is the chain rule, applied systematically from the output back to the input.

Consider a network as a composition of functions: $f(x) = f_L(f_{L-1}(\cdots f_1(x)))$. The loss L depends on the output of the final layer, which depends on the output of the previous layer, and so on. The chain rule says that the derivative of the loss with respect to an early parameter is the product of derivatives along the path from that parameter to the loss. Backpropagation computes all these products in a single backward pass, reusing intermediate results.

Math box: the chain rule in one dimension. If $y = g(u)$ and $u = h(x)$, then $\frac{dy}{dx} = \frac{dy}{du} \frac{du}{dx}$. In a network, u is the output of a layer, y is the loss, and the same identity applies at every layer. Backpropagation is just this, vectorised and applied in reverse order.

Two properties make backpropagation practical. First, its cost is comparable to a forward pass: one backward pass computes gradients for all parameters at once, rather than once per parameter. Second, it is generic: any differentiable architecture can be trained this way, which is why new architectures can be invented without redesigning the training algorithm. Automatic differentiation in modern frameworks (PyTorch, JAX) is backpropagation generalised to arbitrary computational graphs.

The notebook implements backpropagation by hand for a two-layer network in NumPy, then shows the identical computation handled automatically by PyTorch. The comparison is instructive: the manual version exposes the mechanism, the framework version exposes what is genuinely hard (nothing, once the mechanism is understood) and what is merely tedious (everything else).

7.2 Embeddings and representation learning

An embedding is a learned map from a discrete object to a vector in a continuous space, chosen so that the geometry of the space reflects relationships between objects. The concept was introduced in Section 4; here we describe how embeddings are learned and why they matter.

In a language model, each token in the vocabulary is associated with a vector. These vectors are parameters, trained by the same gradient descent that trains everything else. After training, the geometry is surprising and useful: words that appear in similar contexts have similar vectors,

and certain analogical relationships appear as vector arithmetic. The famous example is that $\vec{\text{king}} - \vec{\text{mān}} + \vec{\text{wōmān}} \approx \vec{\text{quēēn}}$. The effect is real but less clean than the popularised version suggests; it holds for some word pairs and not others, and it depends on the corpus.

The deeper point is that embeddings are not restricted to words. Images, users, products, sentences, and entire documents can be embedded. The notebook produces embeddings of MNIST digits in the penultimate layer of the classifier; a 2D projection shows digits clustering by class, which is the image analogue of the word-vector geometry. This is what representation learning means: the model does not merely classify, it constructs a space in which classification becomes easy. Figure 21 shows this space for the digit classifier.

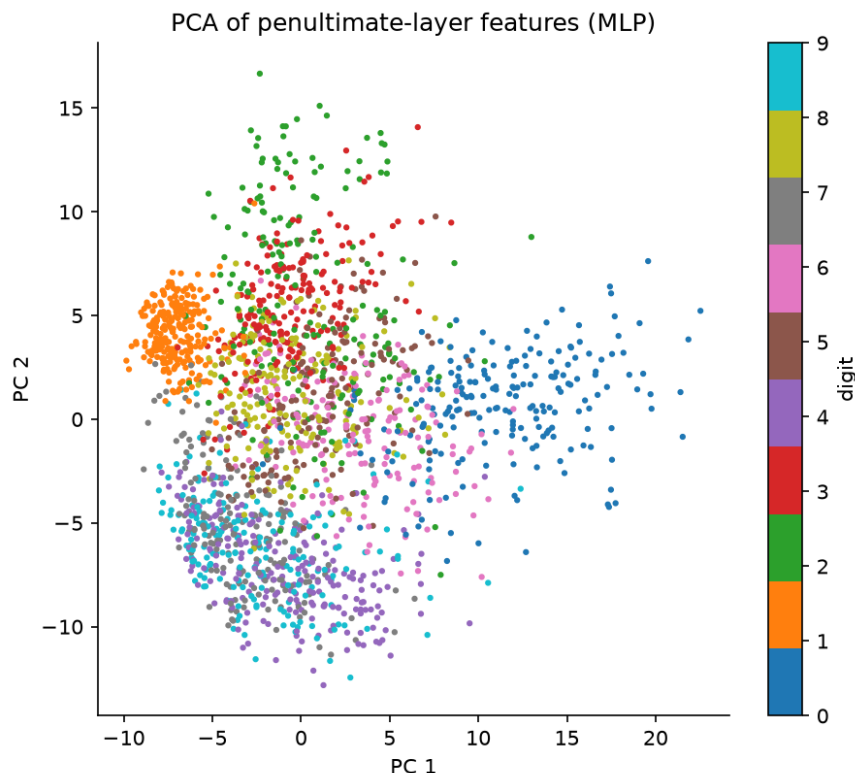


Figure 21: Principal-component projection of the penultimate-layer activations for 2000 test digits. The model was never told to group digits by class; the geometry of the representation reflects the classes anyway. Digits of the same class cluster, and some classes sit near each other in the projection because they are genuinely similar (for instance, 4 and 9). This is what representation learning means. Generated by the companion notebook, Part 7.

7.3 Attention and sequence modeling

Attention is the mechanism that made transformers dominant. At each position in a sequence, the model computes a weighted combination of information from all positions, with weights determined by learned similarity. The weights are *dynamic*: they depend on the inputs, not on their positions. This is what distinguishes attention from the fixed weights of a convolution.

Math box: scaled dot-product attention. Given a query matrix Q , a key matrix K , and a value matrix V (each derived from the input by learned linear projections),

attention is

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V,$$

where d_k is the dimension of the keys. The product $QK^\top$ measures similarity between every query and every key; the softmax converts those similarities into weights; and the product with V produces a weighted average of values.

Three things are worth noting. The $\sqrt{d_k}$ scaling prevents the softmax from saturating when the dimension is large. The softmax is what makes attention a form of content-based routing: high-similarity pairs receive most of the weight, low-similarity pairs receive almost none. And the entire operation is differentiable, so the projections that produce Q , K , and V are learned end-to-end along with everything else.

Multi-head attention runs several attention operations in parallel with different projections, then concatenates the results. This allows the model to attend to different kinds of relationships simultaneously: one head might track syntactic dependencies, another semantic similarity, a third positional adjacency. The division of labour is not imposed; it emerges from training.

The cost of attention is quadratic in sequence length: for a sequence of n tokens, the $QK^\top$ product requires $O(n^2)$ operations. This is why long-context models are expensive, why so much effort goes into efficient attention variants, and why the practical context window of any model is a resource tradeoff rather than a hard limit. Figure 22 shows the full block.

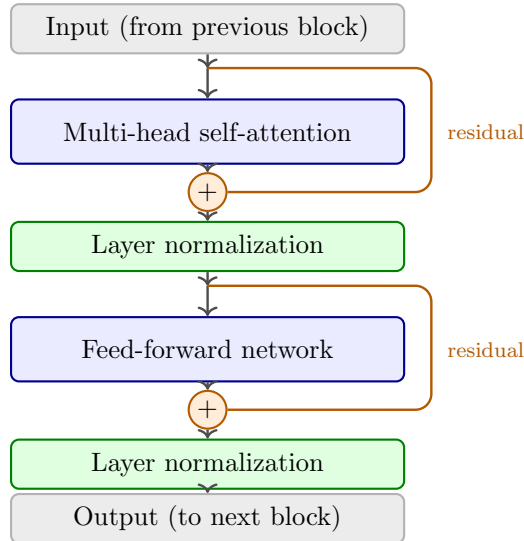


Figure 22: A single transformer block. The input flows through two sublayers: multi-head self-attention and a position-wise feed-forward network. Each sublayer is wrapped in a residual connection (the orange path, which adds the sublayer’s input to its output) and followed by layer normalization. The output of one block is the input to the next; a full model stacks many such blocks. The same structure appears in every current frontier language model. Generated from the discussion in Section 7.

7.4 Pretraining, fine-tuning, and alignment

Modern AI systems are rarely trained in one pass. The dominant pipeline has three stages.

Pretraining. The model is trained on a very large corpus with a self-supervised objective. For language, the standard objective is next-token prediction: given a sequence of tokens, predict the next one. The labels are derived from the text itself, so no human annotation is required, which is why pretraining can scale to trillions of tokens. The result is a base model that has absorbed broad statistical structure of the training data but is not yet useful as an assistant.

Fine-tuning. The base model is further trained on a smaller, curated dataset for a specific task or behaviour. Supervised fine-tuning uses examples of desired input–output pairs. Parameter-efficient methods such as LoRA adjust a small number of additional parameters rather than the full model, which makes fine-tuning accessible without data-centre resources.

Alignment. The model is adjusted to produce outputs that are helpful, harmless, and honest according to human preferences. Reinforcement learning from human feedback (RLHF) is the best-known method: human raters compare pairs of model outputs, a reward model is trained on those comparisons, and the language model is optimised against the reward model. Variants such as DPO (direct preference optimisation) simplify the pipeline. Alignment is an active research area, and it is where the gap between capability and safety is most visible.

The pipeline matters because the stages are doing different things. Pretraining gives breadth; fine-tuning gives specificity; alignment gives behavioural shape. A base model and an aligned model may share architecture and scale while behaving very differently in deployment. Confusing the two is a common source of misleading comparisons.

7.5 Scaling laws and emergent behaviour

The most consequential empirical finding of the past several years is that model performance improves predictably with scale. Loss decreases as a power law in model parameters, dataset size, and compute. This relationship, documented by Kaplan et al. (2020) and refined by the Chinchilla results (2022), is remarkable because it holds across many orders of magnitude and allows practitioners to forecast the effect of a training run before paying for it. Figure 23 shows the frontier.

Two qualifications are important. First, scaling laws describe loss, not capability. Lower loss does not automatically translate into better performance on every task, and the mapping from loss to usefulness is task-dependent. Second, the Chinchilla result showed that earlier models were *undertrained relative to their size*: for a given compute budget, it is better to train a smaller model on more data than a larger model on less. This finding shifted practice toward data-efficient training.

The concept of *emergent abilities* – capabilities that appear abruptly at certain scales rather than improving smoothly – is widely discussed and contested. Some apparent emergences turn out to be artefacts of how performance is measured, particularly when metrics are discontinuous. The honest position is that some capabilities clearly improve smoothly with scale, some appear to emerge, and the distinction is not yet fully understood. We return to this in Section 9.

7.6 Inference, prompting, retrieval, and agents

Training produces a model. Deployment uses it, and inference has its own set of techniques.

Prompting. The simplest way to steer a model is through its input. In-context learning – providing examples in the prompt rather than updating the weights – allows a trained model to

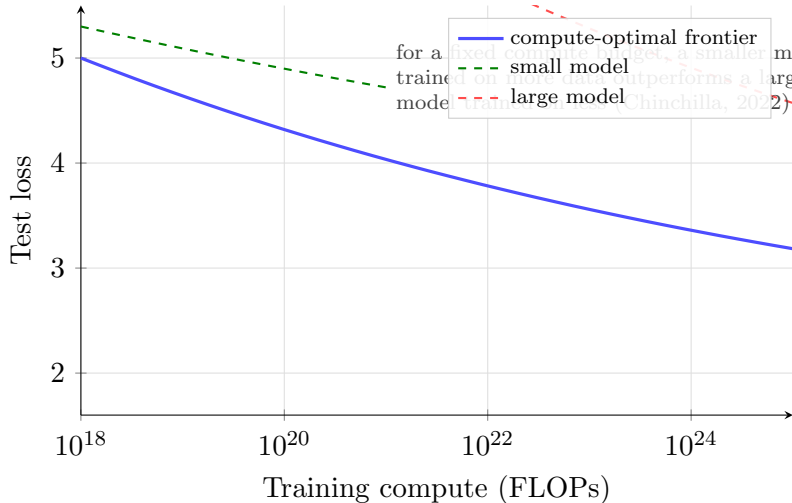


Figure 23: Test loss as a function of training compute, on logarithmic axes. The solid blue curve is the *compute-optimal frontier*: for each compute budget, it shows the loss achieved by the best model-and-dataset configuration. Its straight-line appearance on log-log axes is the empirical signature of a power law. The dashed curves show what happens when the model is poorly matched to the budget: too small a model plateaus early, and too large a model wastes compute before reaching its potential. The Chinchilla result showed that many earlier models were in the second regime, and shifted practice toward training smaller models on more data. The mechanisms that produce these power laws are not well understood; see Section 9.

adapt to a new task without any gradient updates. Chain-of-thought prompting, which asks the model to produce intermediate reasoning steps before its final answer, improves performance on multi-step problems. These techniques work, and the reasons they work are not fully understood; they are among the more interesting open questions in the field.

Retrieval. A model’s knowledge is frozen at the end of training. Retrieval-augmented generation (RAG) addresses this by retrieving relevant documents from an external store and inserting them into the prompt. The model then conditions on both the query and the retrieved context. RAG is the standard approach for grounding models in up-to-date or proprietary information, and it is more tractable than retraining.

Agents. An agent wraps a model in a loop: observe the state, choose an action, observe the result, repeat. The model supplies the decision-making; external tools supply capabilities the model lacks (calculators, code execution, search). Agentic systems are a fast-moving area, and the term is used loosely. The honest summary is that current agents work well in constrained environments and poorly in open ones, largely because the underlying models lack reliable long-horizon planning.

7.7 Hardware and compute economics

Training and serving modern models require specialised hardware. GPUs dominate because their architecture is well suited to the dense matrix multiplications that dominate neural computation. TPUs, developed by Google for the same purpose, are an alternative. Memory bandwidth, not raw compute, is often the binding constraint, which is why techniques such as quantisation (representing weights in lower precision) and mixed-precision training matter enormously in practice.

The economics are as important as the architecture. A frontier training run can cost tens of millions of dollars in compute alone. Inference at scale is a continuing operational cost, not a one-time expense, which is why efficiency research (distillation, pruning, quantisation, caching) receives sustained attention. The concentration of resources required for frontier training has implications for who can build such models, which we discuss in Section 11.

No-nonsense takeaway. Backpropagation is the chain rule applied efficiently; attention is learned content-based routing with quadratic cost; modern systems are pretrained at scale, then fine-tuned and aligned; and deployment involves prompting, retrieval, and agents rather than retraining. The mechanics are not mysterious, but they are resource-intensive, and the economics shape who can participate.

8 From Prototype to Practice: The AI Lifecycle

Parts I and II have so far treated machine learning as an intellectual exercise: define a model, define a loss, optimise, evaluate. That framing is necessary for understanding, but it is incomplete. Systems that reach users operate inside a lifecycle that begins long before training and continues long after deployment. The failures that matter most in practice rarely occur in the model. They occur in the data pipeline, the deployment environment, the monitoring system, or the incentives surrounding the whole enterprise.

This section describes that lifecycle. Its central claim is blunt: most AI systems fail for reasons that have nothing to do with the algorithm. Figure 24 shows the pipeline.

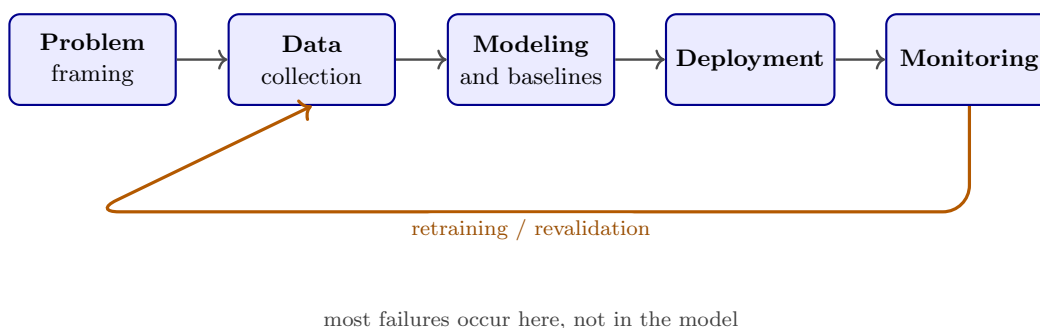


Figure 24: The AI lifecycle. The pipeline runs from problem framing through data collection, modelling, deployment, and monitoring. The return path (orange) is the retraining loop: monitoring feeds back into the data pipeline, which re-enters the modelling stage. The dominant failure modes in deployed systems are data, deployment, and incentive failures rather than algorithmic ones, a pattern discussed in detail here and in Section 9.

8.1 Problem framing and feasibility

The first question is not “which model should we use?” but “is this a problem that machine learning can solve, and should it be solved this way?” Three conditions must hold.

First, *the task must be learnable from available data*. If the required input is not recorded, if the label is subjective or inconsistent, or if the phenomenon depends on information the model cannot access, no architecture will help. A model predicting loan default cannot use information that was never collected; a model diagnosing a disease cannot recover from unreliable labels.

Second, *the objective must be specifiable*. Machine learning optimises a stated loss. If the true objective is vague, such as "make the product better", "reduce harm", the model will optimise whatever proxy is written down, which may diverge sharply from what was intended. This is Goodhart's law in its machine-learning form: when a measure becomes a target, it ceases to be a good measure.

Third, *the cost of errors must be acceptable*. A model that is accurate on average may still be unacceptable if its errors fall disproportionately on vulnerable groups or on rare but consequential cases. Framing must include not only "can we predict this?" but "what happens when we are wrong?"

The most common failure at this stage is skipping it. Teams begin with a dataset and a model, discover months later that the problem was ill-posed, and conclude that machine learning does not work. It usually does. It was simply applied to the wrong question.

8.2 Data collection, cleaning, and governance

Data is where the majority of practical effort is spent, and where the majority of silent failures originate.

Collection. The training distribution must match the deployment distribution. This is easy to state and hard to satisfy. A model trained on historical hiring decisions inherits the biases of those decisions. A model trained on data from one hospital may not transfer to another with a different patient population, different equipment, or different coding practices. Systematic differences between the two distributions – *distribution shift* – are the most common cause of deployed models underperforming their reported metrics.

Cleaning. Real data contains missing values, duplicates, inconsistent formats, and outright errors. Decisions made during cleaning are consequential and often invisible in the final report. Dropping rows with missing values is not neutral: if missingness correlates with the outcome, dropping those rows introduces bias. Imputing values is not neutral either, and the choice of imputation method can change results substantially. Reproducibility requires documenting these choices, not just the final dataset.

Governance. Who collected the data, under what consent, for what purpose, and with what rights of use? These questions have legal answers in many jurisdictions (GDPR, HIPAA, FERPA, and others) and ethical answers that are not settled by law. Data governance is not a bureaucratic overhead; it is the mechanism by which the difference between a legitimate and an illegitimate system is enforced.

Leakage. The most damaging data problem is leakage: information in the training set that would not be available at prediction time. A model predicting hospital readmission that uses the discharge medication list as a feature is not predicting anything useful, because that information does not exist at admission. Leakage produces spectacular validation metrics and complete failure in deployment. Detecting it requires reasoning about the causal structure of the problem, not just the statistics of the dataset.

8.3 Baselines and iteration

The most reliable way to waste a project is to skip the baseline. Before training a neural network, establish what a trivial model achieves. A majority-class predictor. A logistic regression on a few features. A gradient-boosted tree on the tabular data. If the sophisticated model does not beat the baseline substantially, the sophistication is not buying anything, and the burden of proof lies with complexity.

Baselines serve three purposes. They set a floor below which the model is useless. They diagnose whether the problem is actually hard. And they provide a reference point that prevents the subtle drift into overfitting to the validation set through repeated experimentation.

Iteration itself has a discipline. Each experiment should change one thing. Results should be logged with enough context to be reproducible. Negative results should be recorded, because the same dead end will otherwise be explored repeatedly. The notebook demonstrates this discipline in miniature: each experiment changes one variable, and the resulting plot isolates its effect.

8.4 Deployment, monitoring, and maintenance

A model that has been trained is not a system. Deployment involves serving predictions at acceptable latency, integrating with existing software, handling errors gracefully, and—crucially—monitoring performance after release.

Distribution shift over time. The world changes. User behaviour drifts, economic conditions shift, upstream data sources are modified, and the relationship between features and outcomes evolves. A model trained on last year’s data degrades silently. *Data drift* monitoring tracks changes in the input distribution; *concept drift* monitoring tracks changes in the relationship between inputs and outputs. Both are necessary, and neither is sufficient without periodic retraining.

Feedback loops. Deployed models influence the data they will later be trained on. A recommender system shapes what users see, which shapes what they click, which shapes the next training set. A predictive policing system directs patrols to certain neighbourhoods, generating arrests that confirm the model’s predictions. These loops can amplify initial biases and produce self-fulfilling predictions. They are structural, not accidental, and they require deliberate design to mitigate.

Maintenance. Models are not static artifacts. They require retraining, revalidation, and eventual replacement. The engineers who maintain them change teams, the documentation goes stale, and the original assumptions become invisible. This accumulation of unstated context is what practitioners call *technical debt*, and machine learning systems are unusually prone to it.

8.5 MLOps, reproducibility, and security

A cluster of engineering practices has grown up around the operational side of machine learning, collectively called MLOps.

Reproducibility. A result should be reproducible by someone else, from the code, the data, and the configuration. In practice this requires versioning the code, the data, the environment, and the random seeds. The notebook accompanying this article is written to be reproducible in this sense: seeds are fixed, dependencies are pinned, and each experiment is self-contained.

Experiment tracking. Which model was trained with which hyperparameters on which data, and what did it score? Without systematic tracking, teams repeat experiments and lose the reasoning behind past decisions. Tools exist for this; discipline matters more than the tools.

Security and adversarial input. Models can be attacked. Inputs can be crafted to produce specific errors (adversarial examples), training data can be poisoned, and models can leak information about their training set through their outputs. The notebook’s FGSM demonstration—a digit that looks like a 3 to a human but is classified as an 8 with high confidence—is a small illustration of a large problem. Security is not an afterthought; it is a design requirement for any system handling sensitive decisions.

8.6 Case studies

The pattern asserted above is easier to accept with examples. The ten domains below span different sectors and different stakes, but they illustrate the same underlying claim: the model is one component in a sociotechnical system, and the system usually determines the outcome.

High-stakes decisions. *Healthcare.* Predictive models have shown promise in sepsis detection, readmission prediction, and imaging diagnosis. The recurring difficulties are label noise (diagnoses are themselves uncertain), site variation (models trained at one hospital often fail at another), and workflow integration (a correct prediction that arrives too late to change a decision is worthless). Epic’s widely deployed sepsis model was found in independent evaluations to have substantially lower sensitivity and a much higher alert rate than reported, largely because the validation population differed from the deployment population. The algorithmic advances have been real; the deployment challenges have been harder.

Criminal justice. Risk-assessment tools such as COMPAS are used in some jurisdictions to inform bail and sentencing decisions. An investigation by ProPublica found that the tool’s error rates differed across racial groups, even though overall accuracy was comparable. The dispute that followed is instructive less for its conclusion than for its structure: different definitions of fairness (calibration, equalised error rates, demographic parity) are mathematically incompatible under realistic conditions, so the question “is this tool fair?” cannot be answered without first specifying which fairness criterion matters. The technical literature on this impossibility is substantial, and it is one of the clearest cases where an algorithmic question turns out to be a political one.

Hiring. Amazon built and then abandoned a machine learning recruiting tool after it was found to penalise résumés that included the word “women’s” (as in “women’s chess club captain”). The model had been trained on a decade of historical hiring decisions, which reflected the gender imbalance of the industry. The system learned the pattern faithfully. The failure was not in the model but in the assumption that historical data could serve as a neutral ground truth for a decision that was never neutral. The tool was scrapped in 2018, but similar systems remain in wide use.

Finance. Credit scoring, fraud detection, and algorithmic trading are among the oldest applications of statistical learning. Regulatory requirements — explainability, fairness, auditability — constrain model choice in ways that pure accuracy would not. A slightly less accurate model whose decisions can be explained to a regulator and a customer is often preferred, which is one reason tree-based models and logistic regression remain common in this domain despite the availability of more powerful architectures.

Consumer-facing systems. *Customer service.* Deployed chatbots and voice assistants have repeatedly produced failures that no benchmark captured: fabricated policy details, confident but

incorrect answers, and unsettling responses to vulnerable users. These are not isolated bugs but consequences of deploying a probabilistic system in an open environment where users supply adversarial, unusual, or emotionally charged inputs. The gap between a benchmark and a conversation is the gap between a distribution and the world.

Education. Adaptive learning systems, automated essay scoring, and dropout prediction have all been deployed at scale. The ethical stakes are high: a model that misjudges a student can affect their trajectory for years. Automated essay scoring has been shown to be sensitive to essay length and to reward superficial features such as vocabulary sophistication over argument quality, which means students who learn to game the scorer are rewarded, and the system teaches what it measures. The recurring lesson is that these systems are most useful when they augment teachers rather than replace judgment.

Content moderation. Platforms use classifiers to flag harmful content at a scale no human team could match. The systems are necessary and imperfect: they over-flag satire, under-flag novel forms of abuse, and degrade quickly as adversaries adapt. Moderation is a useful case because it combines scale, adversarial dynamics, and high-stakes errors, and because the human review layer that catches classifier mistakes is itself often under-resourced and psychologically damaging to the workers who staff it.

Scientific discovery. *Biology and materials.* AlphaFold’s protein structure predictions and related systems in materials discovery represent a different mode of success. Here the task is well-posed, the ground truth is provided by experiment, and the model’s output is a hypothesis to be tested rather than a decision to be acted on. Generalisation across distributions matters less than insight into mechanism, and a model that produces a correct prediction for the wrong reason may be worse than no model at all. This is why interpretability research has more traction in the sciences than in commercial deployment.

Weather forecasting. Machine learning models such as GraphCast and Pangu-Weather now produce ten-day forecasts that outperform traditional numerical weather prediction on several benchmarks, at a fraction of the compute cost. This is one of the clearest cases of ML surpassing an established method on a well-defined task with an objective metric and decades of high-quality data. It is worth studying precisely because the conditions are unusually favourable: the physics is known, the data is abundant and clean, and the evaluation is unambiguous.

Mathematics and formal reasoning. Mathematics occupies a peculiar position among AI applications. The ground truth is not empirical but formal: a proof is correct or it is not, a decomposition is valid or it is not, and verification is cheap even when discovery is hard. This is the opposite of the situation in weather forecasting, where the ground truth is abundant, or in hiring, where it is contested. It makes mathematics an unusually favourable domain for machine learning, and the results so far bear this out.

Matrix multiplication. Multiplying two $n \times n$ matrices naively takes $O(n^3)$ operations. In 1969, Volker Strassen showed that 2×2 matrices can be multiplied using 7 multiplications instead of the obvious 8, and that this yields a faster algorithm for large n . The general problem — what is the minimum number of multiplications required for $n \times n$ matrices? — is a long-standing open question in algebraic complexity, and it is equivalent to finding low-rank decompositions of a specific tensor. The search space is astronomically large and the known algorithms are the product of decades of careful human work.

In 2022, DeepMind’s AlphaTensor reformulated the problem as a single-player game and applied the same reinforcement learning machinery that had produced AlphaZero. The system discovered

a 4×4 algorithm using 47 multiplications, improving on the previous best of 49 established by Strassen’s method and subsequent refinements. It also found improved algorithms for several other matrix sizes and for arithmetic over finite fields, some of which had practical relevance for specific hardware. The results are verifiable: each algorithm is a concrete decomposition that can be checked in milliseconds.

Two features of this case deserve emphasis. First, the output is a mathematical object, not a prediction; there is no distribution shift problem and no fairness question, only correctness. Second, the improvements are real but incremental. AlphaTensor did not solve the asymptotic complexity problem, and it does not explain why its algorithms work. It found useful objects in a vast search space, guided by a learned heuristic. That is a genuine achievement and a clear illustration of what current methods are good at: combinatorial search with a verifiable objective.

Knot theory. A different mode of use appeared in a 2021 collaboration between DeepMind and mathematicians, who used machine learning to look for relationships among knot invariants. Invariants fall into two families — algebraic (such as the signature) and geometric (such as hyperbolic volume) — and a relationship between them had been conjectured but not established. The model, trained on a large dataset of knots, suggested that the signature and a particular geometric quantity were related. Mathematicians then proved a theorem formalising the relationship, guided by the pattern the model had surfaced.

This is the *conjecture-generation* model: the ML system does not prove anything, and its output is not a decision or a proof but a hypothesis. The human mathematicians supply the rigor, the intuition about what is worth pursuing, and the verification. The division of labour is instructive. The model is a pattern-finding tool operating on a dataset no human could inspect by hand; the mathematicians provide the conceptual scaffolding that turns a statistical association into a theorem. Neither could have produced the result alone.

Formal theorem proving is a third, rapidly developing area. Systems based on the Lean proof assistant, combined with language models trained to generate proof steps, have begun to solve competition-level problems and to formalise results that previously existed only in human-readable form. The IMO 2024 results, in which a combined system achieved silver-medal-level performance on problems stated in natural language, are a milestone. The broader significance is that mathematics offers a domain where AI outputs can be verified mechanically, which makes it a natural testbed for reasoning methods whose correctness is otherwise difficult to assess.

The common thread across all three cases is that mathematics is where the conditions for AI success are unusually clean: the objective is formal, the ground truth is checkable, and the search space — while vast — is well-defined. This is precisely why the results should be read carefully. Success under favourable conditions does not imply success under unfavourable ones, and the contrast between AlphaTensor’s verifiable combinatorial search and a language model’s unverifiable fluency is the contrast that Section 9 will make in general terms.

Physical systems. *Autonomous vehicles.* Self-driving systems handle the common cases well and struggle with the long tail: unusual road configurations, unpredictable pedestrians, construction zones, and adversarial or merely careless behaviour from other drivers. The core difficulty is not perception in the average case but reliable performance in the distributional extremes, and the safety validation of a system whose failure modes are rare and heterogeneous. Years of optimistic timelines have repeatedly been revised, not because the technology is not advancing but because the last increment of reliability is disproportionately harder than the first.

Manufacturing. Predictive maintenance and quality control are among the most economically successful applications of machine learning, precisely because the environment is controlled, the

sensors are calibrated, and the cost of errors is bounded. A model that predicts bearing failure two days in advance is valuable even if it is sometimes wrong, because the cost of a false positive is an unnecessary inspection. This is the inverse of the healthcare and criminal-justice cases: the same statistical error has very different consequences depending on the domain, and the design of the system should reflect that.

The pattern across all ten is the same. The model is a component. The system is an arrangement of data, incentives, institutions, and people. Improving the component without attending to the system rarely produces the intended result.

No-nonsense takeaway. The AI lifecycle runs from problem framing through data governance, baselines, deployment, monitoring, and maintenance. Most failures occur outside the model: in leakage, distribution shift, feedback loops, and the accumulated debt of unstated assumptions. The algorithm is necessary but rarely sufficient.

Further reading for Part II

The single most important paper for understanding the architecture that dominates current practice is Vaswani et al., “Attention Is All You Need” (2017), which introduced the transformer and is unusually readable for a paper of its influence. For the scaling laws that shaped the current era, Hoffmann et al., “Training Compute-Optimal Large Language Models” (2022)—the Chinchilla paper—is the canonical reference, showing that earlier models were undertrained relative to their size. For the lifecycle material, Sculley et al., “Hidden Technical Debt in Machine Learning Systems” (NeurIPS 2015) is the single most cited paper on the gap between prototypes and deployed systems, and its central diagram—a tiny box labelled “ML code” surrounded by much larger boxes—is worth reproducing in the article. Readers who want to build rather than read should work through Karpathy’s nanoGPT, a minimal GPT implementation in roughly 300 lines that reproduces GPT-2 on a single GPU. For the practical side of training and evaluation, the scikit-learn user guide and the PyTorch tutorials are both well-maintained and assume no prior framework experience.

Part III

Limits, Context, and Future

9 Limits and Open Problems

Parts I and II described what is known and what is built. This part addresses what is not. The three sections that follow differ from the preceding ones in kind: they deal with uncertainty, contestation, and genuinely open questions. This is not a retreat from rigor. It is an extension of it. A grounded introduction that stops at the edge of the textbook would leave the reader unprepared for the fact that a great deal of modern AI rests on empirical findings that are not well understood and on engineering practices that work for reasons nobody can fully explain.

This section catalogues the principal limits. It is organised by kind: theoretical understanding, robustness, causal reasoning, planning, efficiency, interpretability, and environmental cost. The final subsection states plainly what remains unknown, without hedging.

9.1 Theoretical understanding: why does deep learning work?

Deep learning works. The empirical record is not in dispute. What is disputed is why.

The core puzzle is that the classical theory of statistical learning predicts that large models should fail. A model with more parameters than training examples has enough capacity to memorise the training set perfectly. Classical theory says such a model will generalise poorly. Yet deep networks with hundreds of billions of parameters, trained on datasets far smaller than their parameter count, generalise well. The standard explanation—regularisation, early stopping, the implicit bias of gradient descent—provides partial accounts, but none is complete.

Several specific puzzles stand out.

The double-descent phenomenon. Classical theory predicts a U-shaped relationship between model complexity and test error: too simple underfits, too complex overfits. In modern deep networks, test error often continues to fall past the point of interpolation, producing a second descent after the classical U-curve has already risen. The notebook’s overfitting lab shows the classical curve on a small dataset; a larger model on a larger dataset would show the modern one. The theory of why is active research.

The role of architecture. Two networks with the same number of parameters can have very different generalisation behaviour, depending on depth, width, connectivity, and activation function. This sensitivity to architecture is not captured by parameter-count arguments, and it suggests that the geometry of the loss landscape matters more than the raw capacity of the model class.

Grokking. In some settings, a model will appear to have memorised its training data for thousands of steps and then, suddenly, transition to a solution that generalises. The transition is not gradual. It has been documented across many tasks, and no single explanation is established.

Emergence and its measurement. Claims that capabilities emerge abruptly at certain scales have been influential, and some of them are probably artefacts of the metrics used to measure them. If a benchmark awards credit for exact-match answers, a model that produces partly correct answers will appear to score zero until the moment it produces a fully correct one, at which point the score jumps. Whether such jumps reflect genuine phase transitions in capability or discontinuities in measurement is an open question, and one with practical consequences for how seriously to take forecasts of future capability.

The honest summary is that deep learning is an empirical technology with a theoretical scaffolding that is incomplete. The scaffolding is real—gradient descent, the universal approximation theorem, statistical learning theory all contribute—but it does not yet explain the phenomenon it was meant to describe.

9.2 Robustness, distribution shift, and adversarial examples

A model’s performance on its training distribution tells you little about its performance elsewhere. This is not a subtle point; it is the single most important practical limit. Figure 25 shows one form this limit takes.

Section 8 treated distribution shift operationally. Here we note its deeper form. Deep networks are surprisingly brittle to small, deliberate perturbations. An image that is classified correctly at one pixel-level perturbation can be misclassified with high confidence at another, even when the perturbation is invisible to a human. This is the adversarial example phenomenon, demonstrated in the notebook with FGSM: a digit that looks like a 3 to a human reader is classified as an 8 with 99% confidence. The perturbation required to produce this is small enough to be undetectable at normal viewing. Figure 26 shows two forms of distribution shift: rotation and noise.

Adversarial examples matter for two reasons. First, they are a security problem: any system that accepts inputs from an adversary—a spam filter, a malware detector, a biometric authentication system—is potentially vulnerable. Second, and more fundamentally, they reveal that the model’s decision boundary is not aligned with the human concept it appears to represent. The model has

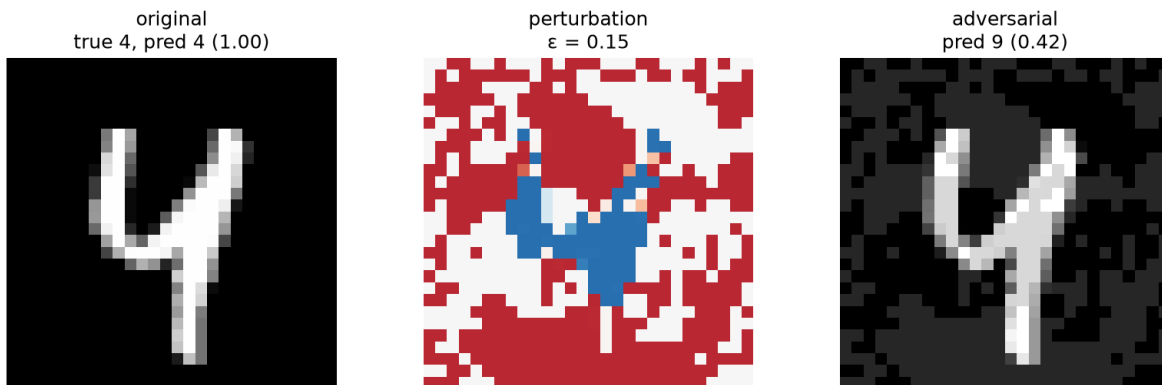


Figure 25: An adversarial example generated by the Fast Gradient Sign Method. Left: the original digit, correctly classified. Centre: the perturbation, scaled for visibility — the actual perturbation is bounded by $\epsilon = 0.15$ and is imperceptible to a human reader. Right: the perturbed digit, confidently misclassified. The model’s decision boundary is not aligned with the human concept it appears to represent; the demonstration makes this visible in ten lines of code. Generated by the companion notebook, Part 9.

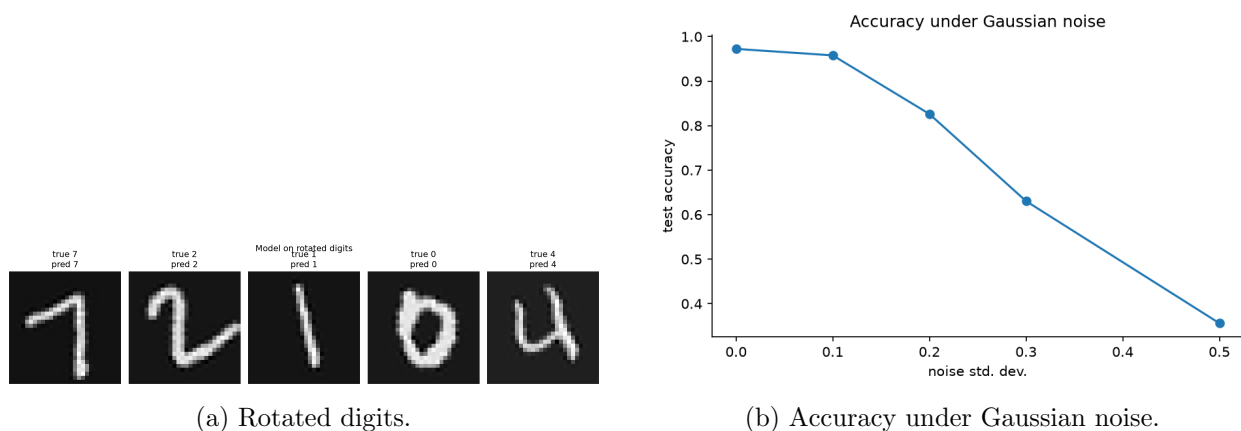


Figure 26: Two forms of distribution shift. Left: rotating the test set by 30 degrees collapses accuracy from 99% to roughly 70%, even though a human reads the rotated digits without effort. Right: accuracy degrades smoothly with noise, then abruptly. The model has learned pixel statistics, not the concept of a digit. Generated by the companion notebook, Part 9.

learned features that happen to correlate with the label on the training distribution, not features that constitute the concept. This is the same insight as the husky-and-snow example, but sharpened: the model is not wrong about a hard case; it is correct on easy cases for the wrong reasons, and we can prove it by manipulating the reasons.

Whether adversarial robustness is achievable in general is an open question. A substantial body of work suggests that achieving robustness against all perturbations comes at a cost in accuracy on clean data, and that the tradeoff may be fundamental rather than an engineering limitation. This is one of the more active theoretical debates in the field.

9.3 Causality and counterfactuals

Machine learning models learn correlations. They do not, in general, learn causal relationships. The distinction matters whenever a decision is being made, because decisions are interventions, and interventions require causal reasoning.

Consider a model that predicts hospital readmission risk. It has learned that patients who receive a particular medication have worse outcomes. Should the hospital stop prescribing the medication? The model cannot answer this question. The correlation may reflect the medication causing harm, or it may reflect the medication being prescribed to sicker patients. Only a causal analysis can distinguish the two, and the correlation alone does not.

Causal inference is a mature field, and combining it with modern representation learning is an active research area. The challenges are substantial: causal structure is generally unobserved, experiments are often impossible, and the assumptions required for identification are strong and unverifiable from observational data alone. Current systems mostly operate in the correlation regime and leave causal questions to human analysts.

The practical consequence is that models are best at prediction and worst at action. A model that predicts what will happen under the existing regime can be useful. A model that recommends what to do requires something more, and it is not clear that current architectures provide it.

9.4 Reasoning, planning, and common sense

Large language models produce text that is locally coherent and often globally plausible, but they are unreliable on tasks that require multi-step reasoning, planning, or common-sense knowledge about the physical and social world.

Chain-of-thought prompting—asking the model to produce intermediate reasoning steps—improves performance on multi-step problems. It does not make the reasoning reliable. Models frequently produce chains that are coherent but wrong, or arrive at correct answers through reasoning steps that are themselves invalid. The relationship between a model’s stated reasoning and its actual computation is not transparent.

Planning is harder still. A plan is a sequence of actions that achieves a goal in a world with constraints, and generating a valid plan requires representing those constraints, checking them, and revising when they are violated. Current agents do this in narrow environments and fail in open ones. Long-horizon tasks, where errors compound over many steps, remain particularly difficult.

Common sense is the least well understood of these. A human reader knows that a glass dropped on a stone floor will break, that a person who is asleep cannot answer the door, and that a promise made under coercion is not normally binding. No current system has this background knowledge in any systematic form. What language models have is a statistical approximation of what people write about such situations, which frequently correlates with the right answer and sometimes produces the wrong one confidently.

9.5 Sample efficiency and continual learning

Humans learn from a handful of examples. Current models typically require thousands or millions. A child shown a single picture of a giraffe can recognise giraffes thereafter; a state-of-the-art vision model may need hundreds of examples per class. This gap in sample efficiency is one of the sharpest contrasts between biological and artificial learning.

A related limitation is continual learning. Humans learn new skills without forgetting old ones. Neural networks do not: training on a new task tends to overwrite the weights that encoded the previous one, a phenomenon called catastrophic forgetting. Techniques such as rehearsal (rehearsing old examples), elastic weight consolidation, and parameter isolation mitigate the problem but do not solve it. The result is that deployed models are typically frozen or retrained from scratch rather than continuously updated, which is a significant operational constraint.

Whether these limitations reflect a fundamental gap between current architectures and biological learning, or a temporary artefact of training methods, is an open question. The honest answer is that nobody knows.

9.6 Interpretability and explainability

A model’s parameters are numbers. Its internal representations are vectors. Its computations are matrix multiplications. None of this is accessible to human inspection in any straightforward way, and the gap between what a model does and what a human can say about it is the interpretability problem.

Two approaches to closing the gap are active. *Post-hoc* explanation methods such as LIME and SHAP produce approximations of why a specific prediction was made, usually by fitting a simpler model locally. These methods are widely used and frequently criticised, because the explanations they produce are approximations that may or may not reflect the model’s actual computation. *Mechanistic* interpretability aims to understand the model’s internal circuits directly, by identifying the components responsible for specific behaviours. The results so far are impressive in narrow settings—certain induction heads in small transformers have been characterised—and incomplete in general.

Interpretability matters for three reasons. It is required for accountability: a decision that cannot be explained cannot be appealed. It is required for trust: users who do not understand a system’s reasoning cannot calibrate their reliance on it. And it is required for safety: a system whose internal states cannot be inspected cannot be audited for hidden objectives or failure modes. The tradeoff between capability and interpretability is often stated as if it were fundamental. It is more accurate to say that the field has invested far more in capability than in understanding, and the consequences of that imbalance are only beginning to be felt.

9.7 Energy, compute, and environmental cost

Training a frontier model consumes a substantial amount of electricity, and the aggregate energy use of the AI industry has grown rapidly. Exact figures are contested and often proprietary, but the order of magnitude is not: individual training runs consume gigawatt-hours, and inference at scale is a continuing operational cost.

Three complications make the environmental picture difficult to assess. First, reported figures vary widely depending on what is included: training, inference, cooling, hardware manufacture, and the energy intensity of the local grid. Second, the industry is opaque: many organisations do not publish energy figures, and those that do release them selectively. Third, efficiency improvements have partly offset the growth in total compute, but not enough to reverse the trend.

The environmental cost is a limit in a different sense from the others in this section. It is not a theoretical barrier but a practical constraint, and it is one that may become binding before the theoretical barriers do. If the trajectory of the past decade continues, the energy and capital requirements of frontier training may concentrate capability in a small number of organisations. The implications of that concentration are discussed in Section 11.

9.8 What we do not know

It is worth stating plainly, and without hedging, the things that remain genuinely unresolved.

- We do not have a theory of why deep learning generalises. The empirical record is consistent with several explanations, none of which is complete.
- We do not know whether adversarial robustness is achievable at reasonable cost.
- We do not know whether current architectures can support reliable multi-step reasoning, or whether something structurally different is required.
- We do not know whether sample efficiency can be improved by orders of magnitude, or whether the gap with biological learning is fundamental.
- We do not know how to build systems whose internal computations can be inspected and verified.
- We do not know whether scaling will continue to produce improvements, or whether it is approaching a regime of diminishing returns.
- We do not know whether the current paradigm—training large models on large datasets—is a path to general capability, a useful engineering technique that will plateau, or something in between.

This list is not a counsel of despair. It is a description of a young field. Many of the technologies the reader relies on daily were developed in the absence of complete theory. The point is not that AI is inherently mysterious; it is that the gap between capability and understanding is wider here than in most engineering disciplines, and that gap has consequences for how confidently anyone—researcher, policymaker, or critic—should speak about what comes next.

No-nonsense takeaway. Deep learning works without a complete theory; models are brittle to distribution shift and adversarial inputs; they learn correlations, not causes; their reasoning and planning are unreliable; their sample efficiency is far below biological learning; their internals are largely opaque; and their energy costs are substantial. The honest response is neither to dismiss the technology nor to treat it as understood, but to be precise about which questions are open and which are settled.

10 Insights from Mathematics and Physics

The previous section catalogued what is not understood about modern AI. This section asks whether other disciplines can help. Mathematics and physics have contributed to machine learning since its inception, and the connection has produced both genuine results and a great deal of overclaiming. The aim here is to separate the two.

Three kinds of contribution can be distinguished, and it is worth naming them at the outset because they are frequently conflated. First, *established results*: theorems and physical principles that constrain what any learning system can do. Second, *productive analogies*: frameworks that guide intuition and suggest new methods but do not constitute explanations. Third, *speculation*: claims that intelligence is a physical phenomenon of a particular kind, which are philosophically interesting and empirically unsupported. The first category is small but important. The second is large and useful. The third is large and dangerous.

10.1 Statistical mechanics and phase transitions

The most substantive connection between physics and machine learning is statistical mechanics. In the 1980s, physicists studying disordered magnetic systems—spin glasses—developed mathematical tools for analysing systems with many interacting degrees of freedom and frustrated interactions. These tools turned out to apply to neural networks.

John Hopfield’s 1982 model of associative memory was explicitly borrowed from spin-glass physics: neurons were treated as spins, connections as couplings, and memory retrieval as relaxation to a low-energy state. The model is simple, and its capacity is limited, but it established that the language of statistical physics could describe learning and memory. The 2024 Nobel Prize in Physics, awarded to Hopfield and Geoffrey Hinton, recognised this line of work as foundational to modern machine learning.

The more technically consequential development was the application of the *replica method*—a technique for averaging over disorder in spin glasses—to the perceptron and related models. Elizabeth Gardner’s 1988 work computed the storage capacity of a perceptron analytically, and the result matched simulations. This was not a metaphor; it was a theorem about a learning system, obtained by importing a technique from physics. Subsequent work extended the approach to more complex architectures, though the mathematics becomes intractable beyond relatively simple models.

The general lesson is that neural networks are high-dimensional dynamical systems, and statistical mechanics is the branch of physics that studies such systems. Where the mathematics is tractable, it yields genuine results. Where it is not—which is most of the time for modern deep networks—the connection remains suggestive rather than explanatory. The energy-landscape picture of learning, in which training moves a system toward low-loss configurations, is a productive analogy and a legitimate research program, but it is not yet a theory of deep learning.

10.2 The strange geometry of high dimensions

A second mathematical contribution is less often emphasised but more directly relevant to why modern methods work: high-dimensional spaces do not behave like low-dimensional ones. The intuitions we develop from two- and three-dimensional geometry systematically mislead.

Three facts are worth stating, because each is counterintuitive and each matters for machine learning.

First, in high dimensions, random vectors are nearly orthogonal. If you draw two random directions in a 1000-dimensional space, the angle between them is close to 90 degrees with overwhelming probability. This is why embeddings can represent many distinct concepts in a relatively low-dimensional space: there is far more “room” in high dimensions than intuition suggests.

Second, most of the volume of a high-dimensional ball lies near its surface, not its centre. A thin shell at the boundary contains almost all the mass. This is why distance-based intuitions fail: the

nearest and farthest neighbours of a random point become nearly equidistant as dimension grows, a phenomenon known as the curse of dimensionality.

Third, concentration of measure: functions of many independent random variables are, with high probability, close to their expected values. A high-dimensional random vector is almost always near the surface of the sphere, and almost never near its centre. This is why random initialisation of neural networks works: random weights in high dimensions produce well-behaved, nearly-orthogonal transformations.

These facts are not merely curiosities. They explain why the parameter spaces of modern models are navigable, why gradient descent does not get stuck as often as low-dimensional intuition predicts, and why generalisation in overparameterised models is less paradoxical than it first appears. They are established mathematics, and they constitute one of the clearest cases where a mathematical insight directly illuminates a practical phenomenon.

10.3 Dynamical systems, optimisation, and criticality

The training of a neural network is a dynamical process, and the tools of dynamical systems theory apply—at least in principle. The loss landscape is a high-dimensional surface, gradient descent is a trajectory on that surface, and questions about convergence and stability are questions about the dynamics.

Several empirical findings from this perspective are worth noting. In high-dimensional non-convex landscapes, saddle points are far more common than local minima; gradient descent escapes saddles relatively easily, which is part of the reason it works so well in practice. Different trained models often lie in connected basins—the “mode connectivity” result—meaning that a simple path through parameter space connects one solution to another. This is surprising from a naive landscape picture and suggests that the geometry of the loss surface is more benign than classical theory would predict.

The concept of *criticality*—the boundary between ordered and disordered phases—has been proposed as an organising principle. Some evidence suggests that trained networks sit near a critical regime, and that this position is associated with desirable properties such as trainability and generalisation. The evidence is suggestive, not conclusive, and the analogy to physical criticality should be treated as a hypothesis rather than a result.

Power laws appear throughout machine learning: scaling laws for loss, distributions of attention patterns, and the spectral properties of weight matrices. Power laws are characteristic of systems near critical points, and their ubiquity in ML is one of the more intriguing empirical patterns of the past decade. Whether it reflects a deep connection to critical phenomena or is an artefact of how these quantities are measured is an open question.

10.4 Thermodynamics and the physical cost of computation

Information processing is a physical process, and physical processes have thermodynamic costs. This connection, established by Rolf Landauer in the 1960s, gives hard constraints on what any computational system can do.

Landauer’s principle states that erasing one bit of information dissipates at least $kT \ln 2$ of energy, where k is Boltzmann’s constant and T is temperature. The bound is tiny in absolute terms, and modern hardware operates far above it, but the principle is fundamental: any irreversible computation has a minimum energy cost. Reversible computation, in principle, could avoid this cost, but practical systems are far from that regime.

For AI specifically, the thermodynamic perspective matters in two ways. First, it bounds the energy efficiency that any learning or inference system can achieve, which is relevant to the energy discussion in Section 9. Second, it connects computation to the physical world in a way that is often lost in abstract discussions of algorithms. A model running on a GPU is a physical system dissipating heat, and the constraints of thermodynamics apply to it as surely as they apply to a steam engine.

The connection is genuine but limited. It tells us what is impossible in principle; it does not tell us how to build a system that approaches the limit, and it does not explain intelligence.

10.5 What physics can and cannot explain about intelligence

The temptation, when physics meets AI, is to reach for grand synthesis: to claim that intelligence is a phase of matter, that learning is thermodynamic relaxation, that consciousness is a physical phenomenon of a specific kind. Some of these claims may eventually be vindicated. None of them is established.

What physics has contributed to AI is real but bounded. It has supplied mathematical tools (the replica method, the language of phase transitions, the geometry of high-dimensional spaces). It has supplied constraints (Landauer’s principle, the limits of computation). It has supplied productive analogies (energy landscapes, criticality) that have guided research and generated testable predictions. These are substantial contributions.

What it has not supplied is an explanation of intelligence. There is no physical theory that predicts the emergence of learning in a network, no thermodynamic principle that determines the architecture of a language model, and no established result that connects the loss surface of a transformer to a physical system in any rigorous sense. Claims to the contrary should be read as speculation dressed in the language of physics, which is a different thing from physics itself.

The honest position, and the one this article adopts, is that mathematics and physics are indispensable collaborators in the study of intelligence and insufficient on their own. They clarify which questions are well-posed and which constraints are real. They do not, so far, explain what they help describe.

No-nonsense takeaway. Statistical mechanics supplied the tools that gave us Hopfield networks and analytic results about perceptrons; high-dimensional geometry explains why overparameterised models work; dynamical systems and criticality provide useful analogies that remain hypotheses rather than results; thermodynamics gives hard limits on computation. Physics and mathematics are collaborators, not explanations. Claims that intelligence has been reduced to physics are speculation, not science.

11 Ethics, Safety, and Governance

The limits in Section 9 were technical: what current systems cannot do, and what we do not understand about them. This section addresses a different class of concern. Even when a system works as designed, it can cause harm, produce injustice, or concentrate power in ways that demand justification. These questions are not secondary to the technical ones. In many applications they are the reason the technical questions matter.

Three kinds of claim appear in this section, and they should be distinguished throughout. First, *documented harms*: cases where deployed systems have caused measurable damage, with evidence in the public record. Second, *contested claims*: consequences that are plausible and debated, where

the evidence is incomplete and reasonable people disagree. Third, *speculation*: concerns about futures that have not materialised, where the argument is often about plausibility rather than evidence. Treating the third as if it were the first, or dismissing the first because the third is speculative, are symmetrical errors.

11.1 Bias, fairness, and discrimination

The most thoroughly documented harms from deployed AI involve discrimination. The COMPAS case, discussed in Section 8, established that a risk-assessment tool can have comparable accuracy across groups while producing substantially different error rates. Amazon’s recruiting tool, similarly, penalised résumés associated with women. Both are documented, both were investigated, and both are instructive about the general mechanism: a model trained on historical data learns historical patterns, including patterns of exclusion.

The conceptual core is that fairness is not one property but several, and they are mutually incompatible in realistic settings. Three definitions are commonly proposed: *calibration* (predicted probabilities mean the same thing across groups), *equalised odds* (error rates are equal across groups), and *demographic parity* (outcomes are distributed equally across groups). ProPublica’s critique of COMPAS appealed to equalised odds; the tool’s developers responded by pointing to calibration. Both sides were correct. The impossibility results—established by Kleinberg, Mullainathan, and Raghavan and by Chouldechova in 2016–2017—show that when base rates differ across groups, no classifier can satisfy all three definitions simultaneously, except in degenerate cases.

This is a mathematical result, not a political one, and its implications are uncomfortable. It means that any claim that an algorithm is “fair” must specify which definition of fairness it satisfies. It also means that the choice among definitions is a normative choice that cannot be delegated to the algorithm. The technical community can clarify the tradeoffs; it cannot resolve them.

A practical consequence is that bias mitigation is not purely a technical exercise. Removing a protected attribute from the feature set does not remove its influence, because correlated features carry the same information. Rebalancing the training data changes one fairness metric at the expense of another. The question of which metric to prioritise, and whose interests to protect, is a question about values, and it belongs to institutions with the legitimacy to make those choices.

11.2 Privacy, surveillance, and data rights

Machine learning is trained on data, and much of that data is about people. Three distinct concerns arise.

Consent and collection. Most people whose data trains modern models did not knowingly contribute it. Web-scraped text, public photographs, and aggregated user behaviour form the substrate of many systems, and the question of whether this constitutes legitimate use is legally unsettled and ethically contested. The GDPR in Europe, various state laws in the United States, and similar frameworks elsewhere establish different answers; none is universally accepted.

Inference and re-identification. Models can infer sensitive attributes from apparently innocuous data. Purchasing patterns predict pregnancy; location traces predict religious affiliation; writing style predicts sexual orientation with substantial accuracy in some studies. Even when the original data does not contain the sensitive attribute, a trained model may recover it. This means that privacy is not preserved by simply excluding sensitive columns from a dataset.

Memorisation. Large language models can reproduce verbatim portions of their training data, including personal information. This has been demonstrated repeatedly, and it is a direct consequence of how these models are trained: a model that compresses its training data well retains details of that data. Removing such content after training is difficult; preventing it during training requires either filtering the corpus or changing the training objective, both of which have costs.

The practical position is that privacy is not a binary property, and that the relevant questions—who collected the data, under what terms, for what purpose, with what rights of redress—are institutional rather than technical. Technical tools (differential privacy, federated learning, data minimisation) reduce certain risks, but they do not substitute for governance.

11.3 Misinformation, manipulation, and synthetic media

Generative models can produce convincing text, images, audio, and video at negligible cost. The consequences are already visible: synthetic voices used in fraud, fabricated images circulated as evidence, and automated text used to flood information channels.

Three observations are worth making, because the discourse around this topic is frequently imprecise.

First, misinformation is not new, and AI is not its primary driver. Coordinated disinformation campaigns, cheap fabrication, and sensational media predate the current generation of models. What has changed is the *cost* of producing plausible fabrications and the *scale* at which they can be distributed. This is a quantitative change with qualitative consequences, not an entirely new phenomenon.

Second, provenance is technically addressable in part. Content credentials, watermarking, and cryptographic signing of media can establish that a piece of content came from a particular source. None of these is foolproof, and none addresses the harder problem of content whose provenance is unknown. But they are more promising than detection, which is fundamentally an adversarial problem: as soon as a detector works, the generator can be trained against it.

Third, the effects on public discourse are contested and empirical questions are being studied. Claims that AI has caused a particular political outcome are usually overdetermined, and the causal attribution is difficult. The honest position is that the risks are real, the mechanisms are partially understood, and the magnitude is debated.

11.4 Labor, economy, and inequality

The economic effects of AI are the subject of substantial uncertainty and substantial disagreement. Three positions are represented in the literature.

Continuation. AI follows the pattern of previous automation: it displaces specific tasks, creates demand for new skills, and raises aggregate productivity. The transition is disruptive for affected workers but not categorically different from prior technological transitions.

Discontinuity. AI differs from prior automation in that it targets cognitive and creative tasks, not only routine manual ones. Legal work, medical diagnosis, software development, and content production are affected. Because these occupations span the wage distribution, the transition may be broader than previous ones.

Concentration. The distinctive feature of the current wave may be less about task displacement than about the economics of production. Training frontier models requires capital, compute, and

data at a scale few organisations can match, which tends to concentrate capability in a small number of firms and countries. The distributional consequences of this concentration, rather than the labour displacement itself, may be the more consequential effect.

The evidence does not settle the matter. Productivity gains have been documented in specific settings; aggregate effects are harder to identify, partly because the technology is recent and partly because macroeconomic measurement is difficult. Anyone claiming with confidence to know the employment effects over the next decade is extrapolating, not reporting.

11.5 Autonomy, accountability, and lethal applications

Systems that make or inform consequential decisions raise a long-standing question: who is responsible when something goes wrong? This question has a long pedigree in the ethics of technology, and AI makes it sharper by inserting a layer of computation between the human decision and its consequence.

Two aspects deserve attention. The first concerns *attribution*: if a model recommends a decision and a human approves it, accountability is diffuse. If the human does not understand the model’s reasoning—and interpretability remains limited, as Section 9 discussed—approval becomes a formality rather than a genuine exercise of judgment. This is sometimes called the “responsibility gap.”

The second concerns *lethal applications*. Autonomous weapons systems and AI-assisted targeting raise questions that the technical community cannot settle on its own: under what conditions, if any, can lethal force be delegated to a machine? The debate is ongoing, and positions range from categorical prohibition to conditional acceptance subject to human oversight. What is clear is that the choices will be made by governments, and that the technical capabilities already exist or will shortly.

11.6 Alignment and existential risk: separating evidence from speculation

No topic in this section requires more care than the question of catastrophic or existential risk from advanced AI. The topic attracts both strong claims and strong dismissals, and both tend to be insufficiently calibrated.

What is credible. A serious body of work, associated with researchers across major labs and academic institutions, argues that aligning highly capable systems with human values is a difficult technical problem, that current methods (RLHF, constitutional AI, and similar) provide limited guarantees, and that as systems become more capable the difficulty increases. This is a reasonable position and is not the same as asserting that catastrophe is likely. The technical content—that current alignment methods are heuristic, that we lack tools to verify the objectives of a sufficiently capable system—is well established.

What is speculative. Claims about specific timelines, about the probability of loss of control, and about the mechanism by which catastrophe would occur rest on assumptions that cannot be verified. The history of AI, as Section 3 noted, includes repeated overconfidence about timelines and capabilities. This does not refute long-term concerns, but it means that confident assertions about particular futures should be read as arguments, not as findings.

What is neither. Some contributions to this debate are neither rigorous analysis nor responsible speculation; they are rhetorical moves designed to direct attention, funding, or regulation. Recognising this requires no special expertise. The test is whether a claim can be stated precisely,

whether it makes a risk more or less likely, and whether the person making it distinguishes their assumptions from their conclusions.

The position this article adopts is that long-term risk is a legitimate subject of serious research, that the technical work on alignment is genuinely difficult, and that claims about catastrophic outcomes should be evaluated with the same standards applied to any other empirical question: what is the mechanism, what is the evidence, and what would change your mind?

11.7 Regulation, standards, and global governance

Governance of AI is a fast-moving area, and any specifics will date quickly. What is stable is the structure of the problem.

Approaches. Regulatory approaches divide broadly into *risk-based frameworks*, which impose stricter requirements on higher-risk applications (the model adopted by the EU AI Act), and *sectoral regulation*, which extends existing rules in healthcare, finance, and other regulated domains to AI systems deployed within them. Both coexist in most jurisdictions, and both face the same difficulty: the technology changes faster than the regulatory process.

Standards. Technical standards (the NIST AI Risk Management Framework and its international counterparts) provide a common vocabulary for risk assessment. They are voluntary in most cases but frequently become de facto requirements through procurement and liability.

Global coordination. AI development is international, and regulatory divergence creates arbitrage: organisations can develop in one jurisdiction and deploy in another. Meaningful coordination on safety-critical issues, particularly in areas with military applications, has so far been limited. Whether this changes is a political question, not a technical one.

Legitimacy. The hardest question is not what regulation should require but who decides. AI governance currently involves a small number of governments, a small number of firms, and a growing community of civil society organisations. The decisions being made—about acceptable risk, about permitted uses, about the distribution of benefits—affect populations that have little voice in the process. Figure 27 shows the layers of governance instruments and the actors who participate at each level.

11.8 Principles versus practice

A striking feature of the past decade is the gap between the volume of ethical principles articulated and the volume of practices changed. Hundreds of organisations have published responsible AI principles; comparatively few have altered their engineering practices, their incentives, or their deployment decisions in response. A principled statement that is not accompanied by enforcement, accountability, or consequence is not a governance mechanism. It is a communications artifact.

This is not a reason for cynicism. It is a reason for attention to the mechanics of accountability: who is responsible when a system causes harm, what the affected party can do about it, and what changes as a result. These questions are rarely addressed in principle documents, and they are the ones that matter. The technical community can contribute to answering them, but only within institutions that have the authority and the incentive to enforce answers.



Figure 27: The AI governance landscape. Instruments are arranged by level of authority, from international coordination at the top to organisational practice at the bottom; each level conditions the levels below it, but not strictly. Actors on the right participate at multiple levels. The gap between principles and practice, discussed above, is the gap between the upper layers where statements are produced and the lower layer where behaviour actually changes.

No-nonsense takeaway. Bias and fairness raise documented harms and a genuine impossibility result: several definitions of fairness cannot be satisfied at once, and choosing among them is a normative decision. Privacy, misinformation, labour, accountability, and lethal applications each raise distinct questions, some documented, some contested, some speculative. Long-term risk is a legitimate research subject and should be evaluated with the same standards as any empirical claim. The gap between published principles and changed practice is the central governance problem.

12 Future Trajectories and Challenges

This section deals with what has not happened yet. That requires a different discipline from the preceding sections. Where the earlier material could cite results and evidence, futures can only be reasoned about, and reasoning about the future has a poor track record in this field. Section 3 recounted the Dartmouth workshop’s two-month estimate, and it is worth keeping that episode in mind.

Throughout this section, three kinds of claim appear, and they should be distinguished. *Trends* are continuations of measurable past behaviour: compute grows, costs fall, capability on benchmarks improves. *Extrapolations* project trends forward under assumptions that may or may not hold. *Scenarios* describe possible futures coherently, without claiming that they are likely. The section closes with scenarios rather than predictions, because that is the only honest form the argument can take.

12.1 AGI: definitions, debates, and timelines

Section 2 introduced the distinction between narrow and general intelligence and noted that AGI does not exist. The debate about whether and when it might is one of the most contentious in the field, and it is worth understanding why the disagreement is so persistent.

The difficulty begins with the definition. AGI is usually defined as a system capable of performing any intellectual task a human can, but “any” is doing a great deal of work. Does it require consciousness? Common sense? Physical embodiment? The ability to learn a genuinely new skill without task-specific training? Different definitions imply different answers to whether current systems are close, far, or on the wrong track entirely.

Two broad positions dominate the discussion.

The scaling position. Current architectures, trained on more data with more compute, will continue to improve and will eventually reach human-level generality. The evidence cited is the empirical regularity of scaling laws, the breadth of capability that language models already display, and the absence of a clear barrier. The predicted timelines vary; the strongest versions anticipate systems arriving within a decade.

The architectural position. Current systems are missing something structural. They learn correlations rather than causes (as Section 9 discussed), they lack persistent memory and reliable long-horizon planning, and their apparent breadth is shallower than it appears. The evidence cited is the brittleness documented in the limits section, the failure of models on tasks that require genuine reasoning, and the observation that scaling has not yet produced a system that can operate reliably in open environments. This position does not necessarily deny that progress will continue, only that scale alone will not be sufficient.

Both positions are defensible, and the disagreement cannot be settled by argument. It will be settled, if at all, by results. What can be said is that neither side has a track record of accurate prediction, and that confident timelines should be treated as expressions of a research program rather than as forecasts. The history of the field provides ample precedent for both overconfidence in progress and premature declarations of a plateau.

12.2 Human–AI collaboration and augmentation

A less speculative direction of change is the integration of AI systems into existing work. This is already happening, and it is easier to reason about because it is observable.

Three patterns are emerging. First, *assistance*: the model performs a subtask, and a human reviews the output. Code completion, draft generation, and summarisation are examples. The value depends on the human’s ability to catch errors, which depends on the task. Where errors are easy to spot, assistance is beneficial; where they are subtle, it can be actively harmful, because it invites unwarranted confidence.

Second, *delegation with oversight*: the model performs an entire task, and a human is responsible for the outcome. This is the pattern in many customer service and content moderation systems. Its principal risk, discussed in Section 11, is that oversight becomes a formality when the volume of decisions exceeds what humans can review.

Third, *collaboration*: human and model work together iteratively, with neither simply reviewing the other. Scientific discovery, mathematical research, and some forms of design are beginning to follow this pattern. It is the most promising and the least well understood.

The common thread is that the value of AI in work is not simply productivity. It depends on the division of labour between human and machine judgment, and on the design of the interface between them. Systems that replace judgment rather than augment it are more efficient and more fragile. This is an engineering question as much as an ethical one.

12.3 AI for science, health, and climate

The domains where AI has produced the clearest genuine advances are those with formal or empirical ground truth and abundant data. This is not coincidence; it is the pattern established throughout this article.

In the sciences, machine learning has become a standard tool in protein structure prediction, materials discovery, and analysis of large observational datasets. The pattern is likely to continue, with two caveats. First, the value is concentrated where the problem is well-posed and evaluation is objective. Second, the output is a hypothesis, not a conclusion; the scientific work of verification remains human, and the tools do not remove the need for domain expertise.

In medicine, the situation is more complicated. Diagnostic performance on curated benchmark data is often excellent, and clinical deployment is often disappointing, for the reasons discussed in Section 8: label noise, site variation, and workflow integration. Progress is real but slower than the benchmark numbers suggest, and the gap between research and practice is where the field's attention should go.

In climate and energy, AI is used for forecasting, optimisation of grid load, materials research for batteries and solar cells, and environmental monitoring. The same caveats apply: the value is greatest where the problem is well-posed and the data is reliable. AI does not substitute for policy or for the physical constraints of the energy transition.

12.4 Education and public understanding

The education question is not whether AI will be used in teaching; it already is. The question is how.

Two failures are visible. The first is automation of assessment without attention to what assessment measures; automated essay scoring is a documented case, discussed in Section 8. The second is the use of AI systems as substitute teachers in under-resourced settings, where the technology is applied precisely because human instruction is unavailable, with predictable results.

The more promising uses augment human instruction: tutoring systems that adapt to the learner, tools that free teachers from administrative work, and simulations that allow exploration of problems otherwise inaccessible. These require investment in the human side of the system, which is often where the funding is not.

Public understanding is a related challenge. AI literacy is not the ability to use tools; it is the ability to evaluate claims about what those tools can do. This article has tried to support that ability, but it is a small contribution to a larger need. The gap between public discourse and technical reality is not closing on its own, and the institutions with the incentive to close it are not always the ones with the capability.

12.5 Open research directions

Several research directions are active and consequential; listing them makes concrete what the field is working on.

- *Efficient architectures.* Attention is quadratic; long-context models are expensive. Alternatives (state-space models, linear attention, mixture-of-experts) are being developed and evaluated.
- *Reasoning and verification.* Reliable multi-step reasoning is the central limitation of current systems, and approaches ranging from tool use to formal verification to new training objectives are being pursued.
- *Interpretability.* Mechanistic understanding of model internals is a long-term research program with early results in narrow settings.
- *Alignment.* Making systems reliably follow human intent under distributional shift, and verifying that they do, is both a technical and a methodological problem.
- *Causal learning.* Combining representation learning with causal inference remains largely open, and the payoff for decision-making systems would be substantial.
- *Evaluation.* Better methods for measuring capability, detecting contamination of benchmarks, and assessing out-of-distribution behaviour.

The relative importance of these directions depends on the assumptions discussed above. If scale is sufficient, alignment and interpretability are the critical paths. If architecture is the bottleneck, the reasoning and causal directions matter more. Both sets of work are proceeding, and the question of which will matter most is itself unresolved.

12.6 Scenarios: plausible, preferable, avoidable

The final move of this section is to describe futures rather than predict them. Three scenarios illustrate the range, each contingent on choices rather than on technology alone.

A continuation scenario. AI develops as a general-purpose technology: capability improves gradually, adoption spreads unevenly, some jobs change, some disappear, new ones appear. Benefits accrue disproportionately to those with capital and skills, as they have for previous general-purpose technologies. Regulation develops slowly and unevenly. This scenario is not utopian or dystopian; it is recognisable from the history of the internet.

A concentration scenario. Capability remains dependent on compute and data at a scale that only a few organisations can command. Benefits and control concentrate correspondingly. Access is mediated by a small number of firms and states. The technology works well; the distribution is the problem. This scenario is already partly visible and is the one most often omitted from public discussion, because it does not fit the frames of either progress or catastrophe.

A disruption scenario. Capability advances faster than institutions can adapt. Labour displacement outpaces the creation of new roles; governance lags; misinformation and fraud scale faster than countermeasures. This scenario is not the same as an existential risk scenario and does not require superintelligence; it is what happens when a powerful technology diffuses faster than the frameworks for managing its consequences.

Which of these unfolds depends on decisions being made now: about access, about regulation, about investment in public institutions and human capability. The technology does not determine the outcome; the choices do. That is the article’s final claim, and the conclusion returns to it.

No-nonsense takeaway. Claims about AGI timelines cannot be settled by argument, and both the scaling and architectural positions are defensible. The nearer term is more tractable: integration into work, science, health, and education will continue, with value concentrated where problems are well-posed and oversight is genuine. The future is contingent on choices, not foreordained by the technology.

13 Conclusion: Participating in the Trajectory

This article began with an observation: AI is simultaneously overdiscussed and underexplained. Thirteen sections later, the observation stands, but the reader is now equipped to do something about it. This closing section does three things: it consolidates what has been established, offers a practical method for evaluating new claims, and returns to the question of participation.

13.1 Summary of key takeaways

The argument of the article can be compressed into seven claims.

1. *AI is a family of technologies, not a single thing.* The terms AI, machine learning, deep learning, and generative AI are nested, and confusion among them is the source of much misunderstanding.
2. *Modern AI is statistical, narrow, and non-conscious.* It learns patterns from data. It does not reason symbolically in the classical sense, it does not have goals of its own, and it does not understand in the way a human does.
3. *The core mechanism is a loop: model, loss, gradient, update.* Everything from logistic regression to frontier language models is a refinement of this loop. The notebook accompanying this article builds it from scratch.
4. *Architecture is a claim about structure.* Model families encode inductive biases, and the choice among them is a statement about what patterns the data contains. No family dominates across all problems.
5. *Most failures are not model failures.* Distribution shift, data leakage, feedback loops, and unstated assumptions about the deployment environment account for more real-world failures than algorithmic limitations.
6. *The theoretical foundations are incomplete.* We do not know why deep learning generalises, whether adversarial robustness is achievable, or whether current architectures can support reliable reasoning. The gap between capability and understanding is wider here than in most engineering disciplines.
7. *The future is contingent on choices.* Capability will improve; how its benefits and risks are distributed depends on decisions about access, governance, and investment, not on the technology alone.

13.2 How to evaluate AI claims

The practical payoff of the article is a set of questions that can be applied to any claim about AI, whether it appears in a research paper, a product announcement, or a news headline.

- *What is being claimed, precisely?* “AI can do X” is almost never informative. What is the task, the conditions, the metric, and the comparison? A narrow system that performs a specific version of X under specific conditions may still be significant, but it is not a general capability.
- *What is the evidence?* A benchmark score is evidence about a distribution. A demonstration is evidence about one example. Anecdotes, cherry-picked successes, and impressive-seeming outputs are the weakest form of evidence, and they are the most common in public discourse.
- *What is the base rate?* If a model is reported to be 95% accurate, what does the trivial baseline achieve? A classifier that always predicts the majority class can achieve high accuracy on imbalanced data while being useless.
- *What happens when it is wrong?* The cost of errors is domain-dependent and matters as much as their frequency. A model that misclassifies digits can be entertaining; one that misclassifies tumours cannot.
- *Who benefits, and who bears the risk?* AI systems are deployed within institutions with incentives. The distribution of benefits and risks is not incidental. It is a design decision, whether or not it is made consciously.
- *What would change the claim?* A claim that cannot be falsified by any evidence is not an empirical claim. It may be a philosophical position, a prediction, or an advertisement. The distinction matters.

These questions are not novel. They are the standard tools of empirical reasoning, applied to a field that has acquired an unusual quantity of hype. The article’s contribution, if it has one, is to have shown that the tools apply here as they do elsewhere: AI is not exempt from ordinary standards of evidence and argument.

13.3 Call to informed participation

The trajectory of AI is not determined by the technology. It is determined by a large number of choices made by a large number of actors: researchers, engineers, executives, regulators, journalists, educators, and citizens. Some of these choices are technical, and they belong to specialists. Many are not, and they belong to everyone.

The reader who has followed this article to this point is now equipped to participate in a way that the reader of a headline is not. That does not mean becoming an AI researcher. It means being able to read a claim and ask what it is based on; to encounter a deployment and ask who is affected; to hear a forecast and distinguish argument from evidence from speculation.

The article’s wager is that a better-informed public discussion produces better decisions. This is not guaranteed. It is also not naive: the history of technology is full of decisions made well and badly, and the difference is rarely technical. It is the difference between questions asked and not asked, and between those who asked them and those who did not.

The notebook is where the understanding becomes concrete. It is also, deliberately, an invitation: the code is written to be modified, the experiments are designed to be extended, and the failures are chosen to be instructive. A reader who runs the notebook and changes something— and sees what happens—has done the thing this article is ultimately about. Not learning about AI, but understanding it well enough to form independent judgment.

That judgment is the point.

No-nonsense takeaway. AI is a statistical, narrow, and imperfect technology whose trajectory depends on choices. The reader now has a working model of how it functions, a set of questions for evaluating claims, and a concrete way—the notebook—to extend that understanding. The article’s purpose was not to produce agreement but to produce competence: the ability to think about AI clearly in a discourse that rarely rewards clarity.

Further reading for Part III

For the theoretical limits, the grokking literature is the most accessible entry point into the puzzle of why deep learning generalises; the original paper by Power et al. (2022) is short and the phenomenon is reproducible on a laptop. For interpretability, the mechanistic interpretability literature is fast-moving; Anthropic’s transformer-circuits work and the survey by Bereska and Gavves (2024) are reasonable starting points, and the “Open Problems in Mechanistic Interpretability” review is useful for seeing where the field is stuck. For the physics connection, the statistical mechanics lineage from Hopfield (1982) through Gardner (1988) is well documented, and the 2024 Nobel Prize citation provides a curated entry point. For the geometry of high dimensions, most standard ML texts cover the curse of dimensionality, but the manifold hypothesis—that natural data lies near a low-dimensional manifold embedded in a high-dimensional space—is treated well in Goodfellow et al. and in the dimension-reduction literature.

For fairness, the two papers to read are Kleinberg, Mullainathan, and Raghavan, “Inherent Trade-Offs in the Fair Determination of Risk Scores” (2016) and Chouldechova, “Fair Prediction with Disparate Impact” (2017), which independently established the impossibility result discussed in Section 11. For governance, the *Oxford Handbook of AI Governance* (2024) and the *Cambridge Handbook of the Law, Ethics and Policy of Artificial Intelligence* (2024) are both comprehensive and current. For alignment, the “AI Alignment: A Contemporary Survey” (2025) covers the technical landscape without the rhetorical excess that characterises much of the public discussion.

Part IV

The Companion Notebook

14 The Notebook

Every claim in this article is reproducible in a single Python notebook. This chapter is a guide to it. The notebook runs on a laptop CPU in under ten minutes, produces seventeen figures, and builds every model from scratch before any framework hides the mechanics. It is not optional. Sections 5, 7, and 9 all make arguments that become concrete when the code runs, and the notebook is where that happens.

The chapter does not reproduce the notebook. The notebook itself is a separate artifact, hosted alongside this article and linked at the end of this chapter. What follows is a walk through it: what each part does, what the reader should expect to see, and what the result means. A reader who works through the notebook alongside this chapter will finish with a working mental model of how machine learning functions. A reader who only reads this chapter will still get the structure of the argument, though not the intuition that comes from running code and watching it fail.

14.1 Where to find the notebook

The complete notebook is hosted at

The notebook is versioned alongside this article. If you find an error or a result that does not reproduce, please open an issue at the repository. Corrections are welcome and will be incorporated into future versions.

A final note on the notebook’s role. The article has argued throughout that modern AI is not mysterious, that its capabilities come from a small number of mechanisms applied at scale, and that the gap between capability and understanding is a gap in explanation rather than in principle. The notebook is the concrete form of that argument. A reader who has worked through it has not merely read about how machine learning works; they have built a model, trained it, watched it fail, and understood why. That is a different kind of knowledge from reading an article, and it is the kind this article has tried to make possible.

14.2 Setup and how to run it

The notebook is a single file in Jupyter text percent format, which means it can be opened directly in VS Code as a notebook, converted to `.ipynb` with `jupyter text --to ipynb`, or uploaded to Google Colab. It has four dependencies: NumPy, Matplotlib, PyTorch, and scikit-learn. All four are pre-installed in Colab. The MNIST dataset and a 200 KB excerpt of Shakespeare’s plays download on first run and cache locally.

Total runtime is approximately eight minutes on a free Colab CPU. The breakdown is roughly: thirty seconds for the scikit-learn baseline, two minutes for the from-scratch MLP, three minutes for the PyTorch models, two minutes for the character-level language model, and the remainder for the smaller experiments. Seeds are fixed at the points where they matter, so the figures are reproducible across runs. The outputs will not be bit-identical across different hardware, but the qualitative results — the shapes of the curves, the failures, the clusters — will be.

A note on reading. The notebook is designed to be modified. Every experiment is stated in a form that invites the reader to change one variable and observe the effect. That discipline — one change at a time — is the same one that Section 8 describes for professional machine learning practice. Readers who only execute the cells in order will learn something; readers who change parameters and re-run will learn considerably more.

14.3 The core loop from scratch

Part 3 of the notebook is the centerpiece. It implements logistic regression on MNIST in twenty lines of NumPy, and it is the part of the notebook that matters most. If a reader works through only one section of the code, this should be it.

The model is $\hat{p} = \text{softmax}(Wx+b)$, where W is a 10×784 weight matrix and b is a 10-dimensional bias. The loss is cross-entropy. The gradient with respect to the logits has the remarkably simple form $\hat{p} - \text{onehot}(y)$, which the notebook uses directly. Training is a loop of forward pass, loss, gradient, update, exactly as described in Section 5.

Three things happen when this code runs.

First, the loss curve falls smoothly, and after five hundred steps the model reaches roughly 92% test accuracy. This is not a state-of-the-art number. It is a baseline, and it establishes what a linear model achieves on this task. Everything in the rest of the notebook is measured against it.

Second, and more importantly, the learned weights are interpretable. Figure 16 shows the ten columns of W reshaped as 28×28 images. Each column is a template: red pixels push the input toward that class, blue pixels push away. The “0” template looks roughly like a 0, the “1”

template looks roughly like a 1. This is what learning produced. Gradient descent started from random weights and nudged 7,850 numbers until the templates classified digits well. There is no rule anywhere in the code about which pixels matter. The rules emerged from the data.

Third, the notebook makes the mechanism explicit. The reader can see the loss, the gradient, and the update rule, and can connect each to the mathematics in Section 4. When the notebook later switches to PyTorch and the same model is expressed in ten lines of framework code, the reader knows what those ten lines are doing. This is the payoff of the from-scratch approach: the framework becomes a convenience, not a mystery.

14.4 What each experiment demonstrates

Parts 4 through 10 of the notebook are a sequence of experiments. Each one changes a single variable. Each one produces a plot. Each one teaches a specific lesson from the main text. The order is deliberate: the reader breaks the model in controlled ways before seeing it succeed on harder tasks.

Learning-rate sweep. Section 4 called the learning rate the single most important hyperparameter. The notebook demonstrates this by training the same model with $\eta \in \{0.001, 0.5, 50\}$. The results are distinctly different: with 0.001 the loss barely moves; with 0.5 it converges smoothly; with 50 it oscillates and diverges. Three curves, one changed parameter, and the reader has seen the difference between slow convergence and instability.

Normalization. The same model, the same learning rate, the same number of steps, but inputs scaled to $[0, 255]$ instead of $[0, 1]$. Training stalls. This is counterintuitive on first encounter: scaling the input should not matter, since a linear model could in principle compensate by scaling its weights. In practice it does matter, because gradient descent operates on an ill-conditioned loss surface when input features have very different scales. Normalization is not a detail; it is the difference between working and not working.

Too few steps, too many. The notebook trains the same model for 10, 100, and 2000 steps and plots the three loss curves on the same axes. The 10-step curve has not begun to converge; the 100-step curve is on its way; the 2000-step curve is converged. This is the simplest form of the underfitting discussion in Section 5.

Why nonlinearity matters. The from-scratch model is extended with one hidden layer. Accuracy jumps from 92% to about 97%. Then the activation function is removed, and the network collapses back to the linear baseline. The notebook plots both loss curves. The lesson, which cannot be overemphasized, is that without nonlinearity, depth is an illusion: a composition of linear functions is itself linear. The two-layer network without an activation is mathematically identical to the single-layer logistic regression, regardless of how many hidden units it has.

PyTorch and the CNN. The same MLP is reimplemented in PyTorch, then replaced with a small convolutional network. The CNN reaches roughly 99% test accuracy, and the notebook shows the same loss curves for both models on the same axes. The comparison is not about which framework is better; it is about how much the inductive bias of convolution buys, and it connects directly to the model-family discussion in Section 6.

What the network learned. Two visualizations. The first (Figure 19) shows the first-layer convolutional filters as 3×3 images; they are edge detectors, which is what the inductive bias predicts. The second (Figure 21) takes the penultimate-layer activations for 2000 test digits, projects them to two dimensions with PCA, and colors them by digit label. The digits cluster by class. No supervision was applied at the clustering level — the model was only told which digit each image represents — and yet the geometry of the representation reflects the classes. This is what representation learning means, and it is the visual payoff of the embeddings discussion in Section 7.

Where it fails. Three visualizations of failure. The confusion matrix shows the 4/9 confusion as the largest off-diagonal entry. The error gallery shows the twenty most confidently wrong predictions, with the true label, the predicted label, and the model’s confidence. The calibration plot compares predicted confidence to actual accuracy and shows that the model is overconfident. Each of these is discussed in Section 5, and each is more instructive than an aggregate accuracy number.

Distribution shift. Three experiments on inputs the model has not seen. Rotate the test digits by 30 degrees: accuracy collapses from 99% to roughly 70%, even though a human reads the rotated digits without effort. Add Gaussian noise at increasing standard deviations: accuracy degrades smoothly, then abruptly. Occlude the center of the image: the model fails on digits a human reads instantly. These are three different forms of the same phenomenon, and together they show why benchmark performance does not predict deployment performance.

Adversarial examples. The FGSM demonstration in Figure 25. A digit is perturbed by an amount that is invisible at normal viewing, and the model confidently misclassifies it. The perturbation is not random; it is computed as the sign of the gradient of the loss with respect to the input, which is the cheapest possible adversarial attack. The demonstration takes ten lines of PyTorch. This is the most consequential experiment in the notebook, because it shows that the model’s decision boundary is not aligned with the human concept it appears to represent. The model is correct on easy cases for the wrong reasons, and we can prove it by manipulating the reasons.

Overfitting lab. Train on 100 examples and watch training and validation loss diverge. Then add weight decay and repeat. Weight decay delays but does not eliminate the divergence. This is the classical picture of overfitting from Section 5, and it is worth seeing once, even though the picture is incomplete for modern deep networks, as Section 9 explains.

Generation. A tiny autoencoder, briefly. The point is not sample quality; it is that the same machinery that classifies can also generate. The model learns a 64-dimensional latent representation of digits, samples from it, and decodes the samples back into images. They look like digits. This is a small preview of the generative models discussed in Section 6, and it makes the generative-model discussion less abstract.

14.5 The character-level language model

Part 12 of the notebook is the bridge from images to text. It is deliberately small: a character-level MLP with a context window of five characters, trained on 200 KB of Shakespeare for three epochs. The training loop is identical to the digit classifier. The loss is the same cross-entropy. The optimizer is the same Adam. Only the representation of the data has changed.

The results are instructive. After a few minutes of training, the model produces text that is recognisably English-shaped but semantically empty:

```
ROMEO: the have so my lord to the  
To be or not the have so my lord
```

The local structure is right: word boundaries, common letter sequences, even the shape of a Shakespearean line. The global structure is absent: nothing in the passage means anything. The model has learned the statistics of English at a five-character scale and nothing beyond it.

This gap between local fluency and global meaning is the pedagogical point of the section. It is also, in miniature, the gap that large language models are often said to have closed and, on closer inspection, may not have closed entirely. Section 9 discusses the ambiguity at length. The notebook makes it visible.

The section is also the reader's first encounter with sequence data. Images have a fixed size; text does not. The context window is a representation choice, and it is a choice with consequences: a five-character window cannot capture dependencies longer than five characters, and the model's output reflects that limit. A reader who increases the window to twenty and re-trains will see the output improve; a reader who reduces it to one will see it collapse. These are instructive experiments in the same spirit as the learning-rate sweep, and they connect to the tokenization and context-window discussion in Section 7.

14.6 Exercises and extensions

The notebook is written to be extended. The exercises in Appendix C provide a starting point, and the following projects are worth considering for a reader who wants to go further.

Reproduce the classical U-curve. Train on a fixed small sample and increase model capacity. Does the classical bias-variance tradeoff appear? At what capacity does validation error start to rise? Now train on the full dataset and repeat. The curve will look different. This is the double-descent phenomenon discussed in Section 9, and it is reproducible with about fifteen lines of modification.

Visualize intermediate representations. For a single input digit, plot the activations of each convolutional layer as the input passes through the network. The features become progressively more abstract: edges, then strokes, then partial digits, then the class prediction. This is the most direct way to see what a deep network does, and it takes one forward hook.

Train on your own handwriting. Export a few dozen images of your own digits and fine-tune the classifier. The accuracy will drop sharply. This is distribution shift, personalized, and it is one of the most memorable experiments a reader can do.

Fine-tune a pretrained model. Section 7 describes the pretrain-fine-tune-align pipeline for language models. The notebook does not implement it, because it is expensive. But a reader who wants to see the pipeline in action can load a small pretrained model from HuggingFace and fine-tune it on a small classification task in about twenty lines. This is a natural extension of the character-level section, and it shows what the framework ecosystem provides that the from-scratch approach does not.

Break the notebook deliberately. The most useful exercise is to find something the notebook does that you do not understand, and change it until you do. Remove the softmax and replace it with a linear output. Use mean squared error instead of cross-entropy. Shuffle the labels. Each of these changes is a small experiment, and each produces a result that is instructive in a different way.

No-nonsense takeaway. The companion notebook reproduces every experiment in the article, from a from-scratch logistic regression in NumPy to a CNN, an autoencoder, and a character-level language model. It runs in under ten minutes on a laptop CPU. The from-scratch section is the centerpiece; the failure modes are the most instructive. The notebook is not a supplement to the article. It is where much of the understanding is built.

A Mathematical Primer

This appendix collects the mathematical notation and results used in the main text. It is a reference, not a tutorial. Readers who want a fuller treatment should consult Deisenroth, Faisal, and Ong (2020), cited in Appendix E.

A.1 Notation

Throughout the article:

- Scalars are lowercase italics: x , y , η .
- Vectors are bold lowercase: $\mathbf{x}$, $\mathbf{w}$.
- Matrices are bold uppercase: $\mathbf{W}$, $\mathbf{X}$.
- The i -th component of $\mathbf{x}$ is x_i ; the (i, j) -th entry of $\mathbf{W}$ is W_{ij} .
- $\mathbf{x}^\top$ denotes the transpose of $\mathbf{x}$.
- $\|\mathbf{x}\|$ denotes the Euclidean norm $\sqrt{\sum_i x_i^2}$.
- $\nabla_\theta L$ denotes the gradient of L with respect to the parameters θ .
- $\mathbb{E}[X]$ and $\text{Var}[X]$ denote expectation and variance of a random variable X .

A.2 Linear algebra

Vectors. A vector $\mathbf{x} \in \mathbb{R}^n$ is an ordered list of n real numbers. The *inner product* of two vectors is

$$\mathbf{x}^\top \mathbf{y} = \sum_{i=1}^n x_i y_i.$$

It measures similarity: $\mathbf{x}^\top \mathbf{y} = 0$ when the vectors are orthogonal, and it is maximised when they point in the same direction.

Matrices. A matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$ has m rows and n columns. The product $\mathbf{AB}$ is defined when the number of columns of $\mathbf{A}$ equals the number of rows of $\mathbf{B}$, and its (i, j) -th entry is

$$(\mathbf{AB})_{ij} = \sum_{k=1}^n A_{ik} B_{kj}.$$

Matrix multiplication is associative but not commutative: $\mathbf{AB} \neq \mathbf{BA}$ in general.

Matrix as a linear map. A matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$ defines a function from $\mathbb{R}^n$ to $\mathbb{R}^m$ by $\mathbf{x} \mapsto \mathbf{Ax}$. The composition of two linear maps is the product of their matrices: applying $\mathbf{B}$ then $\mathbf{A}$ is the same as applying $\mathbf{AB}$. This is the reason deep networks are called compositions of layers.

A.3 Calculus and optimisation

Derivatives. The derivative of a scalar function $f : \mathbb{R} \rightarrow \mathbb{R}$ at a point x is

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}.$$

It measures the rate of change of f at x .

Partial derivatives and gradients. For a function $f : \mathbb{R}^n \rightarrow \mathbb{R}$, the partial derivative $\partial f / \partial x_i$ is the derivative with respect to x_i holding the other variables fixed. The gradient is the vector of partial derivatives:

$$\nabla f(\mathbf{x}) = \left(\frac{\partial f}{\partial x_1}, \dots, \frac{\partial f}{\partial x_n} \right)^\top.$$

It points in the direction of steepest ascent.

Chain rule. If $y = g(u)$ and $u = h(x)$, then

$$\frac{dy}{dx} = \frac{dy}{du} \cdot \frac{du}{dx}.$$

In the multivariate case, if $\mathbf{u} = h(\mathbf{x})$ and $y = g(\mathbf{u})$, then

$$\frac{\partial y}{\partial x_i} = \sum_j \frac{\partial y}{\partial u_j} \frac{\partial u_j}{\partial x_i}.$$

Backpropagation is this identity applied systematically through the layers of a network.

Gradient descent. To minimise a differentiable function $L(\theta)$ over parameters θ , the update rule is

$$\theta \leftarrow \theta - \eta \nabla_\theta L(\theta),$$

where $\eta > 0$ is the learning rate. Stochastic gradient descent uses an estimate of the gradient computed on a small batch of examples rather than the full dataset.

A.4 Probability

Random variables and distributions. A random variable X is a quantity whose value depends on chance. A discrete random variable is described by a probability mass function $p(x) = P(X = x)$, with $p(x) \geq 0$ and $\sum_x p(x) = 1$. A continuous random variable is described by a probability density function $p(x)$, with $p(x) \geq 0$ and $\int p(x) dx = 1$.

Expectation and variance. For a random variable X ,

$$\mathbb{E}[X] = \sum_x x p(x) \quad \text{or} \quad \mathbb{E}[X] = \int x p(x) dx,$$

and $\text{Var}[X] = \mathbb{E}[(X - \mathbb{E}[X])^2]$. The standard deviation is the square root of the variance.

Conditional probability and Bayes' rule. The conditional probability of A given B is

$$P(A | B) = \frac{P(A \cap B)}{P(B)},$$

provided $P(B) > 0$. Bayes' rule follows:

$$P(A | B) = \frac{P(B | A) P(A)}{P(B)}.$$

It is used throughout machine learning for updating beliefs in light of evidence.

Common distributions. The *categorical* distribution places probability over a finite set of outcomes, such as the ten digit classes. The *Gaussian* (normal) distribution over $x \in \mathbb{R}$ has density

$$p(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x - \mu)^2}{2\sigma^2}\right),$$

where μ is the mean and σ^2 the variance. The multivariate Gaussian generalises this to vectors.

A.5 Information theory

Entropy. For a discrete distribution p over outcomes x ,

$$H(p) = - \sum_x p(x) \log p(x).$$

It measures the average surprise of the distribution and is the minimum number of bits per symbol required to encode samples from p .

Cross-entropy. For two distributions p and q ,

$$H(p, q) = - \sum_x p(x) \log q(x).$$

It measures the cost of encoding samples from p using a code optimised for q . Minimising cross-entropy is the standard training objective for classification.

KL divergence. The Kullback–Leibler divergence is

$$D_{\text{KL}}(p \parallel q) = H(p, q) - H(p) = \sum_x p(x) \log \frac{p(x)}{q(x)}.$$

It is non-negative and zero if and only if $p = q$. It is not symmetric, so it is a divergence rather than a distance.

A.6 Useful identities

For reference:

- Softmax: for a vector $\mathbf{z} \in \mathbb{R}^K$,

$$\text{softmax}(\mathbf{z})_i = \frac{\exp(z_i)}{\sum_{j=1}^K \exp(z_j)}.$$

The output is a probability distribution over K classes.

- Sigmoid: $\sigma(z) = 1/(1 + e^{-z})$. Maps $\mathbb{R}$ to $(0, 1)$.
- ReLU: $\text{ReLU}(z) = \max(0, z)$. The standard activation function in deep networks.
- Derivative of sigmoid: $\sigma'(z) = \sigma(z)(1 - \sigma(z))$.
- Derivative of ReLU: 1 for $z > 0$, 0 for $z < 0$.

B Glossary

Terms are defined as they are used in this article. Definitions are brief; fuller treatments are in the main text at the locations indicated.

Activation function A nonlinear function applied elementwise to the output of a neural network layer. Common choices are ReLU, sigmoid, and tanh. Without nonlinearity, a deep network collapses to a linear model. See Section 6.

Adversarial example An input deliberately perturbed to cause a model to misclassify it, often by an amount imperceptible to a human. See Section 9.

AI (artificial intelligence) The broadest term for computational systems that perform tasks associated with intelligent behaviour. See Section 2.

AGI (artificial general intelligence) A hypothetical system capable of performing any intellectual task a human can. Does not currently exist. See Section 12.

Attention A mechanism that computes a weighted combination of information from different positions in a sequence, with weights determined by learned similarity. The core operation of the transformer. See Section 7.

Backpropagation The algorithm for computing gradients of a loss with respect to all parameters in a network, by applying the chain rule from output to input. See Section 7.

Batch (minibatch) A small subset of the training data used to estimate the gradient in one step of stochastic gradient descent.

Bias (statistical) A systematic error in a model’s predictions, independent of the particular training sample. See Section 5.

Bias (ethical) A systematic disadvantage to a group produced by a model or system. The two senses of “bias” are distinct and should not be confused. See Section 11.

Calibration The property that a model’s predicted probabilities match observed frequencies: among predictions made with probability p , approximately p should be correct. See Section 5.

CNN (convolutional neural network) A neural network that applies learned filters across the input, encoding translation equivariance. Suited to images and other data with local structure. See Section 6.

Chain-of-thought prompting A prompting technique that asks a language model to produce intermediate reasoning steps before its final answer. See Section 7.

Cross-entropy A loss function measuring the cost of encoding samples from a true distribution using a predicted distribution. Standard for classification. See Appendix A.

Diffusion model A generative model that learns to reverse a gradual noising process. Underlies most current image-generation systems. See Section 6.

Distribution shift A difference between the distribution of data the model was trained on and the distribution it encounters in deployment. The most common cause of real-world model failure. See Sections 5 and 8.

Double descent The empirical observation that test error can decrease again past the point of interpolation, contrary to classical theory. See Section 9.

Dropout A regularization technique that randomly disables units during training. See Section 5.

Embedding A learned map from discrete objects to vectors in a continuous space, chosen so that similar objects are close together. See Sections 4 and 7.

Epoch One pass through the training dataset.

FGSM (Fast Gradient Sign Method) A method for generating adversarial examples by perturbing the input in the direction of the gradient of the loss. See Section 9.

Fine-tuning Further training of a pretrained model on a smaller, task-specific dataset. See Section 7.

Foundation model A large model pretrained on broad data, intended to be adapted to many downstream tasks. See Section 7.

GAN (generative adversarial network) A generative model trained by pitting a generator against a discriminator. See Section 6.

Gradient The vector of partial derivatives of a function. Points in the direction of steepest ascent. See Appendix A.

Gradient descent An iterative optimisation algorithm that updates parameters in the direction of negative gradient. See Section 5.

Grokking The empirical phenomenon in which a model appears to memorise its training data and then, after many more steps, transitions to a solution that generalises. See Section 9.

Hallucination A confident but incorrect output from a generative model, particularly a language model. The term is metaphorical; the model has no experience of perceiving anything. See Section 9.

Inductive bias The assumptions a learning algorithm makes about the structure of the problem, prior to seeing data. Architecture choice is inductive bias selection. See Section 4.

Inference The process of using a trained model to produce predictions. Also used in statistics for the process of drawing conclusions from data.

Interpretability The degree to which a model’s internal computations can be understood by humans. See Section 9.

Kernel trick A method for computing inner products in a high-dimensional feature space without explicitly constructing that space. See Section 6.

Learning rate The step size η in gradient descent. The single most important hyperparameter. See Section 4.

LLM (large language model) An autoregressive model trained to predict the next token in a sequence, typically with billions of parameters. See Section 7.

Loss function A scalar summary of how badly a model is performing, which training seeks to minimise. See Section 5.

ML (machine learning) A subset of AI in which systems learn patterns from data rather than being explicitly programmed with rules. See Section 2.

MLOps Engineering practices for deploying, monitoring, and maintaining machine learning systems in production. See Section 8.

MNIST A dataset of handwritten digits, widely used as a benchmark and as a teaching example. The companion notebook uses it.

Overfitting The tendency of a model to memorise its training data rather than learn generalisable patterns. See Section 5.

Parameter A number in a model that is adjusted during training. Distinct from a hyperparameter, which is set before training.

Precision and recall Precision is the fraction of positive predictions that are correct; recall is the fraction of actual positives that are correctly identified. See Section 5.

Pretraining The initial stage of training a foundation model on a large, broad dataset. See Section 7.

Prompt The input text given to a language model.

RAG (retrieval-augmented generation) A technique that retrieves relevant documents and inserts them into a model’s prompt to ground its output in external information. See Section 7.

Reinforcement learning (RL) A paradigm in which an agent learns by interacting with an environment and receiving rewards. See Section 5.

RLHF (reinforcement learning from human feedback) A method for aligning language models with human preferences, using a reward model trained on human comparisons. See Section 7.

Self-attention The application of attention within a single sequence, where queries, keys, and values are all derived from the same input.

Self-supervised learning A learning paradigm in which labels are derived from the data itself, typically by predicting one part of the input from another. The dominant paradigm for pretraining large models. See Section 5.

Softmax A function that converts a vector of real numbers into a probability distribution. See Appendix A.

Supervised learning A learning paradigm in which the training data contains labelled examples. See Section 5.

Transformer A neural network architecture based on self-attention, introduced in 2017. The substrate of every current frontier model. See Sections 6 and 7.

Underfitting The opposite of overfitting: the model is too simple or trained for too few steps to capture the structure of the data. See Section 5.

Unsupervised learning A learning paradigm in which no labels are provided and the goal is to discover structure. See Section 5.

VAE (variational autoencoder) A generative model that learns a latent variable representation. See Section 6.

Validation set A held-out subset of the data used to tune hyperparameters. Distinct from the test set, which is used once for final evaluation. See Section 5.

Weight decay A regularization technique that penalises large parameters. See Section 5.

C Exercises and Discussion Questions

The questions below are grouped by part. Those marked **[notebook]** require running or modifying the companion notebook; those marked **[discussion]** have no single correct answer and are intended for group discussion.

C.1 Part I: Foundations

1. Define AI, machine learning, deep learning, and generative AI, and give one example of each that is not discussed in the text. **[discussion]**
2. The Dartmouth workshop proposal estimated that ten researchers could make significant progress on machine intelligence in two months. What assumptions did this estimate rest on, and which of them proved wrong? **[discussion]**
3. In the notebook, train logistic regression on MNIST with the pixels normalised to $[0, 1]$ and without normalisation. Report the test accuracy in both cases and explain the difference. **[notebook]**
4. Explain in your own words why a linear model cannot represent the XOR function. Draw the decision boundary.

5. The no-free-lunch theorem states that no learning algorithm is uniformly better than any other. Why, then, does machine learning work in practice? **[discussion]**
6. Compute the entropy of a fair coin and of a coin that lands heads with probability 0.99. Interpret the difference.
7. Find a current news article about AI and identify which of its claims are empirical, which are forecasts, and which are normative. **[discussion]**

C.2 Part II: Mechanics and Practice

1. Implement softmax in NumPy and verify numerically that its outputs sum to 1 for a random input vector. **[notebook]**
2. The notebook implements backpropagation by hand for a two-layer network. Compare the gradients it computes to those produced by PyTorch’s automatic differentiation on the same inputs. Do they agree to within numerical precision? **[notebook]**
3. Sweep the learning rate over at least five values spanning several orders of magnitude. Plot training loss against step for each. Identify the regimes of slow convergence, stable convergence, and divergence. **[notebook]**
4. Remove the nonlinear activation from the two-layer MLP. What happens to test accuracy? Explain why. **[notebook]**
5. Train the classifier on 100 examples and plot training and validation loss on the same axes. At what step does the gap begin to widen? Now add weight decay and repeat. Does it delay the divergence? **[notebook]**
6. Explain why gradient-boosted trees often outperform neural networks on tabular data. What does this tell you about inductive bias?
7. Choose a model family from Table 1 and describe a problem for which its inductive bias is clearly appropriate and a problem for which it is clearly inappropriate. **[discussion]**
8. In the notebook’s error gallery, examine the digits the model is most confidently wrong about. Are any of them genuinely ambiguous? What does this suggest about the ceiling on achievable accuracy? **[notebook]**

C.3 Part III: Limits, Context, and Future

1. Rotate the MNIST test images by 30 degrees and evaluate the classifier without retraining. Report the accuracy drop. Now retrain on rotated images and test on unrotated ones. What does this tell you about distribution shift? **[notebook]**
2. Generate an adversarial example using FGSM with increasing perturbation magnitude ϵ . At what value of ϵ does the model first misclassify the digit? Can you see the difference? **[notebook]**
3. The fairness impossibility result says that calibration, equalised odds, and demographic parity cannot all hold when base rates differ. Construct a small numerical example with two groups and two outcomes that illustrates this. **[notebook]**

4. Section 9 lists seven open problems. Choose one and write a paragraph on what evidence would convince you it had been solved.
5. A hospital wants to deploy a model that predicts sepsis. What questions should be asked before deployment, and which of them are technical rather than institutional? [**discussion**]
6. Claims about catastrophic AI risk are often made confidently and with little evidence. What would a rigorous version of the argument look like? What would count as evidence for or against it? [**discussion**]
7. Consider the three scenarios in Section 12. Which do you consider most likely, and what would have to be true for a different one to unfold? [**discussion**]
8. Identify one AI system you interact with regularly. Who benefits from it, who bears the risk, and what would you need to know to evaluate whether it should exist in its current form? [**discussion**]

D Notes on Sources and Evidence Quality

This article draws on sources of very different kinds, and it makes claims of very different strengths. This appendix explains how those differences are handled and provides guidance for readers who want to evaluate AI claims encountered elsewhere.

D.1 Three kinds of claim

Throughout the article, claims are of three broadly different kinds, and the reader should be able to tell which is which.

Established results. Theorems, empirical findings that have been replicated, and documented facts. Examples: the fairness impossibility results (Section 11), the Chinchilla scaling findings (Section 7), the adversarial example phenomenon (Section 9). These claims can be verified by consulting the cited sources.

Engineering practice. What works in practice, as distinct from what theory predicts. Examples: the choice of Adam as a default optimiser, the use of dropout for regularization, the preference for gradient-boosted trees on tabular data (Section 6). These claims are empirical but not universal: what works depends on the setting, and the field’s conventions shift.

Speculation. Claims about futures that have not materialised or about phenomena that are not well understood. Examples: AGI timelines (Section 12), the interpretation of criticality in trained networks (Section 10), specific mechanisms of catastrophic risk (Section 11). These are legitimate subjects of research and debate, but they should not be treated as findings. Where the article uses them, it says so.

The distinction is not always clean. Some claims sit at the boundary: scaling laws are established within a range of scales, but their extrapolation to much larger scales is speculation. Where the boundary matters, the article tries to be explicit.

D.2 What this article is not

The article is not a survey of the literature, and it does not attempt to be comprehensive. Many important results and perspectives are not mentioned. In particular:

- It does not cover reinforcement learning in depth, despite its importance to the field.
- It does not discuss multimodal models, despite their growing prominence.
- It does not cover federated learning, differential privacy, or the technical side of privacy-preserving ML in any detail.
- It does not attempt to adjudicate the disputes it describes (about AGI timelines, about fairness definitions, about catastrophic risk). It clarifies the terrain.

The article also does not claim that its treatment of any topic is complete. It is an introduction, and introductions select.

D.3 Evaluating sources

Readers who want to go beyond this article will encounter a wide range of sources, from peer-reviewed papers to blog posts to social media. The following questions are useful for assessing any of them.

Is the source accountable? A claim published under a named author’s or institution’s name, with the possibility of correction, is more reliable than an anonymous claim circulated without a traceable origin.

Is the claim specific? ”AI can diagnose disease” is not a claim that can be evaluated. ”A model trained on this dataset achieved this accuracy on this benchmark, under these conditions” can be.

Is the evidence proportionate to the claim? A single demonstration supports a claim about a single case. A benchmark score supports a claim about the benchmark. Neither supports a general claim about capability. The gap between what evidence establishes and what is claimed is the most common weakness in AI discourse.

Is the base rate acknowledged? A result is only meaningful against a baseline. A reported improvement of a few percentage points may be substantial or negligible depending on what the previous state of the art was and how the metric is defined.

Is the comparison fair? Comparisons between models are often made under conditions that favour one over the other: different training data, different compute, different evaluation. Fair comparison requires matching the conditions.

Who benefits from the claim? This is not a reason to dismiss a claim; it is a reason to examine it. A company announcing that its model is state-of-the-art has an interest in the claim being believed, and the claim may still be true. The relevant question is whether the evidence would be equally convincing if it came from a neutral source.

D.4 A note on citations

Full citations for works referenced in the main text are collected in Appendix E. Where the article refers to a specific empirical finding—the COMPAS case, the Chinchilla result, the AlphaTensor improvement—the original source is cited there. Where it refers to general practice or widely known results, the citation is to a representative source rather than an exhaustive one.

Readers who want to check a specific claim should consult the original source rather than relying on this article’s summary. Summaries lose detail, and the detail is often where the interesting qualifications live.

D.5 A final caution

The field is moving quickly. Specific results cited in this article will be superseded, benchmark records will be broken, and current best practice will look dated within a few years. The framework – how to think about what these systems are, what they can do, and how to evaluate claims about them—is more durable. That framework is what the article has tried to provide.

E Annotated Bibliography

This appendix collects the sources referenced throughout the article, with brief annotations describing each one’s scope and intended audience. It is organized by theme rather than by section, since many sources span multiple parts of the article.

General introductions and textbooks

Russell, S., & Norvig, P. (2024). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson. The standard textbook. Comprehensive, careful, and covers both symbolic and statistical traditions. Best used as a reference rather than read cover to cover. The 4th edition adds substantial material on machine learning and deep learning.

Nilsson, N. J. (2009). *The Quest for Artificial Intelligence: A History of Ideas and Achievements*. Cambridge University Press. The definitive history of the field, written by a participant. Covers the Dartmouth workshop, the AI winters, expert systems, and the statistical turn with clarity and good judgment. Pairs well with Section 3.

Mathematical foundations

Deisenroth, M. P., Faisal, A. A., & Ong, C. S. (2020). *Mathematics for Machine Learning*. Cambridge University Press. Freely available at mml-book.github.io. Covers linear algebra, probability, optimisation, and the specific mathematical tools used in ML. Assumes undergraduate preparation and assumes no ML background. The best single resource for a reader who wants the mathematics behind Section 4.

Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press. Freely available at deeplearningbook.org. The definitive technical reference for deep learning. Predates transformers, so it is strongest on fundamentals (backpropagation, regularization, optimisation) and weaker on the current era. Chapter 5 (machine learning basics) and Chapter 6 (deep feedforward networks) are the most relevant for this article’s scope.

Machine learning and deep learning

Karpathy, A. (2019). “A Recipe for Training Neural Networks.” Blog post, karpathy.github.io. Short, opinionated, and accurate. Describes the practical discipline of training a network—start simple, verify each component, debug systematically—in a way that no textbook does. The best single piece for a reader who wants to understand how a practitioner thinks.

scikit-learn developers. *scikit-learn User Guide*. scikit-learn.org. Well-written, example-driven documentation for the classical ML algorithms discussed in Sections 5 and 6. Assumes no prior framework experience.

Large language models and transformers

Vaswani, A., et al. (2017). “Attention Is All You Need.” *Advances in Neural Information Processing Systems*. The paper that introduced the transformer. Unusually readable for a paper of its influence. Sections 3.2 (scaled dot-product attention) and 3.3 (multi-head attention) correspond directly to Section 7 of this article.

Hoffmann, J., et al. (2022). “Training Compute-Optimal Large Language Models.” arXiv:2203.15556. The Chinchilla paper. Showed that for a given compute budget, training a smaller model on more data outperforms training a larger model on less. This finding reshaped practice and is discussed in Section 7.

Karpathy, A. (2022–2023). *nanoGPT*. github.com/karpathy/nanoGPT. A minimal GPT implementation in roughly 300 lines. Reproduces GPT-2 on OpenWebText. The best way to see the transformer architecture in code without the framework overhead of a production codebase.

History and philosophy of AI

Turing, A. M. (1950). “Computing Machinery and Intelligence.” *Mind*, 59(236), 433–460. The founding document of the philosophical discussion. Short, clearly written, and still worth reading in full. The Turing test is introduced here, along with Turing’s responses to the standard objections.

Limits, interpretability, and safety

Power, A., Burda, Y., Edwards, H., Babuschkin, I., & Misra, V. (2022). “Grokking: Generalization Beyond Overfitting on Small Algorithmic Datasets.” arXiv:2201.02177. The original grokking paper. Short, reproducible, and puzzling. The phenomenon—a sudden transition from memorisation to generalisation after thousands of steps—is discussed in Section 9.

Bereska, L., & Gavves, E. (2024). “Mechanistic Interpretability for AI Safety—A Review.” arXiv:2404.14082. A current survey of the mechanistic interpretability field, covering circuits, features, and the open problems. The relevant background for Section 9.

Amodei, D., et al. (2016). “Concrete Problems in AI Safety.” arXiv:1606.06565. An early and influential framing of safety problems—side effects, reward hacking, scalable oversight—that remains a useful vocabulary. Accessible to non-specialists.

Ethics, governance, and policy

Kleinberg, J., Mullainathan, S., & Raghavan, M. (2016). “Inherent Trade-Offs in the Fair Determination of Risk Scores.” arXiv:1609.05807. Establishes the impossibility result discussed in Section 11: calibration, equalised odds, and demographic parity cannot be simultaneously satisfied when base rates differ across groups.

Chouldechova, A. (2017). “Fair Prediction with Disparate Impact: A Study of Bias in Recidivism Prediction Instruments.” *Big Data*, 5(2), 153–163. Independently derived the same impossibility result, with the COMPAS case as its motivation. Should be read alongside Kleinberg et al.

Bullock, J. B., et al. (Eds.) (2024). *The Oxford Handbook of AI Governance*. Oxford University Press. Comprehensive and current. The chapters on regulatory approaches and international coordination are directly relevant to Section 11.

Smuha, N. A. (Ed.) (2024). *The Cambridge Handbook of the Law, Ethics and Policy of Artificial Intelligence*. Cambridge University Press. Complements the Oxford volume with more legal and policy detail, including sector-specific chapters on law enforcement, armed conflict, and public administration.

Practical resources and courses

Sculley, D., et al. (2015). “Hidden Technical Debt in Machine Learning Systems.” *Advances in Neural Information Processing Systems*. The most cited paper on the gap between ML prototypes and deployed systems. Its central diagram—a tiny box labelled “ML code” surrounded by much larger boxes for data collection, feature extraction, configuration, serving, and monitoring—is the canonical illustration of Section 8’s thesis.

Howard, J., & Gugger, S. *fast.ai: Practical Deep Learning for Coders*. course.fast.ai. A free course that teaches deep learning top-down: build working models first, understand the theory later. A useful complement to the bottom-up approach of this article’s notebook.